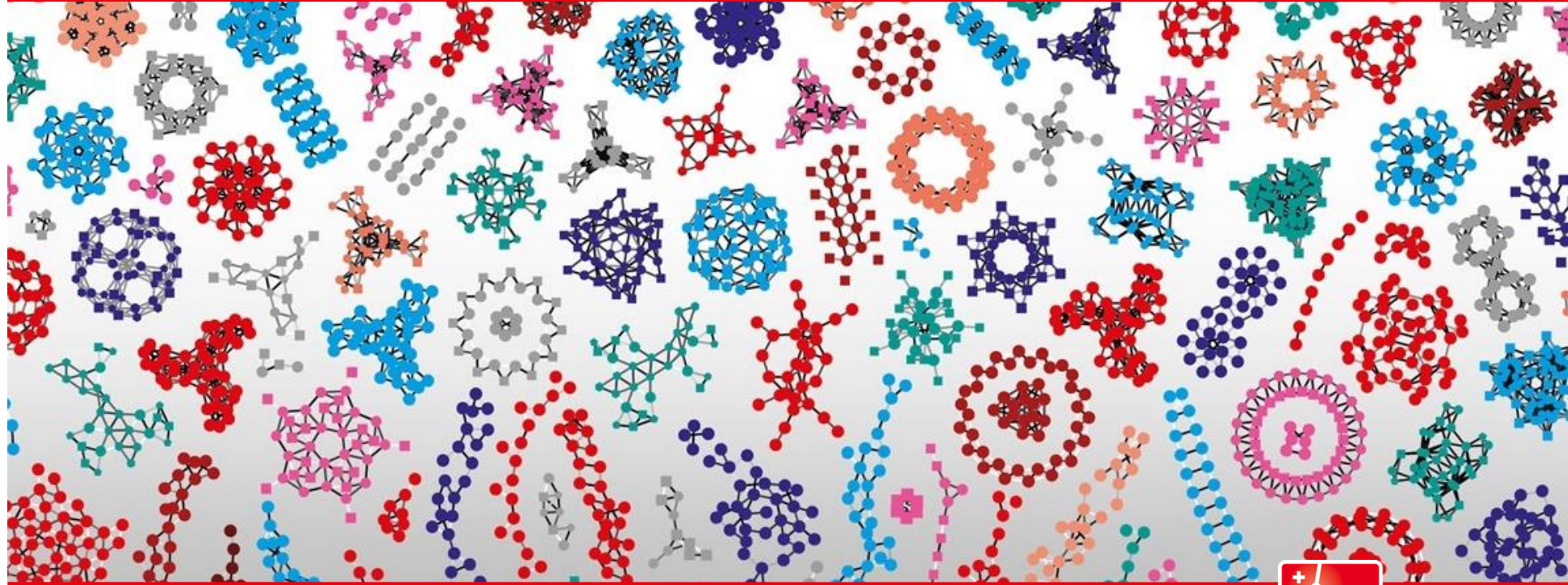




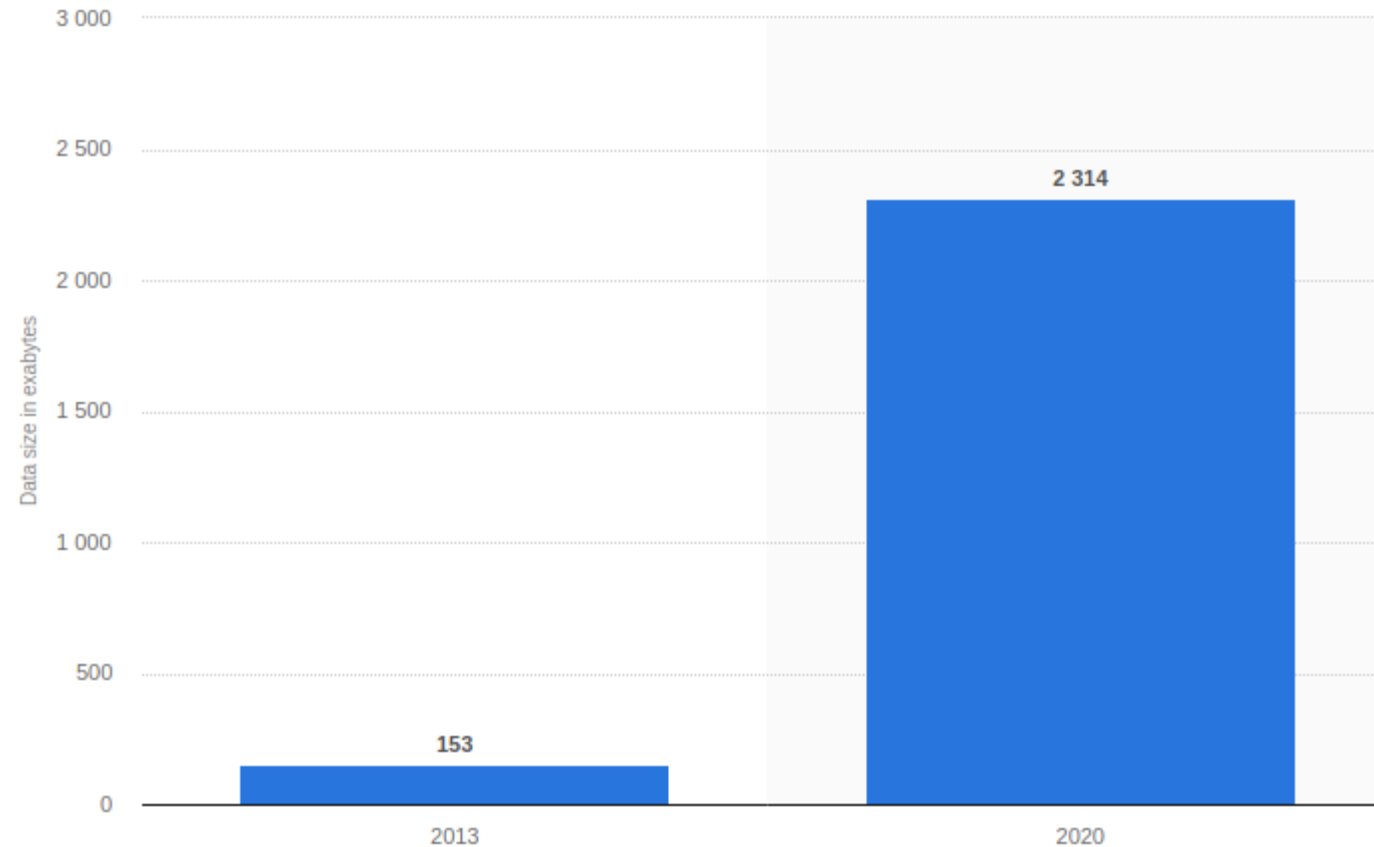
Swiss Institute of
Bioinformatics

SIB Swiss Institute of Bioinformatics

Introduction to Machine Learning with Python, 2026

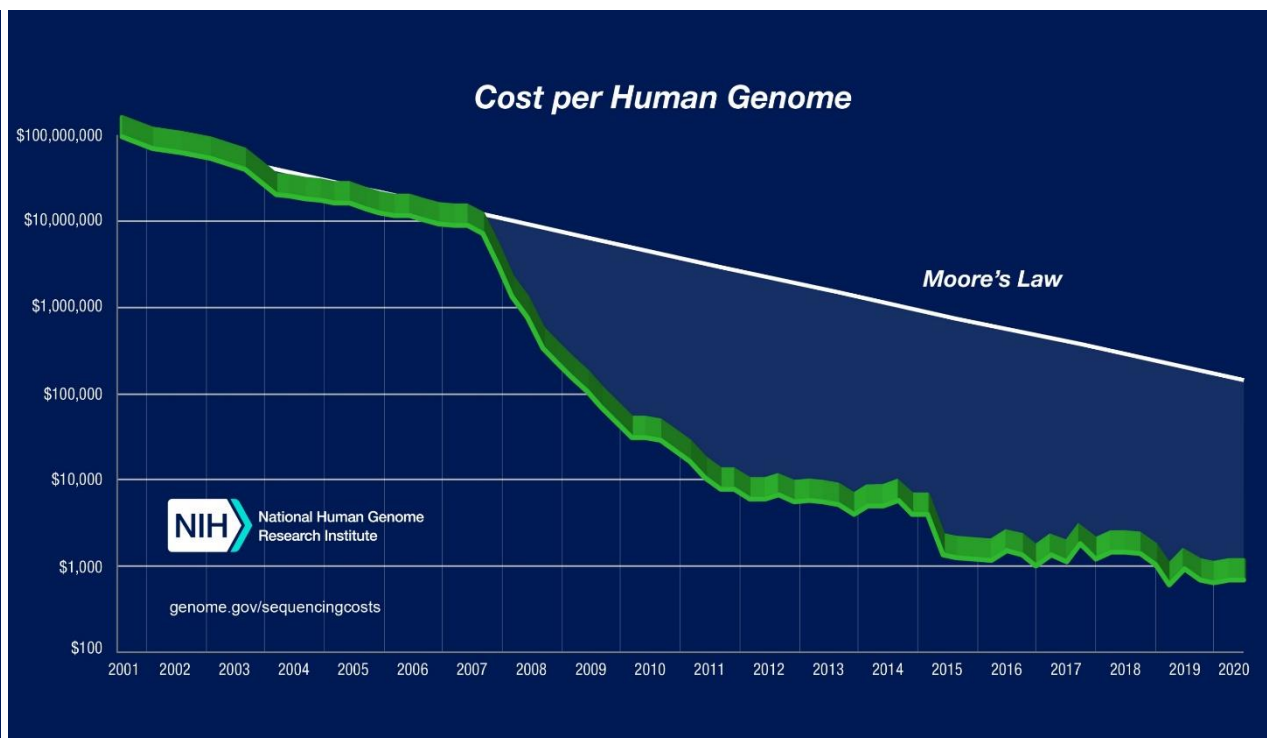
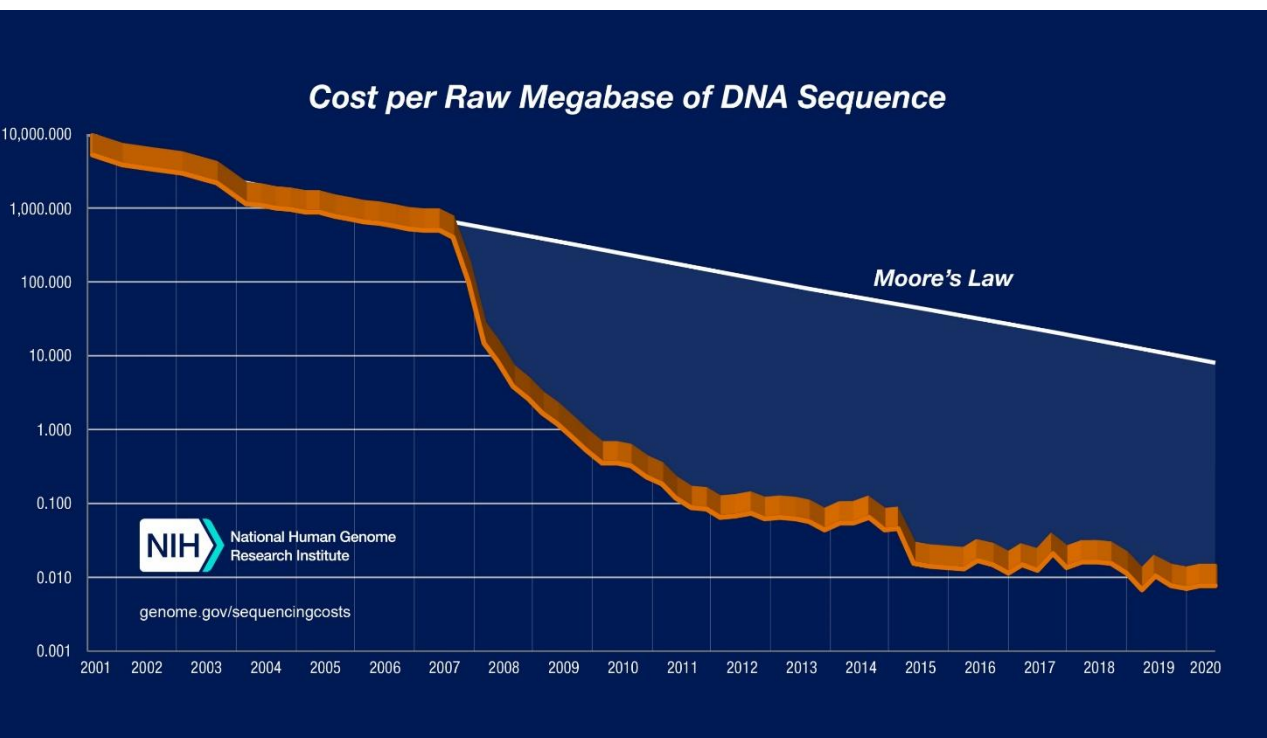
Wandrille Duchemin and Markus Müller

Total amount of global healthcare data generated and projections for end 2020 (in exabytes)

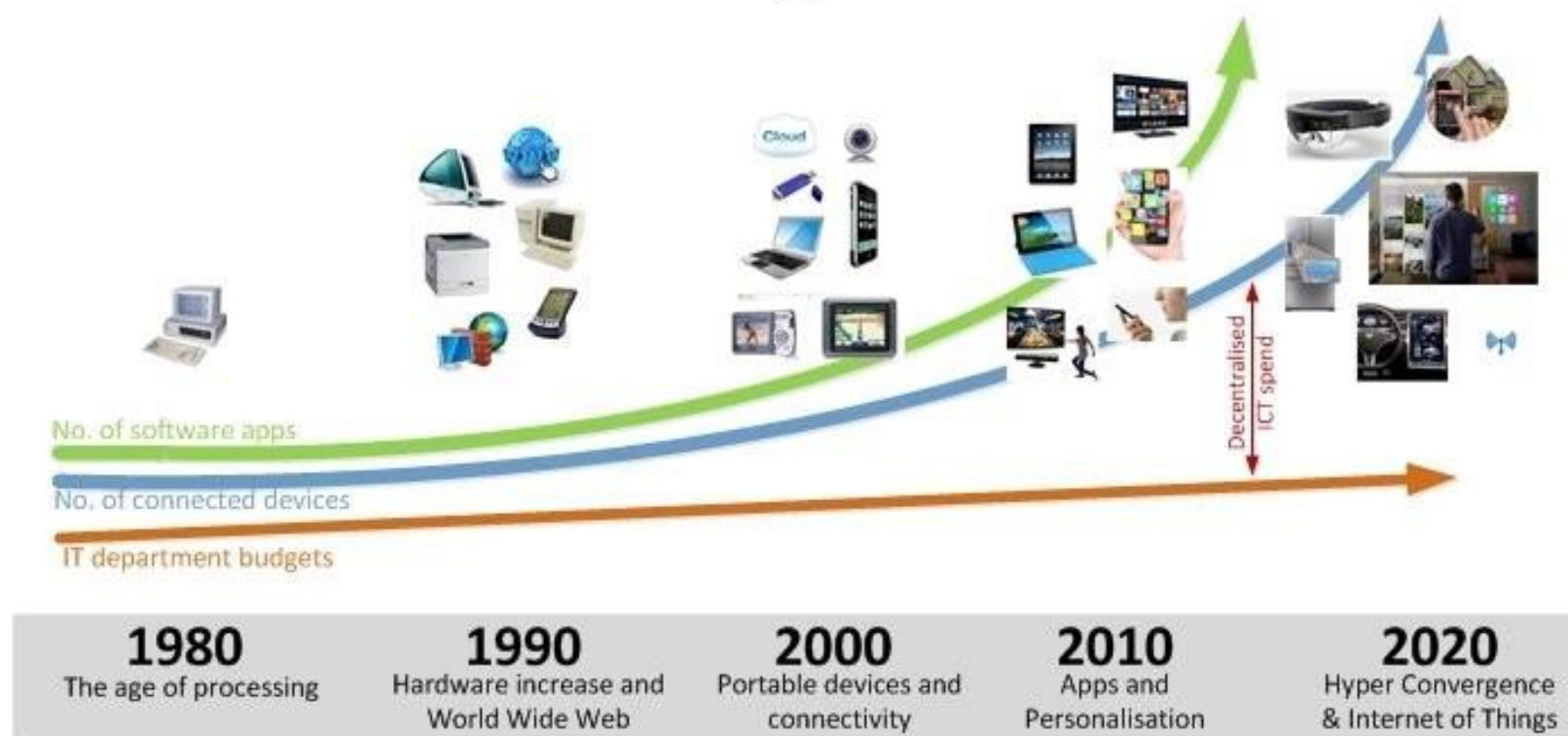


1 exabyte = 1 billion
gigabytes

<https://www.statista.com/statistics/1037970/global-healthcare-data-volume/>



Technology Timeline



<https://www.linkedin.com/pulse/technology-increase-vs-department-budgets-sam-errington/>

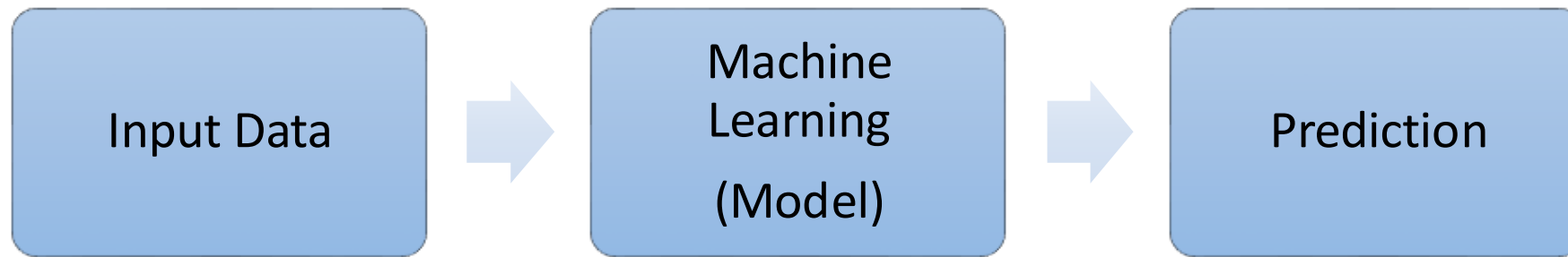
From data to knowledge



Artificial Intelligence (AI)

- The term “artificial intelligence” was coined in 1956 at “Dartmouth College” workshop, where the first AI computer program called “Logic Theorist” was presented by Allan Newell
- AI systems are computer programs that perform or support problem solving and decision making

Machine learning (ML)

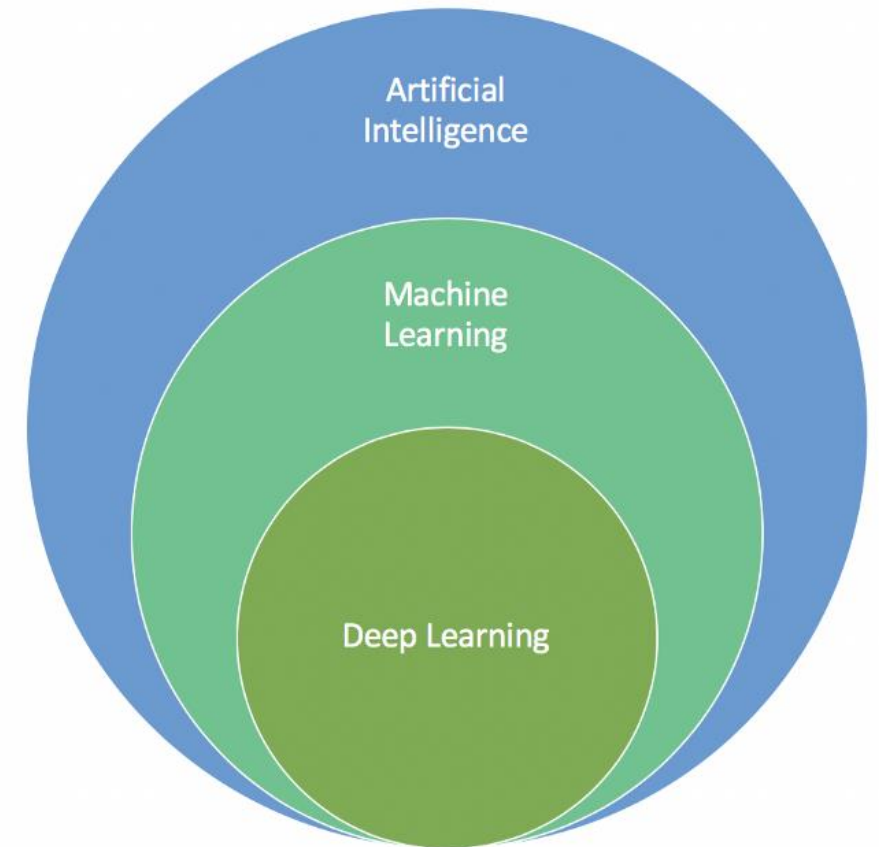


- The term machine learning was coined in 1959 by Arthur Samuel (Samuel devised a successful self-learning Checkers-playing program)
- Learning begins with observations or data
- The system looks for patterns in data and makes better decisions in the future based on the examples that we provide
- The fundamental goal of machine learning is to generalize beyond the examples in the training set.

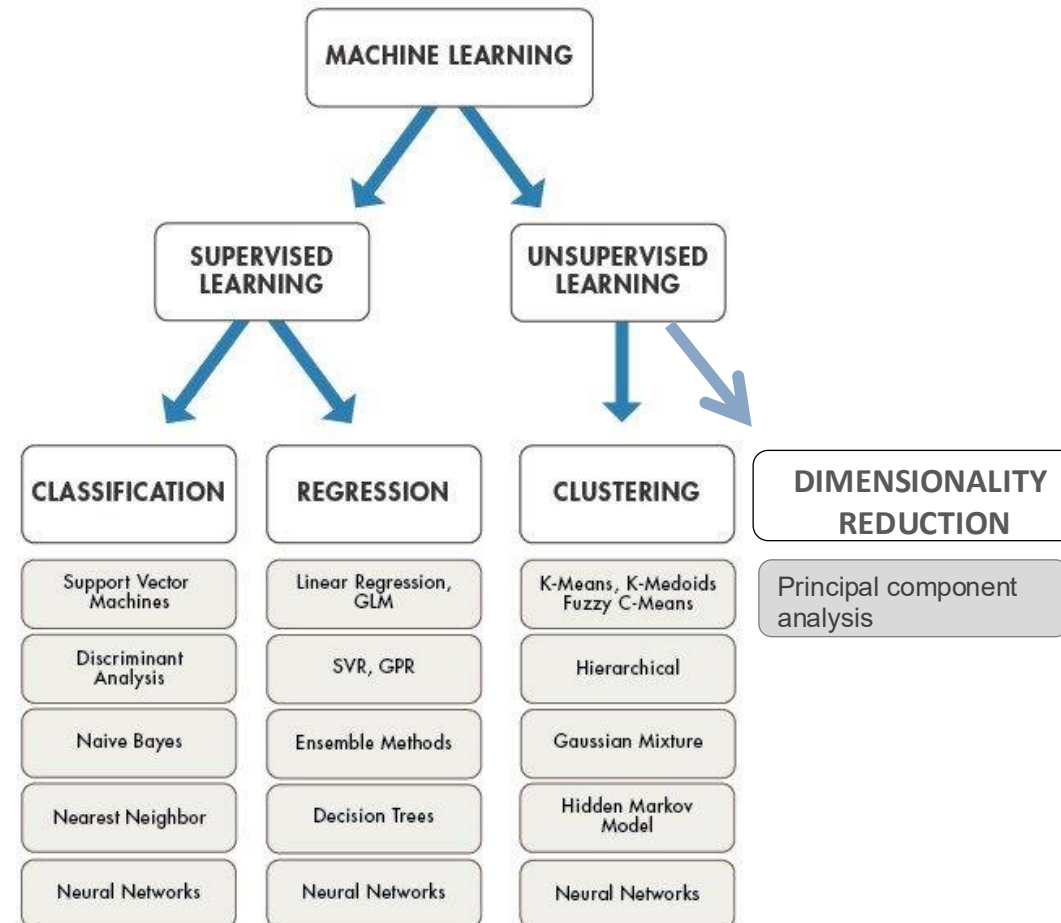
Some definitions

- Machine learning

- application (or subset) of AI that allows machines to learn from data using general purpose algorithms
- variety of algorithms to iteratively learn, describe and improve data in order to predict better outcomes
- extensive use of statistical techniques



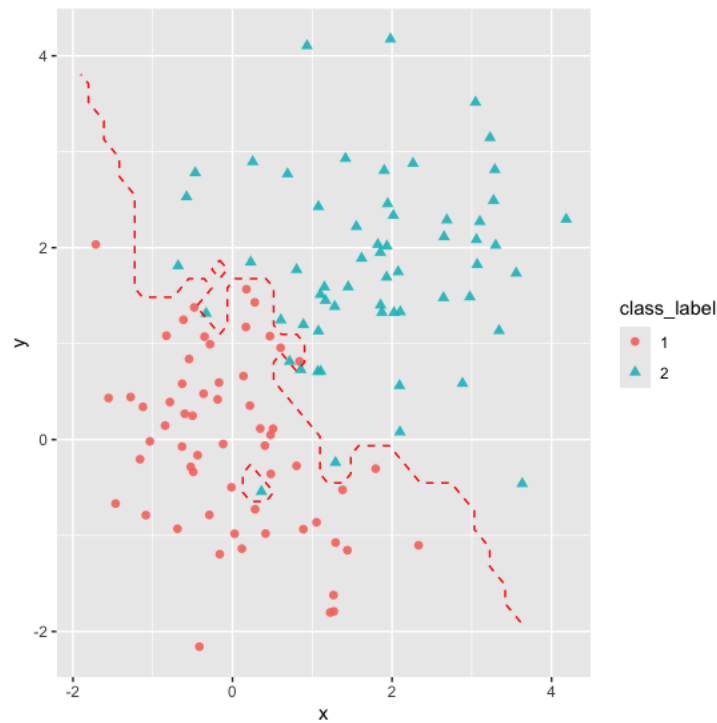
Machine learning taxonomy



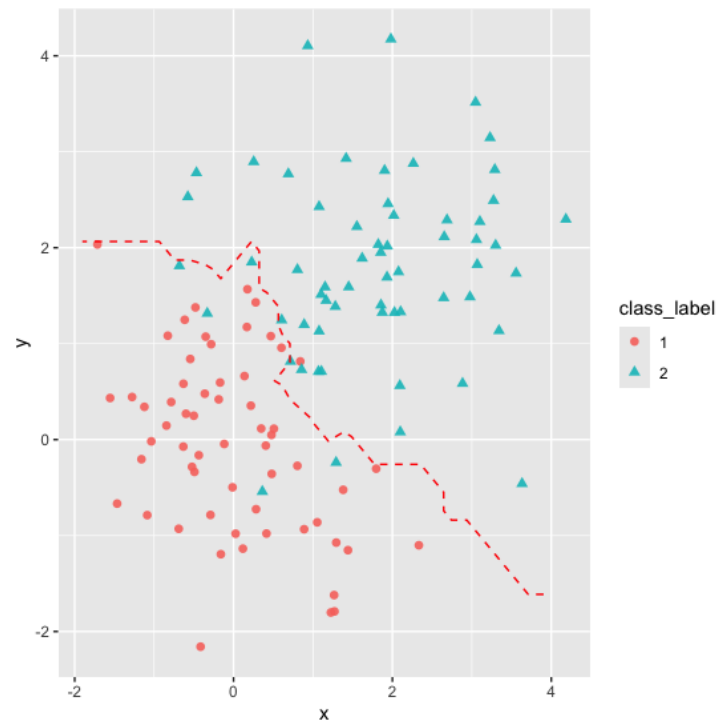
Supervised versus unsupervised learning

- Supervised learning
 - Machine learning model is trained on labeled examples and used to predict the label of unlabeled examples.
 - Prediction limited to input labels
- Unsupervised learning
 - Machine learning algorithm that does not require labels, such as a clustering algorithm.
 - Prediction of new labels
- Semi-supervised learning
- Reinforcement Learning
 - Learning via trial and error to maximize a future reward

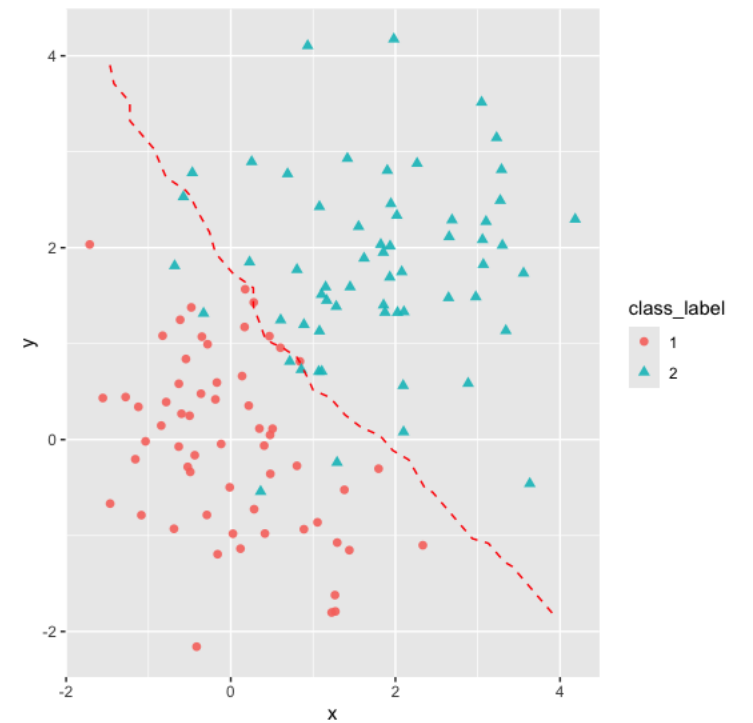
Building a model from data: which one?



A



B



C

What is a good model?

- Infinite number of models that fit training data equally well
- Model is not defined by training data alone!
- Additional criteria needed:
 - Smoothness
 - Low complexity model (especially for small training data set)
 - A model must accurately predict independent test data, if training and test data are identically distributed
- See cross validation

Machine learning in omics

Input data

DNA sequence

DNA sequence

DNA sequence

Gene expression

Gene expression data

Histone and TF ChIP-seq data

DNA sequence + gene expression + ...

DNA sequence + histone mods + ...

DNA sequence + histone mods + ...

Sequence variants + gene expression + ...

Task

Identify transcription start sites, splice sites, exons, etc.

Identify TF binding sites

Identify genes

Predict regulatory relationships

Identify biomarkers for a disease

Partition and label the genome with chromatin state annotation

Predict gene function

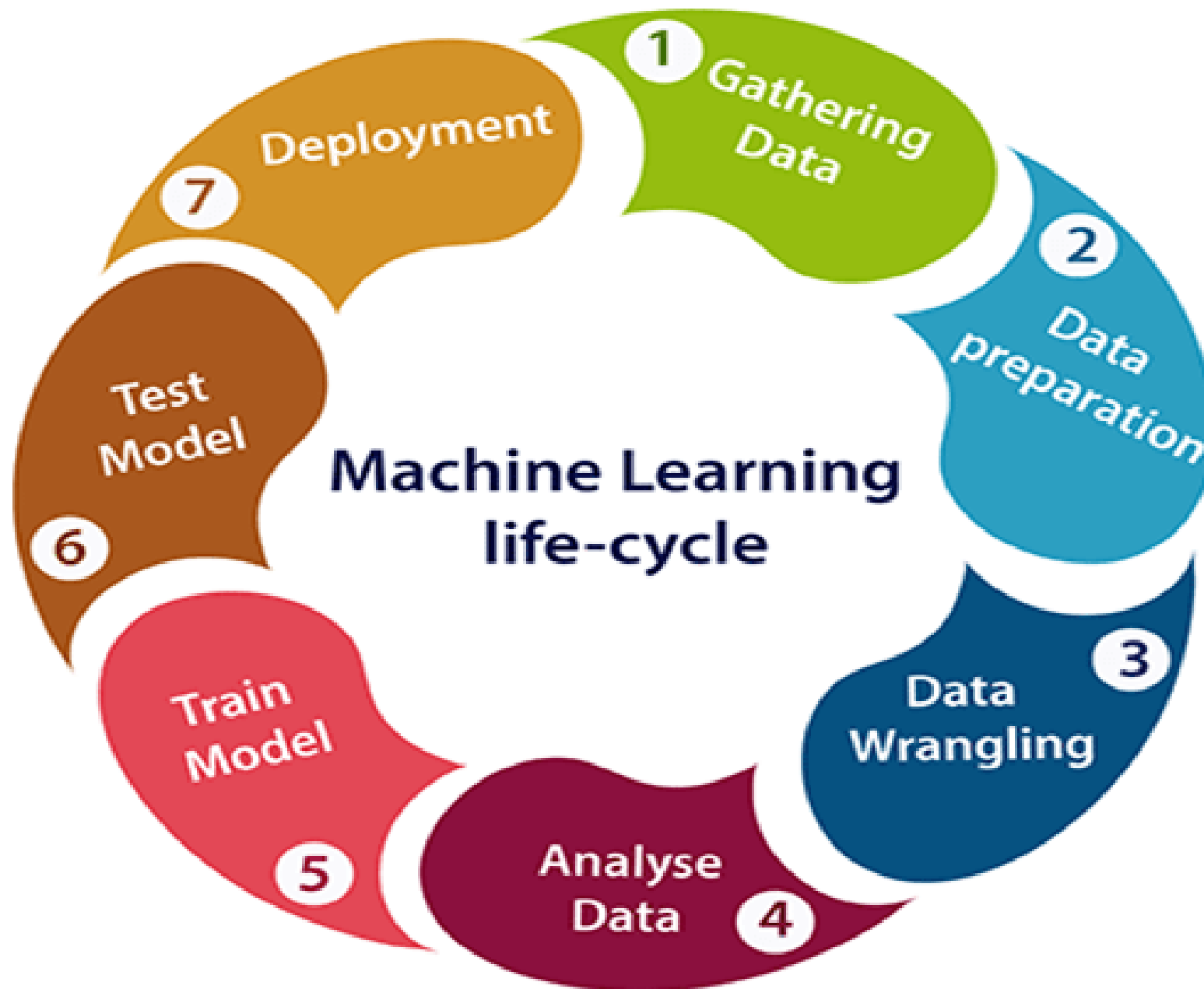
Predict gene expression

Predict variant deleteriousness

Predict disease phenotype or prognosis

Planning a ML Project

- Understand the problem
- Define the problem and project scope
- Define success metrics
- Find support and manage expectations
- Discuss with colleagues and peers
- Know when to stop project
- Understand the data
- Do Biblio and study existing solutions and algorithms
- Use benchmark models
- Use replicate runs if model contains random component
- Only use models you understand
- Plan for development, testing and production stages



Understand Your Data

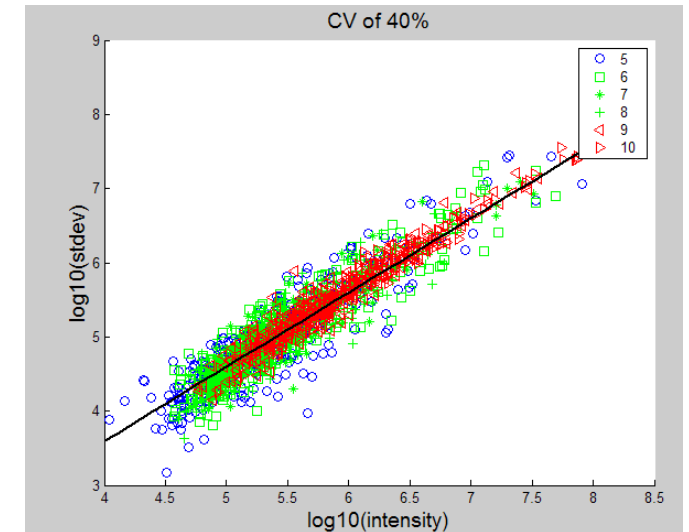
- Are data consistent?
- Are there outliers or wrongly annotated datapoints?
- Are there biases, especially if data comes from different labs or measurement devices?
- Is there enough data for what I want to do?
- What can I do to handle very large data?

Understanding the data

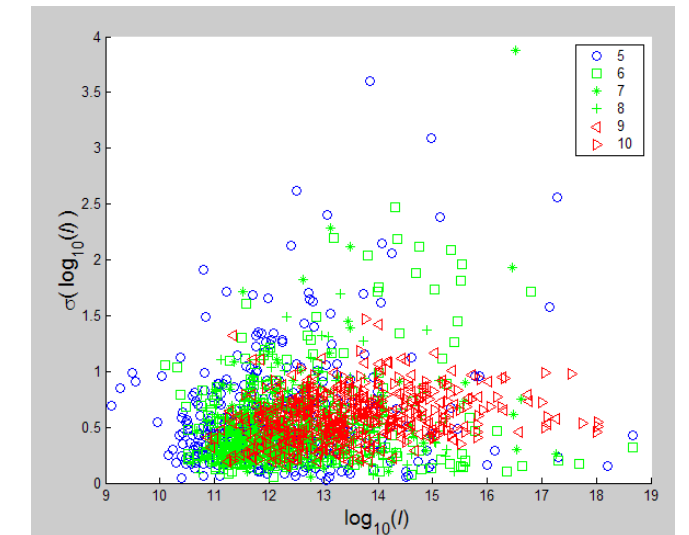
- Association
 - Discover correlations between features and target value (histograms, scatterplots)
 - Discover correlations between features (pair plots)
- Visualization in 2D or 3D
 - High dimensional data often lives on low dimensional manifolds. We can try to capture this low dimensional structure by finding suitable 'projections' of the data onto 2D or 3D spaces (PCA, ICA, MDS, tSNE, UMAP)
- Clustering
 - Discover groupings inside the input data (example: cluster DNA sequences into functional groups).

Variance Stabilization

- In Omics data the standard deviation σ of a signal often grows with increasing signal value.
- This can be detrimental for some ML techniques that assume that each feature value has the same error (e.g. k-means)
- If the dependency is linear a simple log-transform can stabilize the variance
- For more complex situations check [Huber et al. Bioinformatics 2002](#)
- Removing batch effects and confounding factors



$$\sigma = \alpha \cdot y \Rightarrow \log_{10}(\sigma) = \log_{10}(y) + \log_{10}(\alpha)$$



$$\sigma(\log_{10}(y)) = \text{const}$$

Missing Values (MV)

- Omics data (especially proteomics) often have missing values, which can be caused by leaky measurement or data processing
- There are two main strategies to deal with MVs: discard or impute them
- MVs can occur at random or depend on feature values
 - For missing at random we can only replace or impute them by the best guess, the global non-missing mean ([Donders et al, J. Clin. Epi., 2006](#))
 - In the latter case, we can try to predict the MVs using the feature values by EM, KNN, SVD, or regression technique ([Troyanskaya et al, Bioinformatics, 2001](#))

Feature Selection

- Many ML methods work better when the number of noisy features is small (curse of dimensionality)
- Feature selection is a way to detect uninformative features and remove them before applying the ML method.
- sklearn offers various feature selection methods in [sklearn.feature_selection](#)

Turning categorical values into numerical ones

Many ML tools (eg. neural nets, logistic regression) can only handle numerical features and we need to change categorical values to numbers

- Simple encoding: replace categories by numbers
 - Introduces unnatural order in categories
- One hot encoding:
 - Create a new feature f_c for each category c
 - For the category $\hat{c}$ of a data point set: $f_c = \begin{cases} 1 & \text{if } c = \hat{c} \\ 0 & \text{if } c \neq \hat{c} \end{cases}$
 - Blows up number of features if there are many categories
- Target encoding:
 - $f_c = E(y|c_i = c)$, y target value
 - For supervised learning or regression

$$\begin{matrix} & a & b \\ \begin{pmatrix} a \\ a \\ b \end{pmatrix} & \rightarrow & \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \end{matrix}$$

Reproducibility in machine learning

- Computational reproducibility: other researchers should be able to replicate the results of a model, given the full access to data, code and metainformation. This allows them to check whether the analysis was properly done.
- Generalizability: the ML model performs equally well on independent, *similar* test data from other labs.
- Credibility of ML relies on reproducibility
- Irreproducible results can easily arise in ML ([Kapoor and Narayanan, arXiv, 2022](#))
- The most prominent source of error is ‘data leakage’ (see below), when information from the test data is included in the training data.
- See [reproducibility workshop](#) from Princeton University

Vector and matrix calculus cheatsheet

$$\text{Vector } \mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_D \end{pmatrix}; \mathbf{x}^T = (x_1 \quad \cdots \quad x_D) \quad \|\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{x} = \sum_{j=1}^D (x_j \cdot x_j)^2$$

$$\text{Matrix } A = \begin{pmatrix} A_{11} & \cdots & A_{1D} \\ \vdots & \ddots & \vdots \\ A_{N1} & \cdots & A_{ND} \end{pmatrix}; (AB)_{nm} = \sum_{i=1}^D A_{ni} B_{in}$$

$$(A^T)_{ij} = A_{ji}$$

$$(AB)^T = B^T A^T$$

$$AA^{-1} = A^{-1}A = \mathbb{I} \quad (A^{-1})^T = (A^T)^{-1}$$

$$\mathbb{I} = \begin{pmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{pmatrix}$$

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{pmatrix} \frac{\partial}{\partial x_1} f(\mathbf{x}) \\ \frac{\partial}{\partial x_i} f(\mathbf{x}) \\ \frac{\partial}{\partial x_D} f(\mathbf{x}) \end{pmatrix};$$

$$\nabla_{\mathbf{x}}(\mathbf{x}^T \mathbf{x}) = 2\mathbf{x}$$

$$\nabla_{\mathbf{x}}(\mathbf{x}^T A \mathbf{x}) = (A + A^T)\mathbf{x}$$

$$\nabla_{\mathbf{x}} f(g(\mathbf{x})) = \frac{d}{dy} f(y)_{y=g(\mathbf{x})} \nabla_{\mathbf{x}} g(\mathbf{x})$$

$$\begin{aligned} \nabla_{\mathbf{x}}(f(\mathbf{x})g(\mathbf{x})) \\ = \nabla_{\mathbf{x}} f(\mathbf{x})g(\mathbf{x}) + f(\mathbf{x})\nabla_{\mathbf{x}} g(\mathbf{x}) \end{aligned}$$

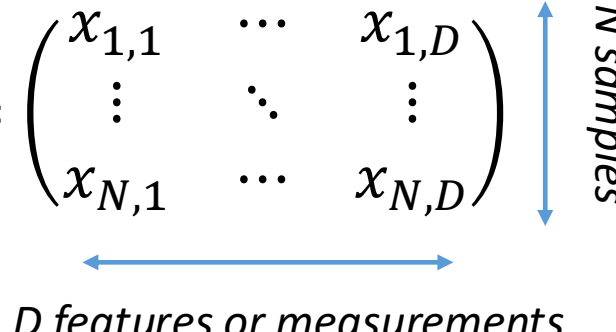
Some Notations:

N data items or vectors (rows of $\mathbf{X}$): $\mathbf{x}_i \in \mathbb{R}^D, 1 \leq i \leq N$

D measurements or features

Data matrix: $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_N)^T = \begin{pmatrix} x_{1,1} & \cdots & x_{1,D} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,D} \end{pmatrix}$

$N \times D$

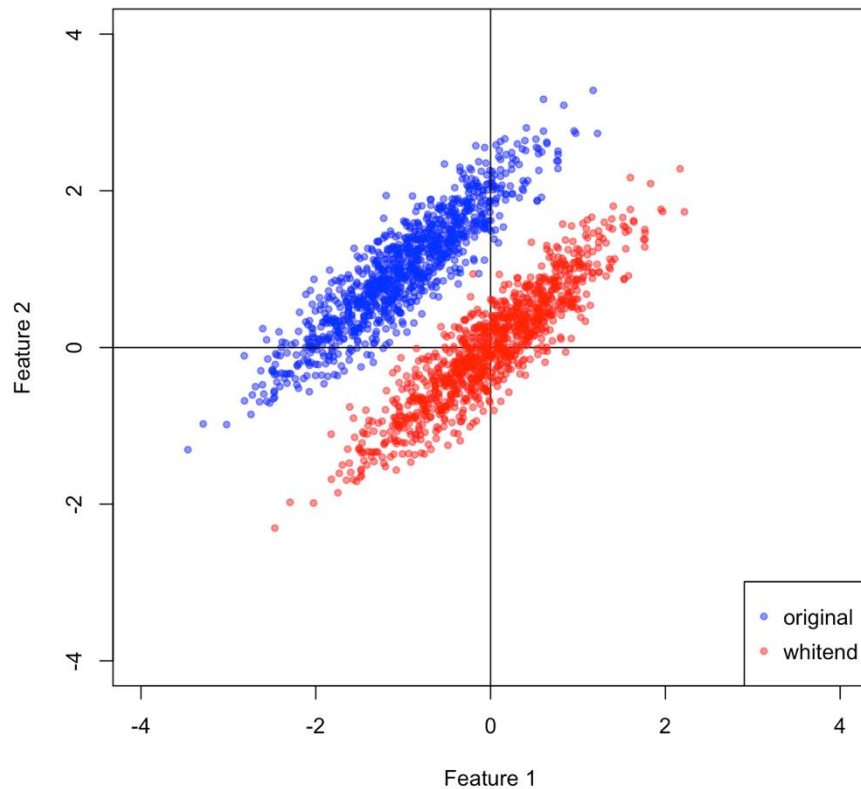


D feature vectors (columns of $\mathbf{X}$): $\boldsymbol{\varphi}_i = \begin{pmatrix} X_{1i} \\ \vdots \\ X_{Ni} \end{pmatrix}, \boldsymbol{\varphi}_i \in \mathbb{R}^N, 1 \leq i \leq D$

Zero centering:

Zero centering:

$$m_j = \frac{1}{N} \sum_{i=1}^N X_{ij}, X_{ij} \mapsto X_{ij} - m_j$$



Columns or features have 0 means!

Principal Component Analysis (PCA)

We try to find a direction $\mathbf{w}$ along which the data has the largest variance. We assume X is zero-centered:

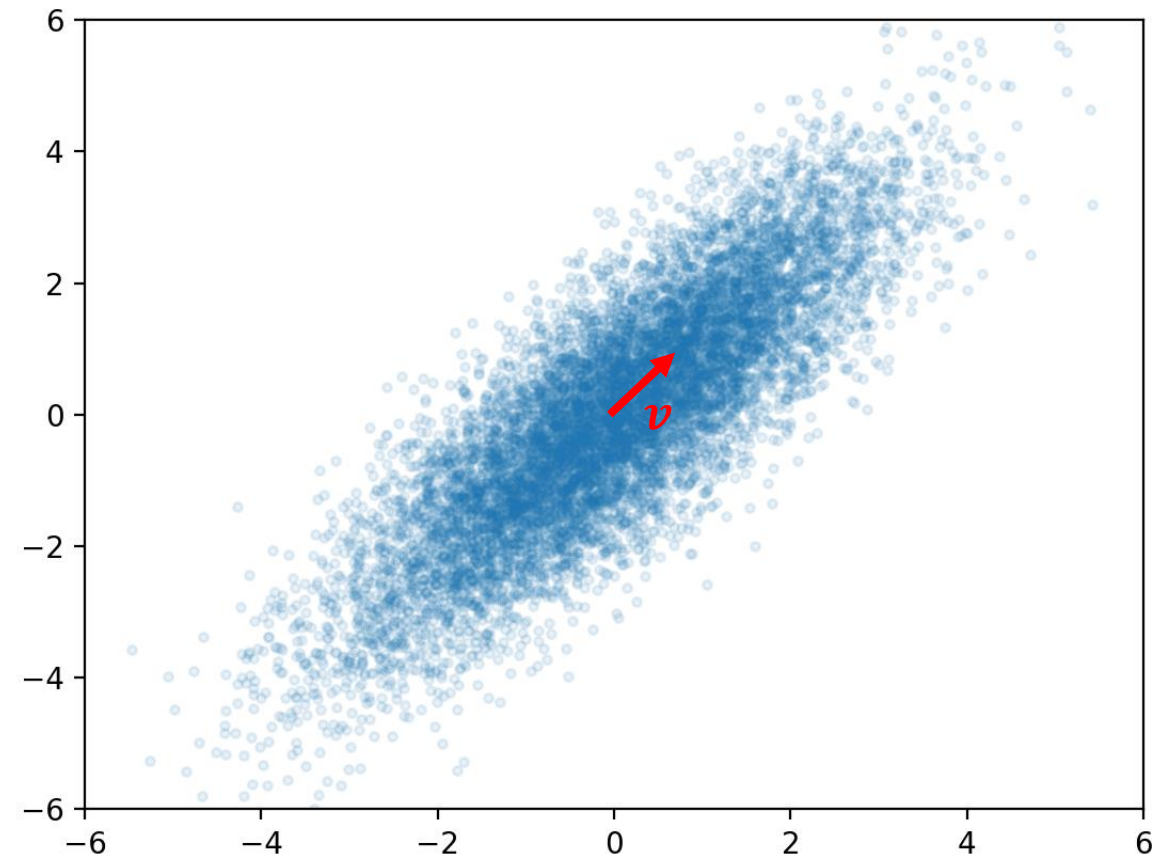
$$\mathbf{v} = \operatorname{argmax}_{\|\mathbf{w}\|=1} \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w})^2 = \operatorname{argmax}_{\|\mathbf{w}\|=1} \frac{1}{N} \mathbf{w}^T X^T X \mathbf{w}$$

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w} - \langle \mathbf{x}_i \rangle \cdot \mathbf{w})^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w})^2$$

$$\sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w})^2 = \sum_{i=1}^N \sum_{j=1}^D (X_{ij} \cdot w_j)^2 =$$

$$\|X\mathbf{w}\|^2 = (X\mathbf{w})^T (X\mathbf{w}) = \mathbf{w}^T X^T X \mathbf{w}$$

Zero centered X



PCA

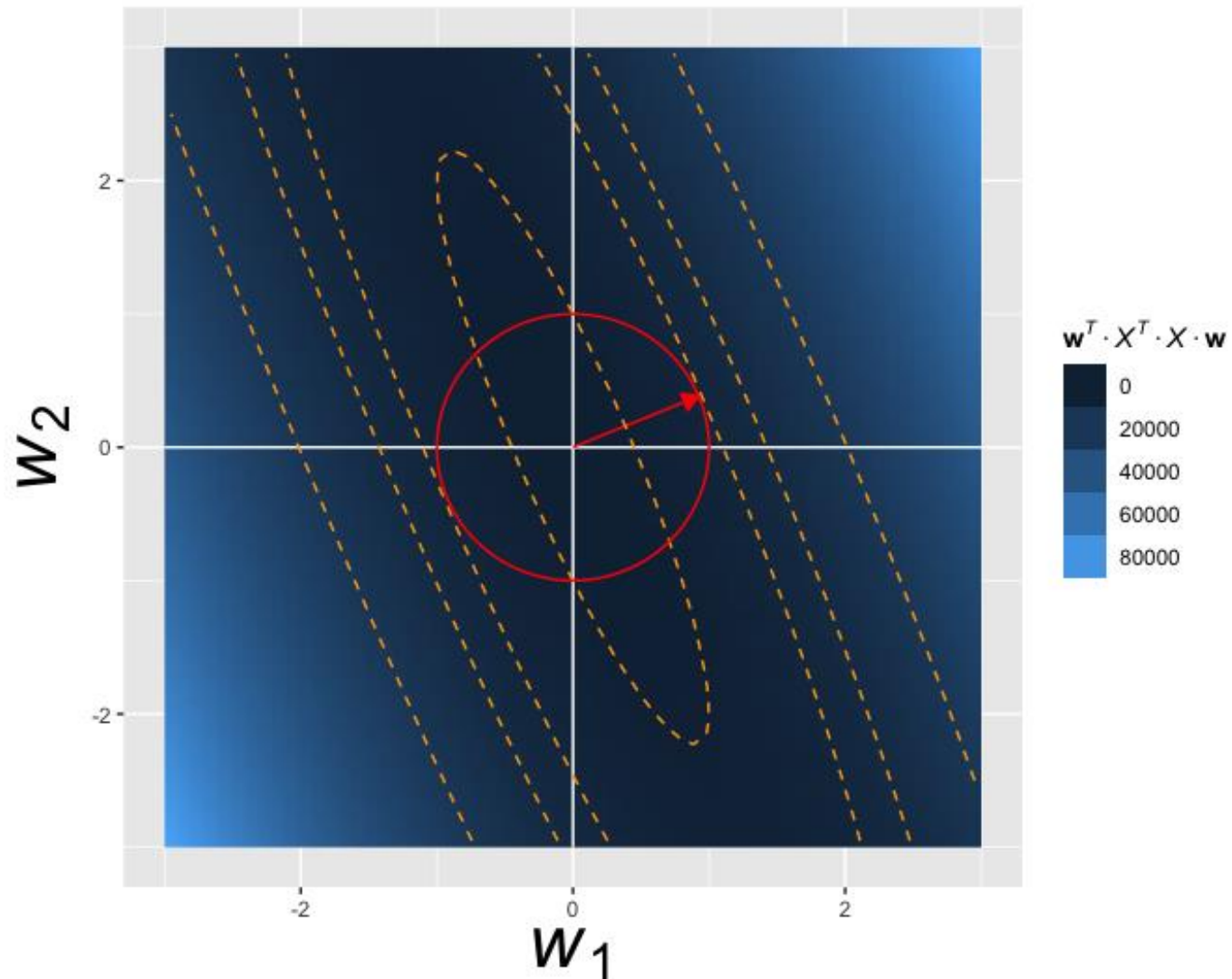
$$\mathbf{v} = \operatorname{argmax}_{\|\mathbf{w}\|=1} \frac{1}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}$$

$$\nabla_{\mathbf{w}} \frac{1}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} = \lambda \mathbf{w}$$

$$\Rightarrow \frac{1}{N} \mathbf{X}^T \mathbf{X} \mathbf{v} = \lambda \mathbf{v}$$

$$\Rightarrow \frac{1}{N} \mathbf{X}^T \mathbf{X} \mathbf{v}_k = \sigma_k^2 \mathbf{v}_k$$

$$\sigma_1^2 \geq \sigma_2^2 \geq \dots \geq \sigma_D^2$$

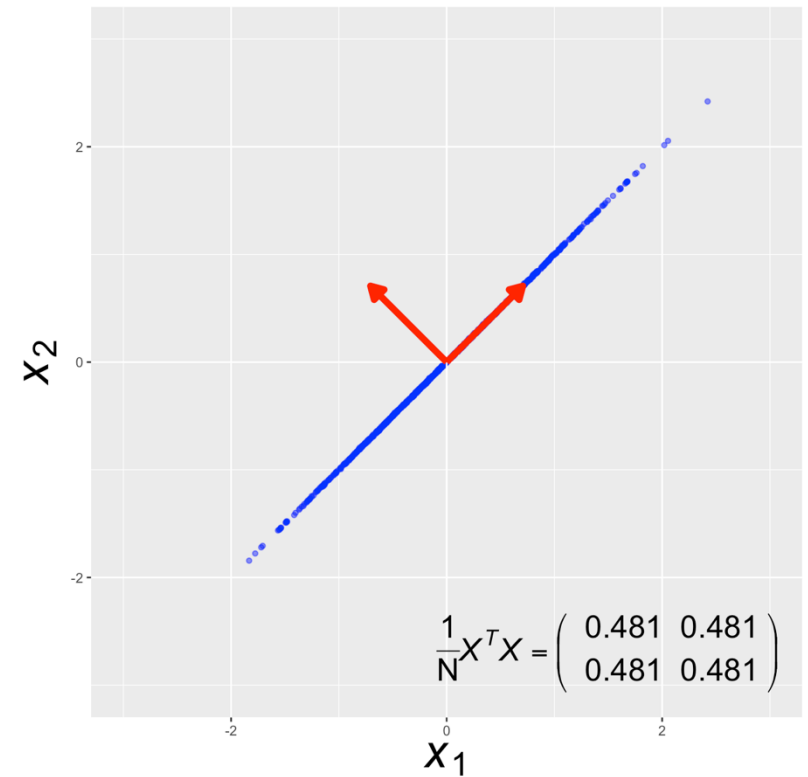
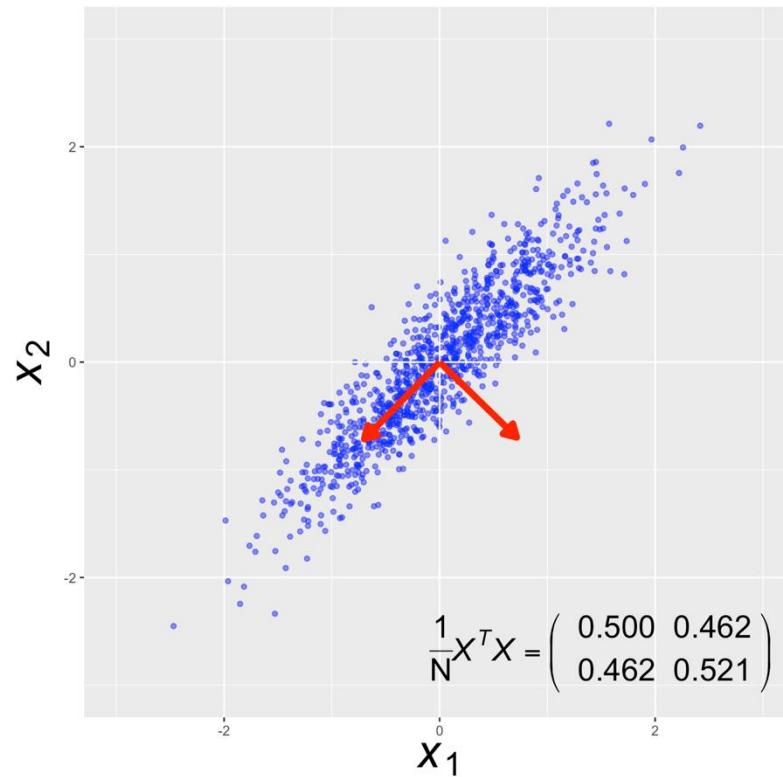
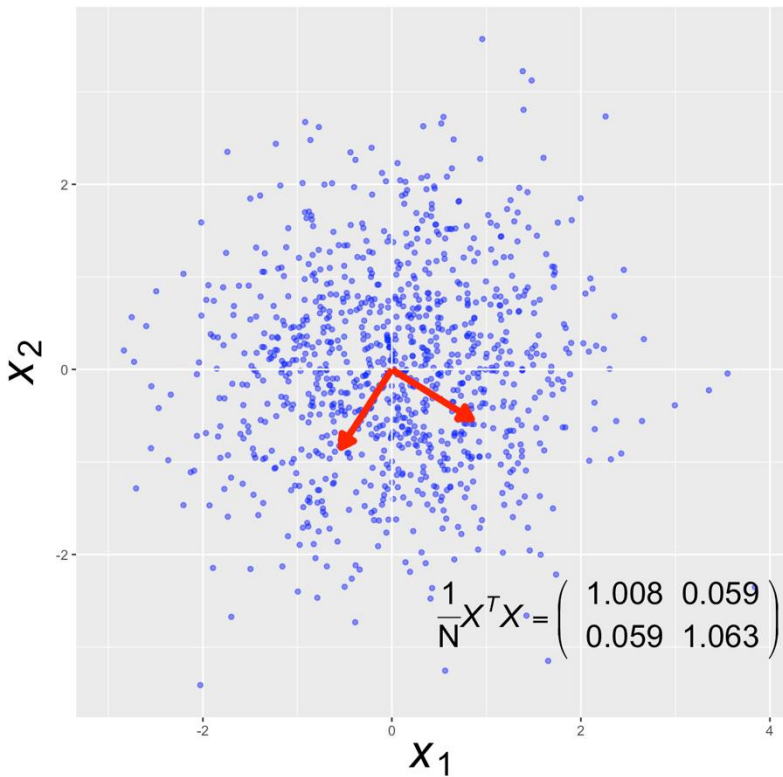


$$\mathbf{X}^T \mathbf{X}$$

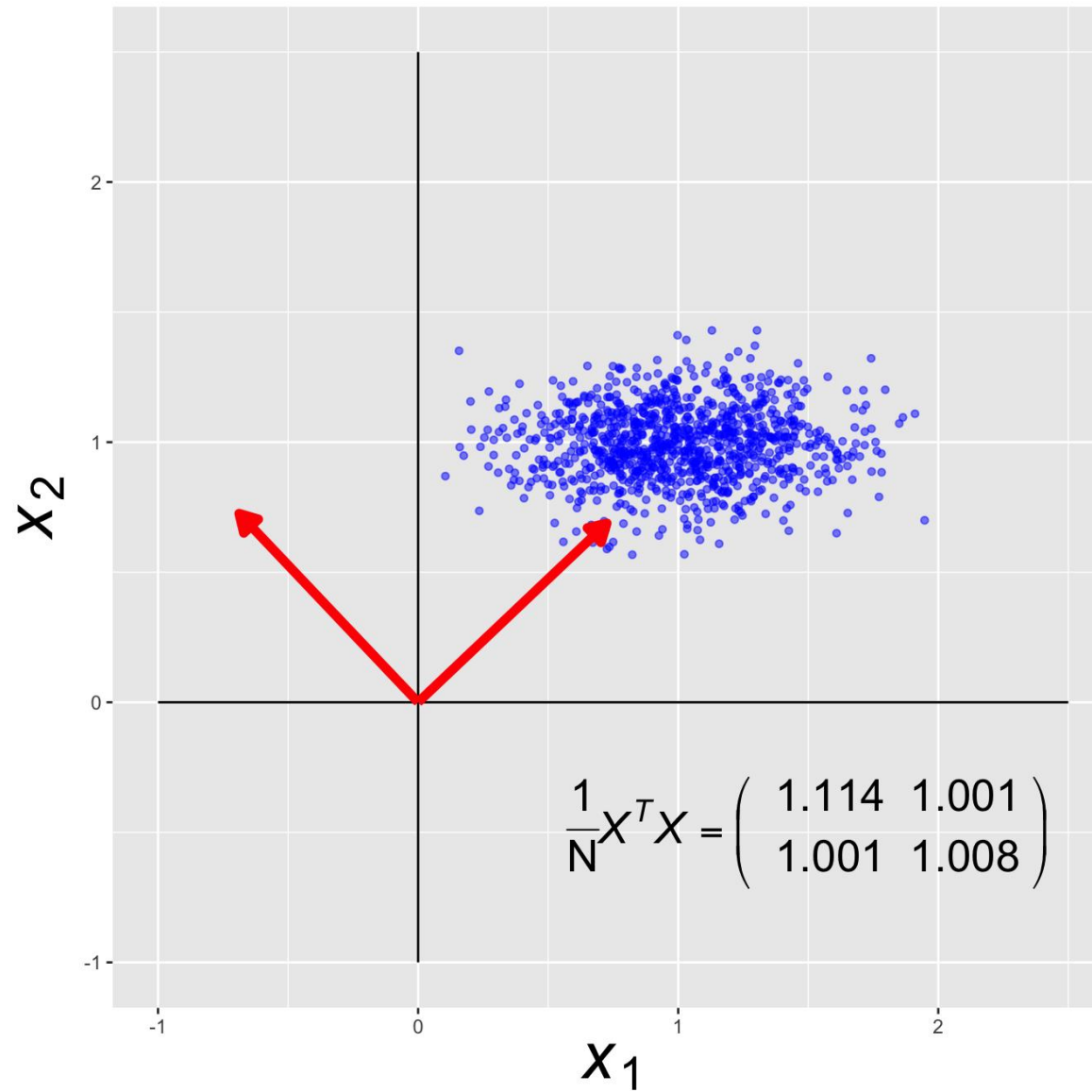
- $\mathbf{X}^T \mathbf{X}$ is a $D \times D$ matrix
- $\frac{1}{N} (\mathbf{X}^T \mathbf{X})_{nm} = \frac{1}{N} \sum_{i=1}^N X_{in} X_{im} =$ covariance matrix if $\mathbf{X}$ zero centered
- If $\mathbf{x}_i \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}) \Rightarrow \frac{1}{N} \mathbf{X}^T \mathbf{X} \approx \boldsymbol{\Sigma}$
- $(\mathbf{X}^T \mathbf{X})^T = \mathbf{X}^T \mathbf{X} \Rightarrow \mathbf{X}^T \mathbf{X}$ has D real orthogonal eigenvectors $\mathbf{v}_i \in \mathbb{R}^D, 1 \leq i \leq D$
- Eigenvalues of $\frac{1}{N} \mathbf{X}^T \mathbf{X}$: $\lambda_i \in \mathbb{R}, \lambda_i = \sigma_i^2 \geq 0$
- If feature vectors are orthogonal: $\boldsymbol{\varphi}_i^T \boldsymbol{\varphi}_j = \delta_{ij}$, then

$$\frac{1}{N} \mathbf{X}^T \mathbf{X} = \begin{pmatrix} \sigma_1^2 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_D^2 \end{pmatrix}$$

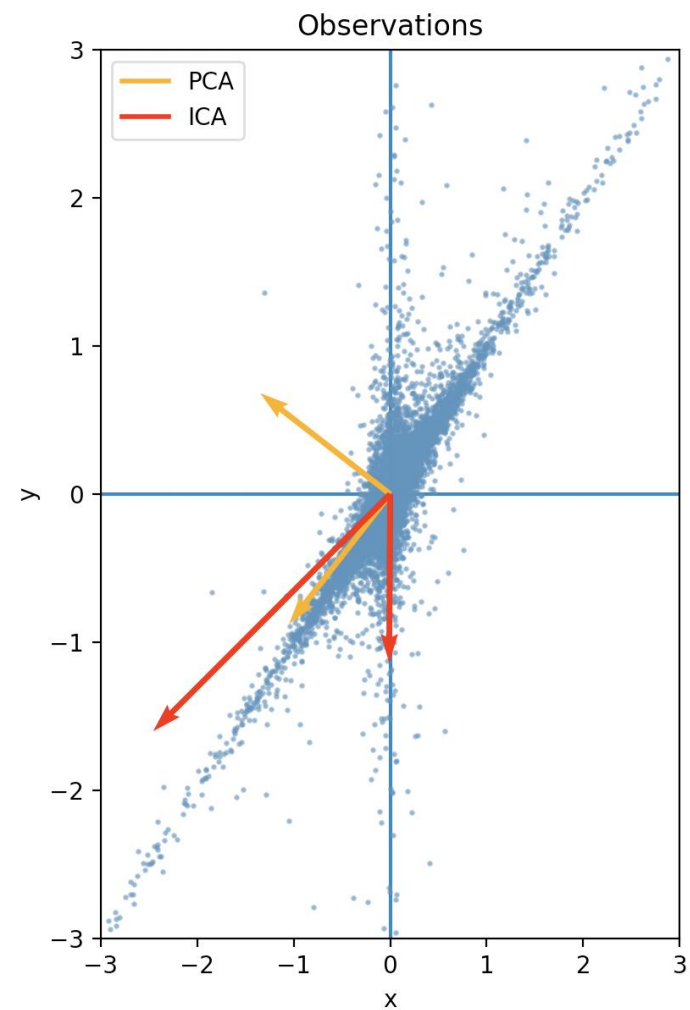
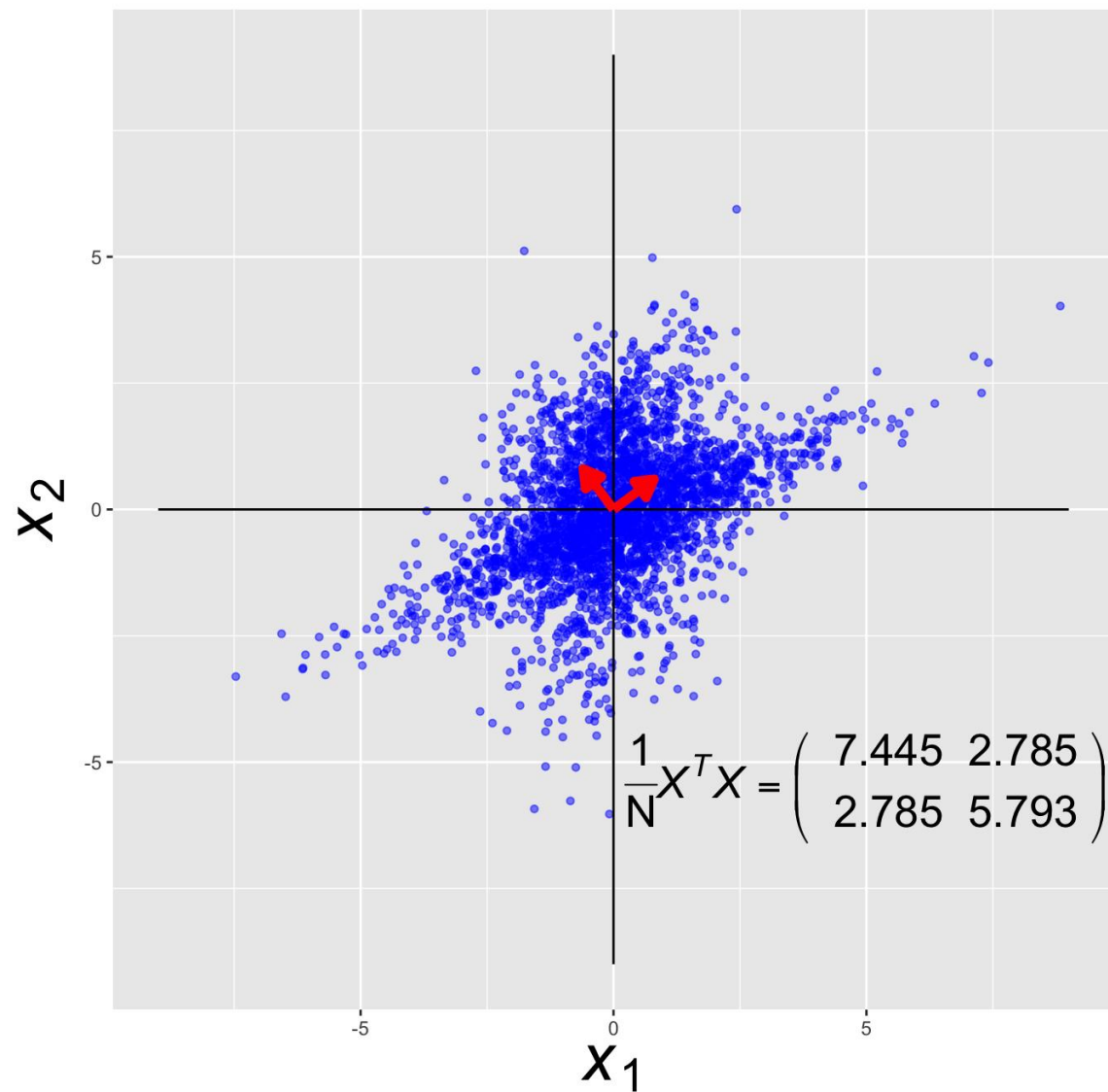
PCA



PCA



PCA



More notations:

D eigenvectors of $\mathbf{X}^T \mathbf{X} : \mathbf{v}_i \in \mathbb{R}^D, 1 \leq i \leq D$

$$\mathbf{X}^T \mathbf{X} \cdot \mathbf{v}_i = \sigma_i^2 \mathbf{v}_i$$

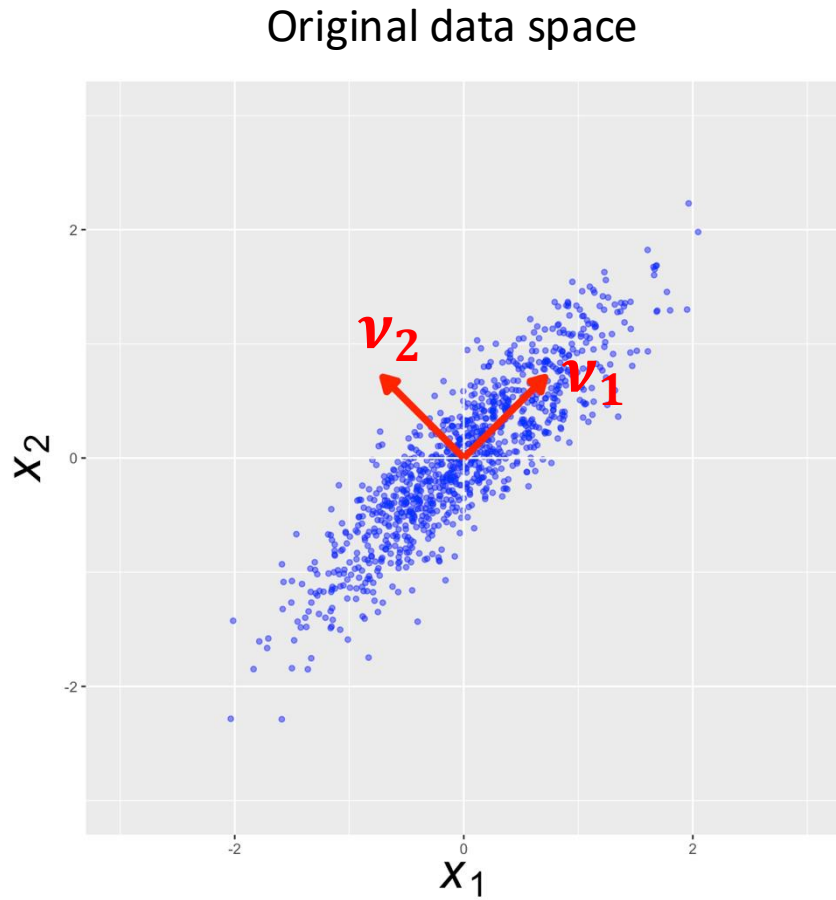
PC matrix: $\mathbf{V} = (\mathbf{v}_1, \mathbf{v}_2 \quad \cdots \quad \mathbf{v}_D)^T = \begin{pmatrix} v_{1,1} & \cdots & v_{1,D} \\ \vdots & \ddots & \vdots \\ v_{D,1} & \cdots & v_{D,D} \end{pmatrix} \quad \downarrow \quad \begin{pmatrix} \sigma_1^2 \\ \vdots \\ \sigma_D^2 \end{pmatrix}$

$D \times D$

$\sigma_1^2 \geq \sigma_2^2 \geq \cdots \geq \sigma_D^2$

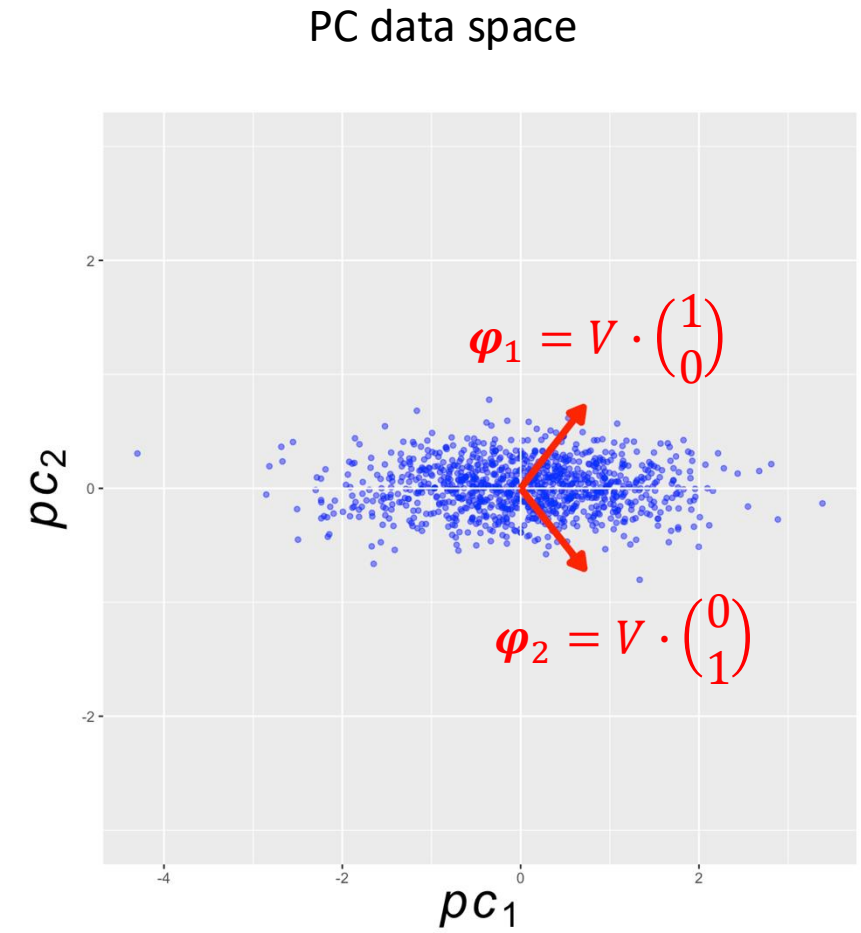
$(\mathbf{v}_1, \mathbf{v}_2 \quad \cdots \quad \mathbf{v}_D)$ are orthogonal eigenvectors $\Rightarrow \mathbf{V}$ is orthogonal (rotation)
 $\Rightarrow \mathbf{V}^{-1} = \mathbf{V}^T$

Transform to PC space:



$$pc_i = V \cdot x_i$$

$$x_i = V^T \cdot pc_i$$



PCA is an orthogonal transformation of the data!

PCA in scikit-learn:

- `class sklearn.decomposition.PCA(n_components=None, *, copy=True, whiten=False, svd_solver='auto', tol=0.0, iterated_power='auto', n_oversamples=10, power_iteration_normalizer='auto', random_state=None)`
- *n_components*: number of PC to keep
 - If $0 < n_components < 1$: amount of variance to be explained
- *whiten*: unit variance for each PC
- *svd_solver*: depending on the values of N , D , and *n_components*, different algorithms to compute PCs are more efficient.

PCA in scikit-learn:

- `pca = PCA().fit( $\mathbf{X}$ )`; $\mathbf{X}$ should be zero-centered
 - `$\mathbf{V}$  = pca.components_`
- `$\mathbf{PC}$  = pca.transform( $\mathbf{X}$ )`
- `$\mathbf{pc}_i = \mathbf{V} \cdot \mathbf{x}_i$` or in matrix form `$\mathbf{PC} = \mathbf{XV}^T$`
- Feature directions `$\boldsymbol{\varphi}_i$` in PC space:

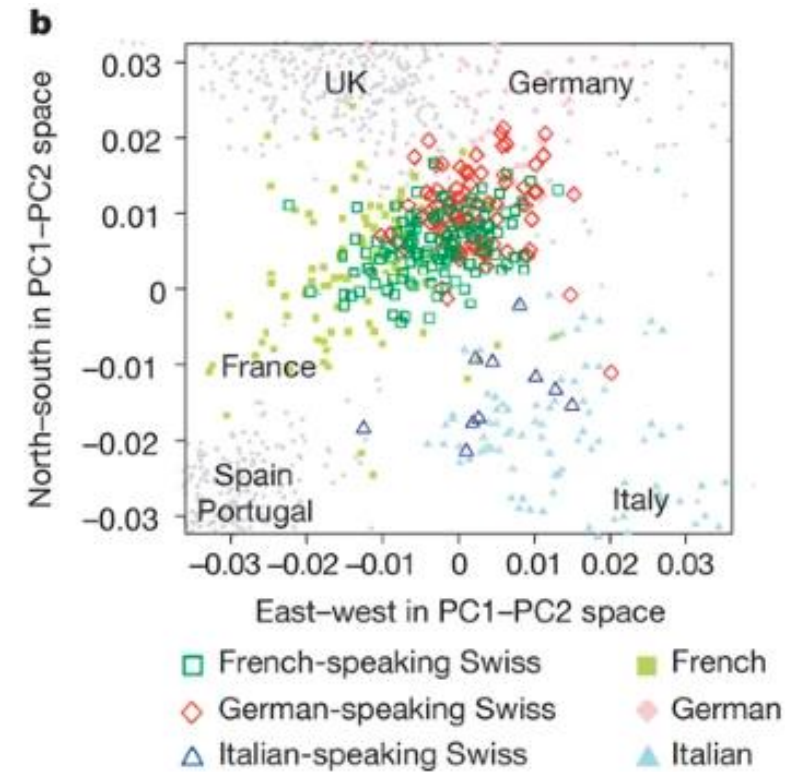
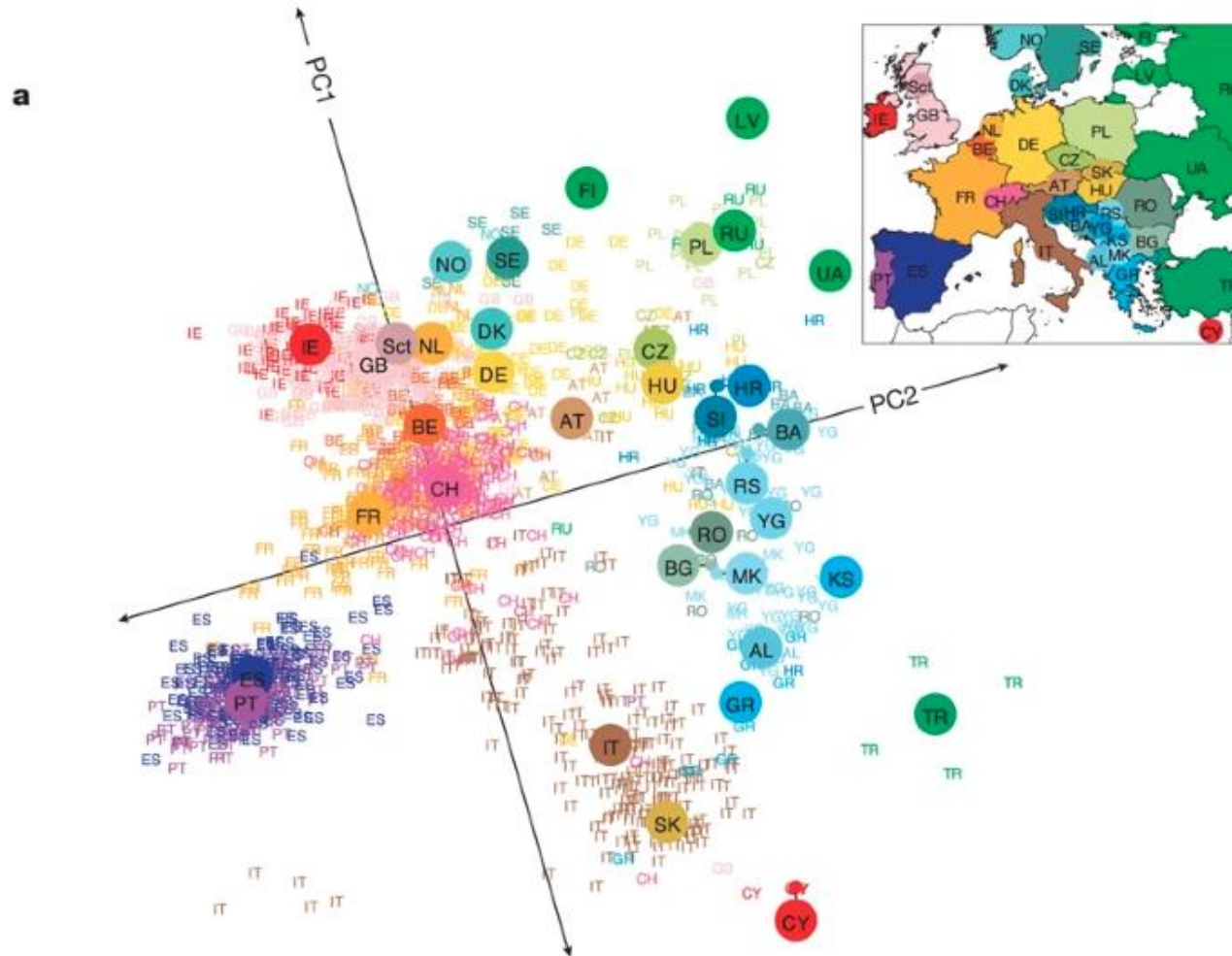
$$\bullet \mathbf{V}\mathbf{e}_i = \mathbf{V}_{.i} = \begin{pmatrix} \dots & V_{1i} & \dots \\ \dots & \vdots & \dots \\ \dots & V_{Di} & \dots \end{pmatrix}$$

PCA: what is it good for?

- Visualization
- Dimension reduction, data compression, data smoothing
- Decorrelate or whiten signals

PC projection

Figure 1: Population structure within Europe.

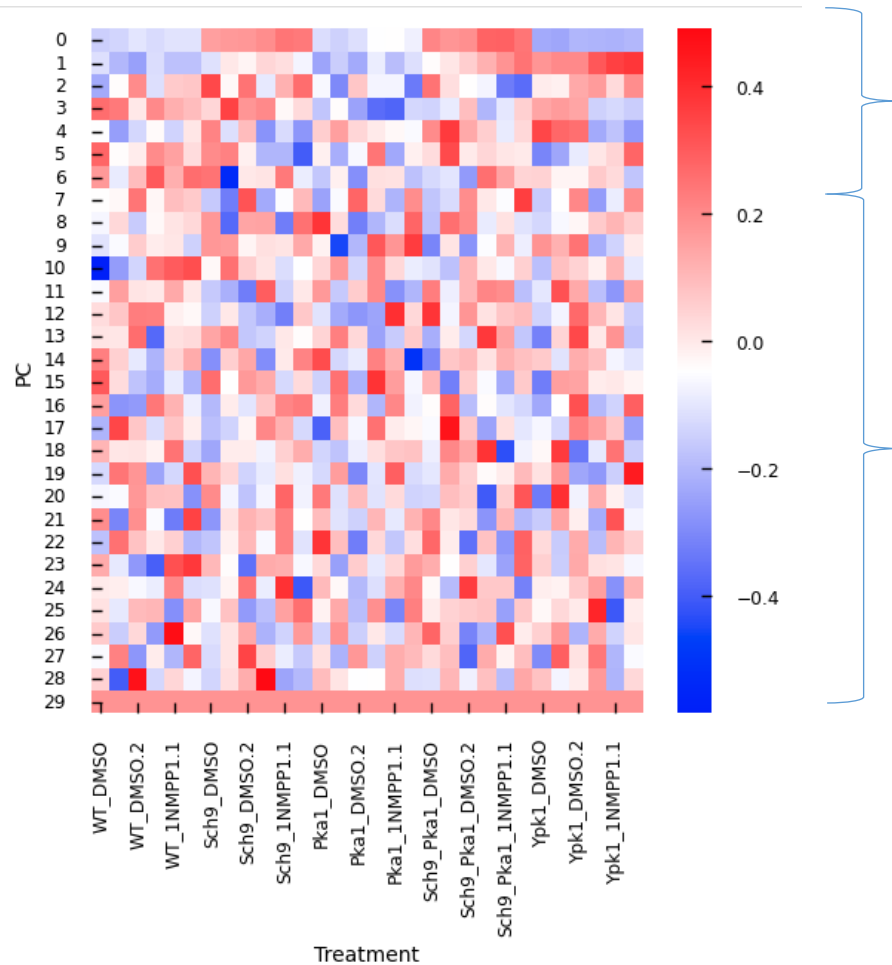


[Novembre et al. Nature 2008](#)

197,146 loci in 1,387 individuals with grandparental ancestry from same location

PCA for data compression and cleaning:

V



Informative components



Checkout scikit-learn example:
[Image denoising using kernel PCA](#)

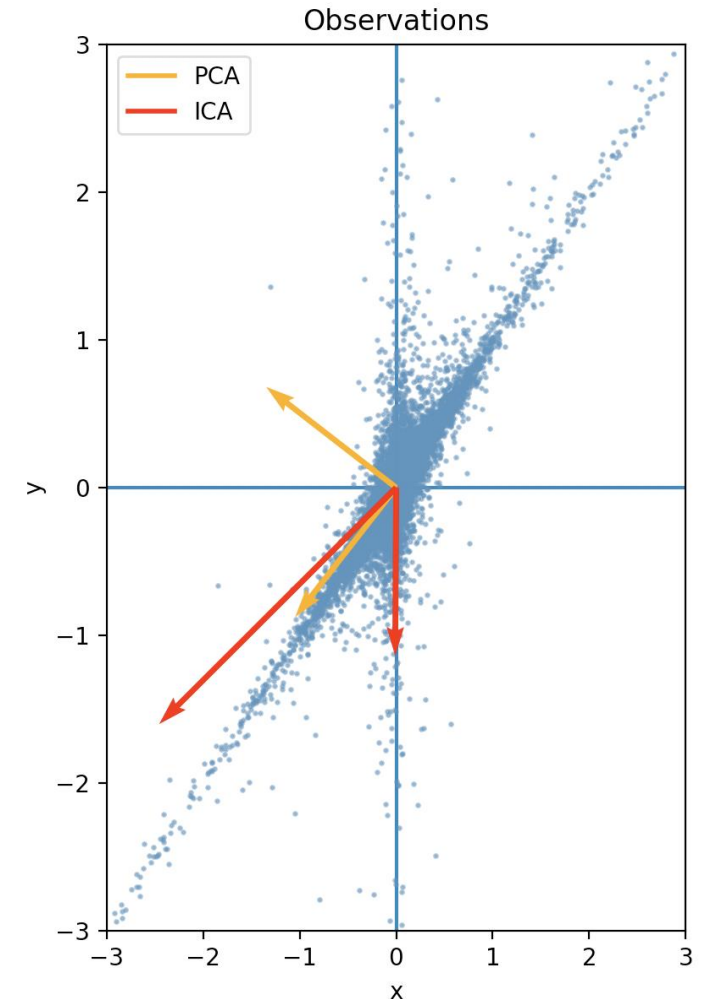
Independent Component Analysis (ICA)

We try to find direction $\mathbf{w}$ along which the data deviates the most from a Gaussian distribution:

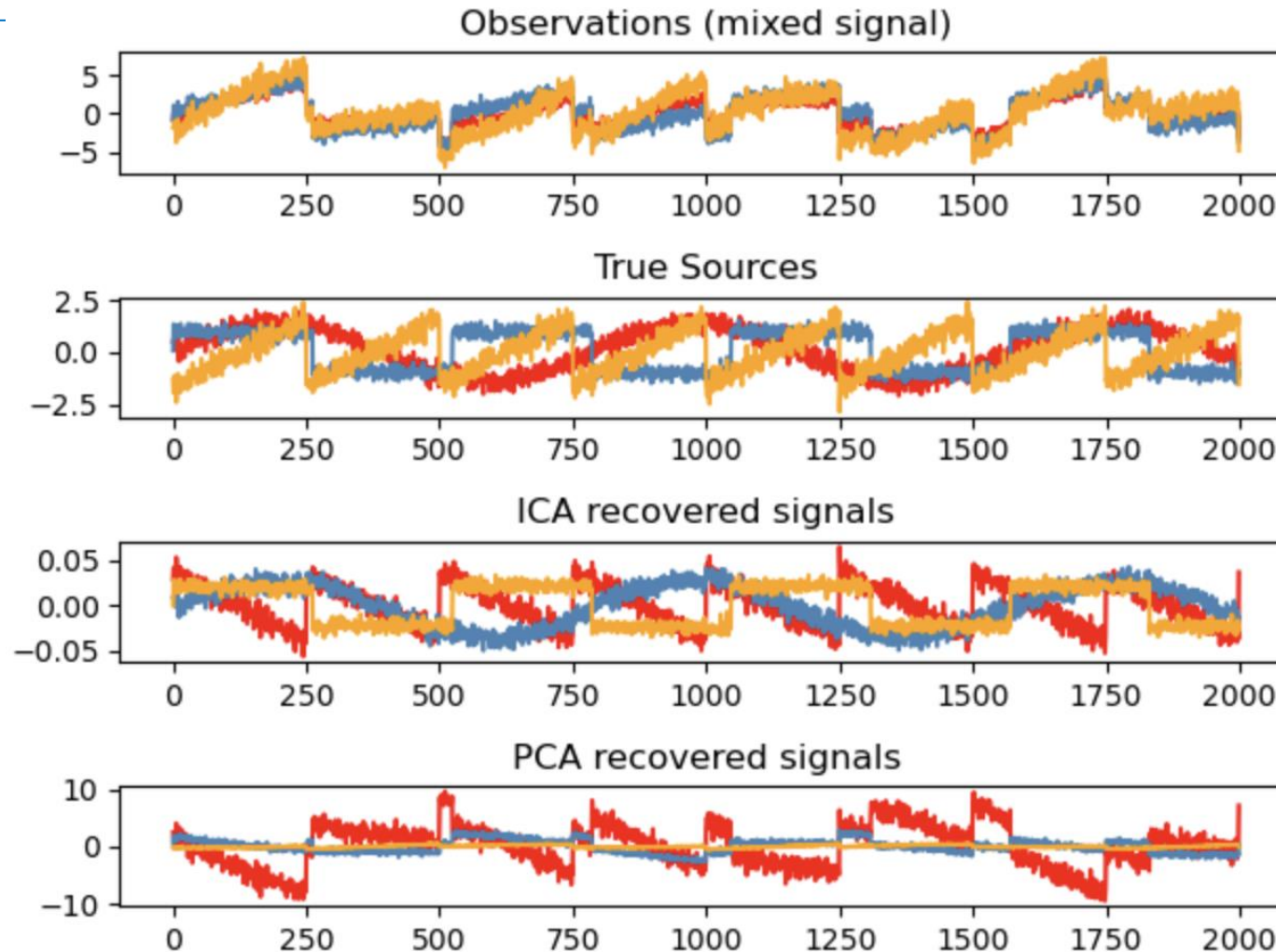
$$\mathbf{w} = \underset{\|\mathbf{w}\|=1}{\operatorname{argmax}} DG(t_1, t_2, \dots, t_N), t_i = \mathbf{x}_i \cdot \mathbf{w}$$

where $DG(t_1, t_2, \dots, t_N)$ is a function that measures how far away $\{t_1, t_2, \dots, t_N\}$ are from a Gaussian distribution.

Empirical functions DG were found that work well in practice and are fast to calculate ([Hyvärinen & Oja, Neural Networks, 2000](#)).



Independent Component Analysis (ICA)



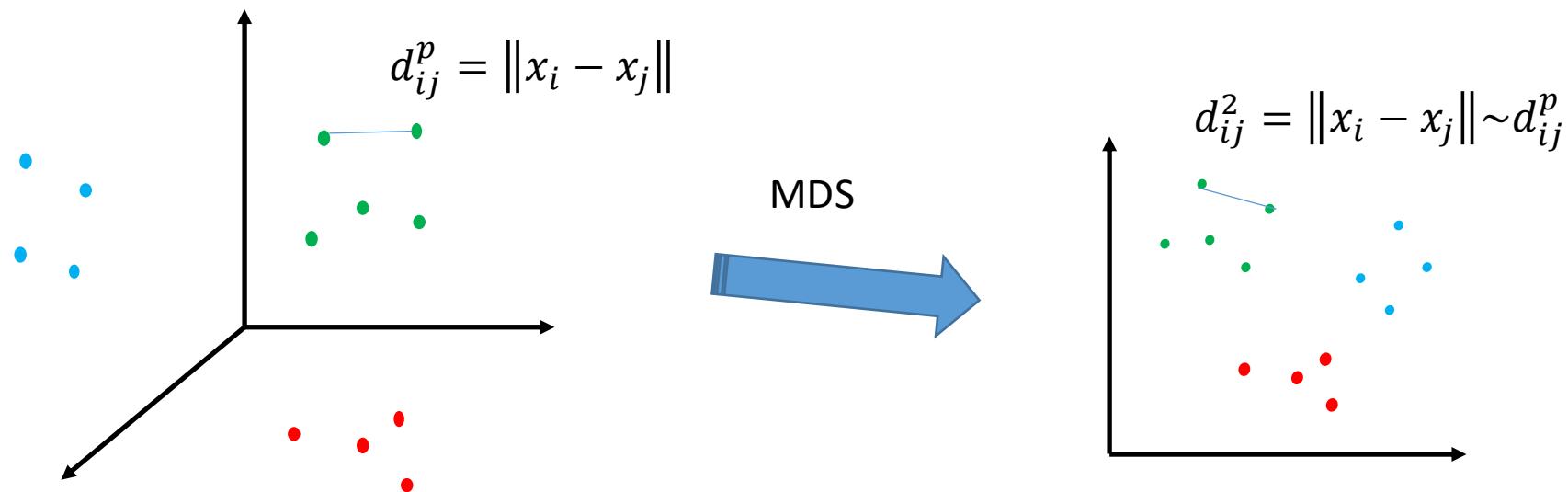
Multidimensional scaling (MDS)

- visual representation of distances between sets of objects
- objects that are more similar (or have smaller distances) are closer together on the MDS than objects that are less similar (or have larger distances)
- MDS can also serve as a dimension reduction technique for high-dimensional data

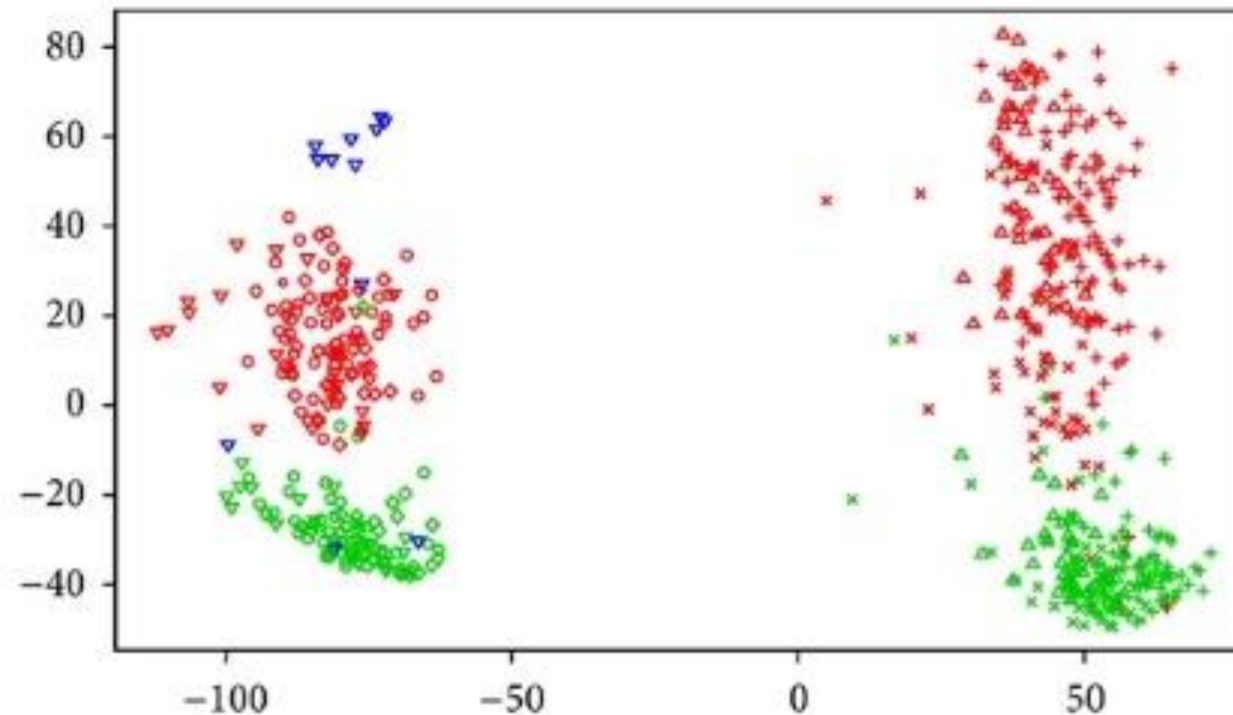
Multidimensional scaling (MDS)

- Step by step

1. Assign points to arbitrary coordinates in p-dimensional space.
2. Compute euclidean distances among all pairs of points
3. Compare the similarity matrix with the original input matrix by evaluating the stress function. Stress is a goodness-of-fit measure, based on differences between predicted and actual distances.
4. Adjust coordinates of each point in the direction that reduces stress.
5. Repeat steps 2 through 4 until stress can't be minimized further.



Multidimensional scaling (MDS)



○ GSE10072
▽ GSE7670
◇ GSE31547
× GSE19804
+ GSE19188

△ GSE18842
● Lung cancer
● Control
● Unknown

Taminau et al. (2014) ISRN Bioinform.

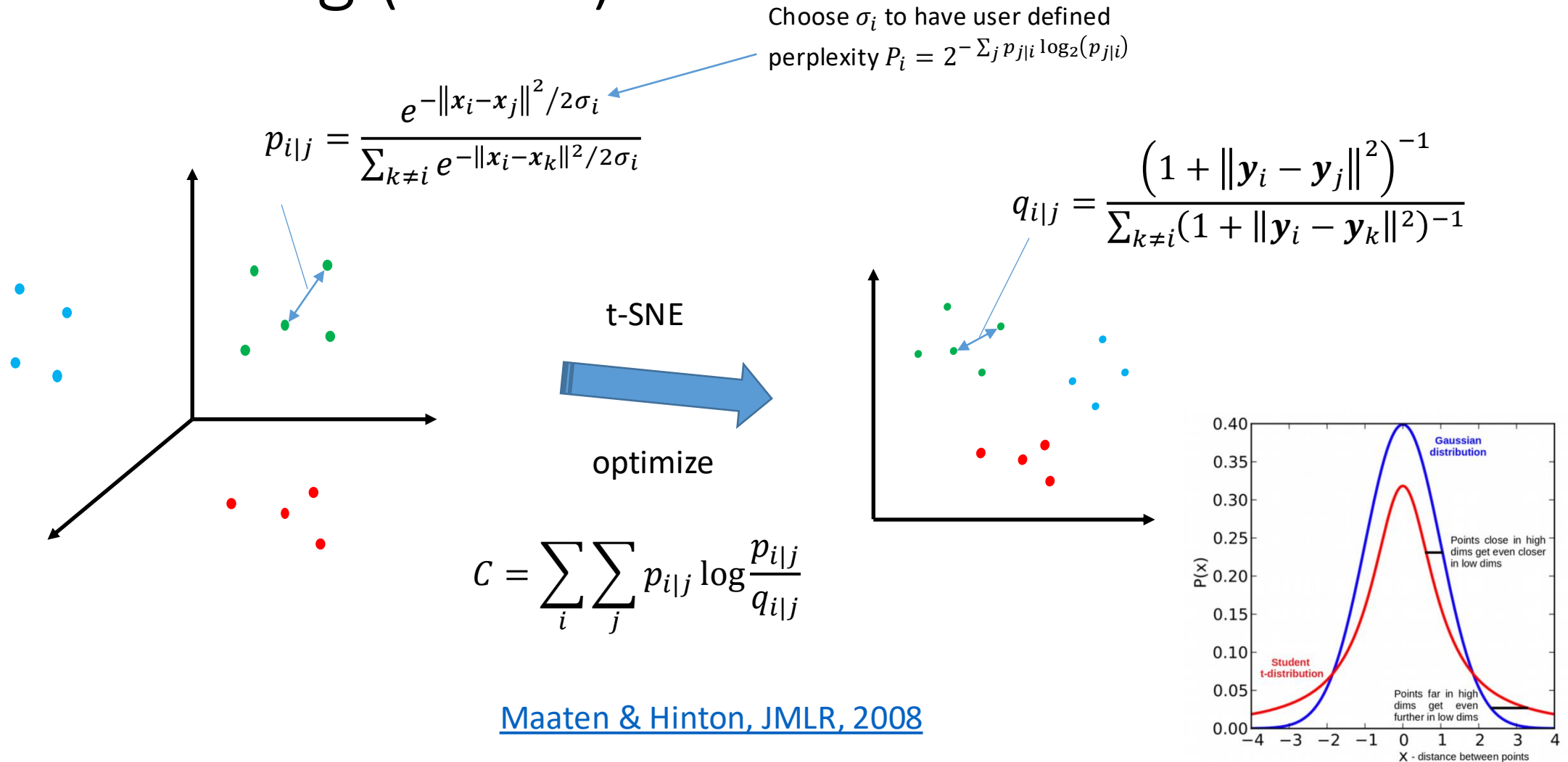
t-distributed Stochastic Neighbor Embedding (t-SNE)

- unsupervised, non-linear technique primarily used for data exploration and visualizing high-dimensional data
- In concept similar to MDS, but with improved distance metrics
- t-SNE differs from PCA by preserving only small pairwise distances or local similarities whereas PCA is concerned with preserving large pairwise distances to maximize variance
- maps a set of high-dimensional points to two dimensions, such that close neighbours remain close and distant points remain distant

t-distributed Stochastic Neighbor Embedding (t-SNE)

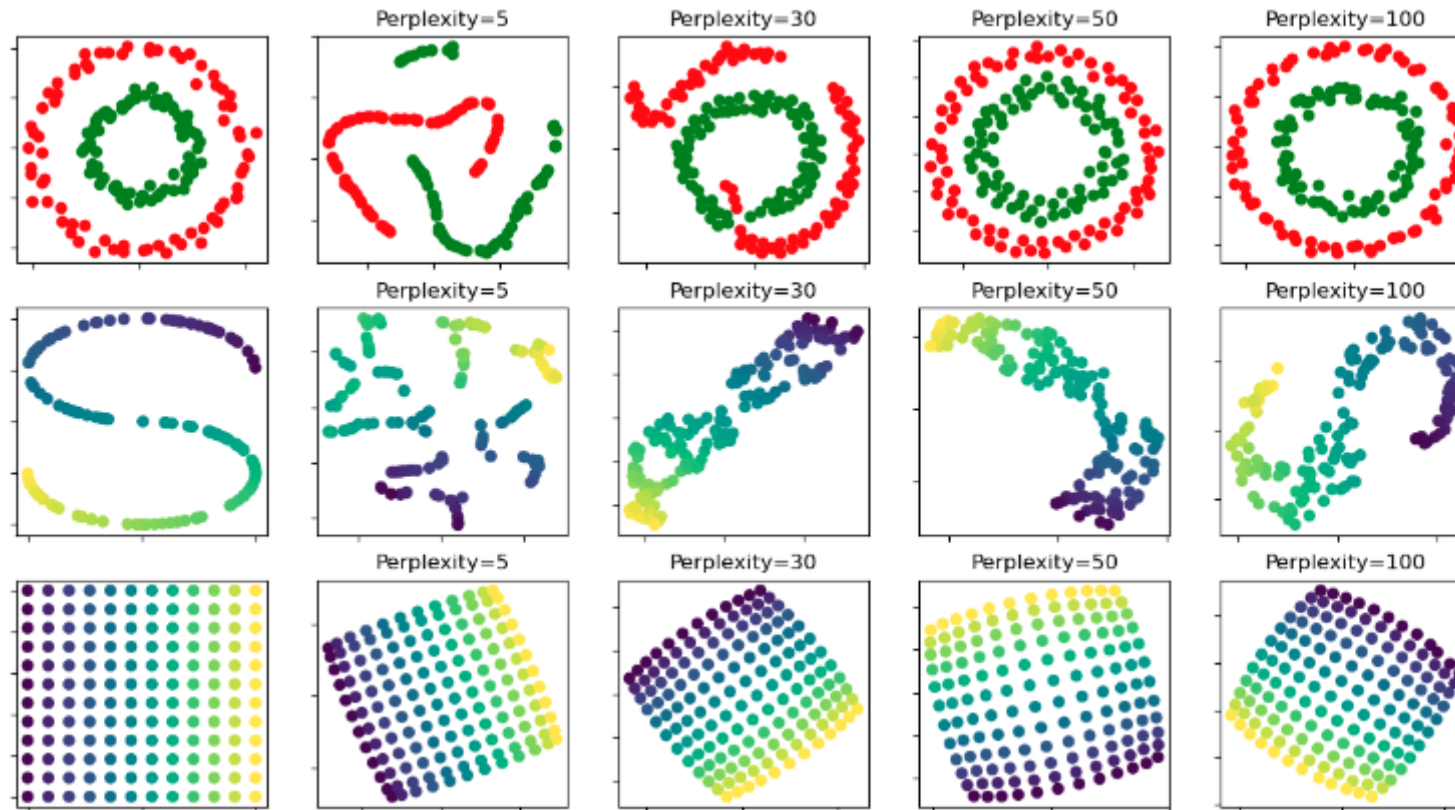
- Informally, the algorithm places all points on the 2D plane, initially at random positions, and lets them interact as if they were physical particles.
- The interaction is governed by two laws: first, all points are repelled from each other; second, each point is attracted to its nearest neighbours
- The most important parameter of t-SNE, called perplexity, determines how many of its nearest neighbours each point is attracted to.
- t-SNE prioritizes local cluster structure within the data. It does not reflect global distances between clusters.

t-distributed Stochastic Neighborhood Embedding (t-SNE)



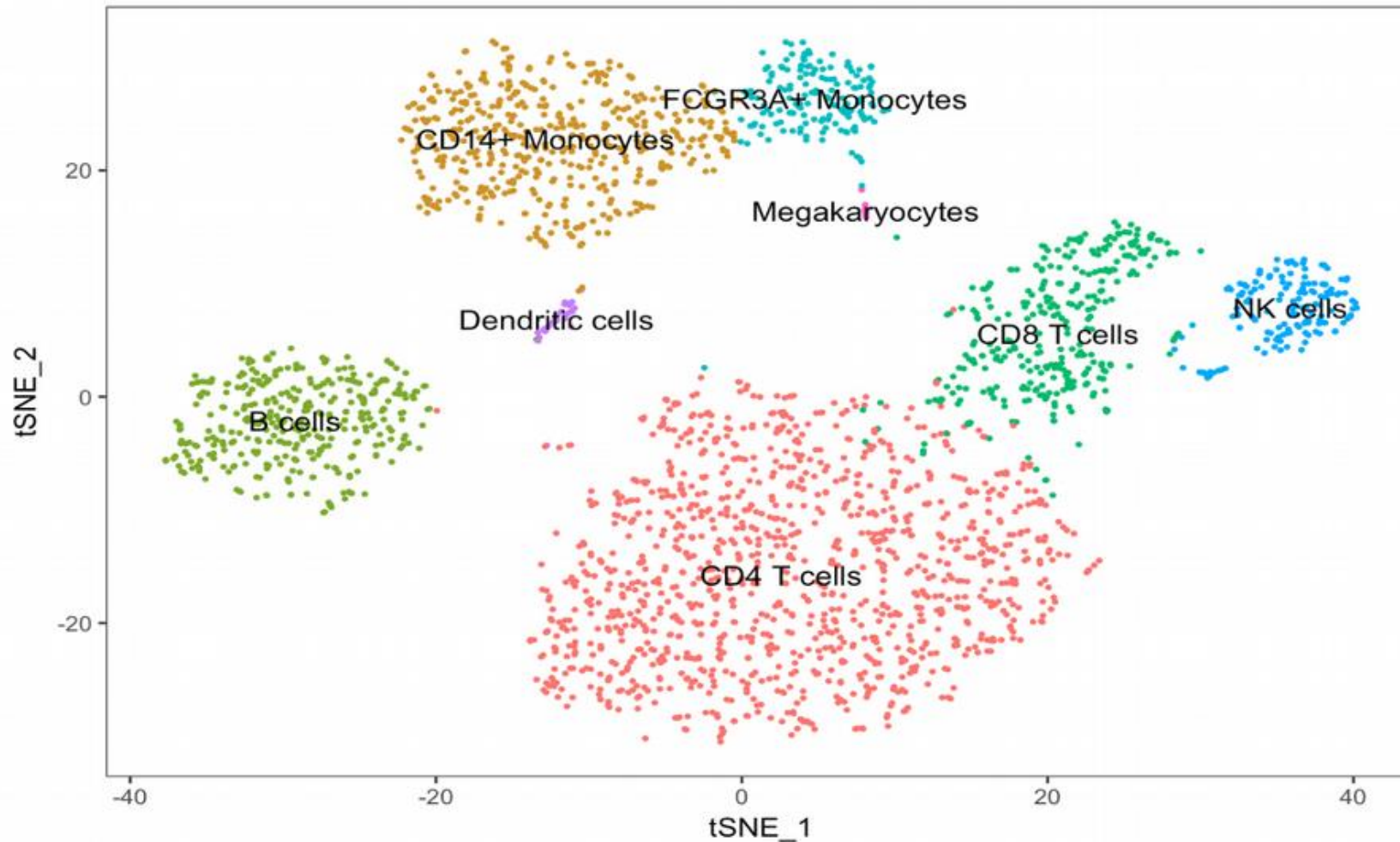
t-SNE Perplexity

https://scikit-learn.org/stable/auto_examples/manifold/plot_t_sne_perplexity.html



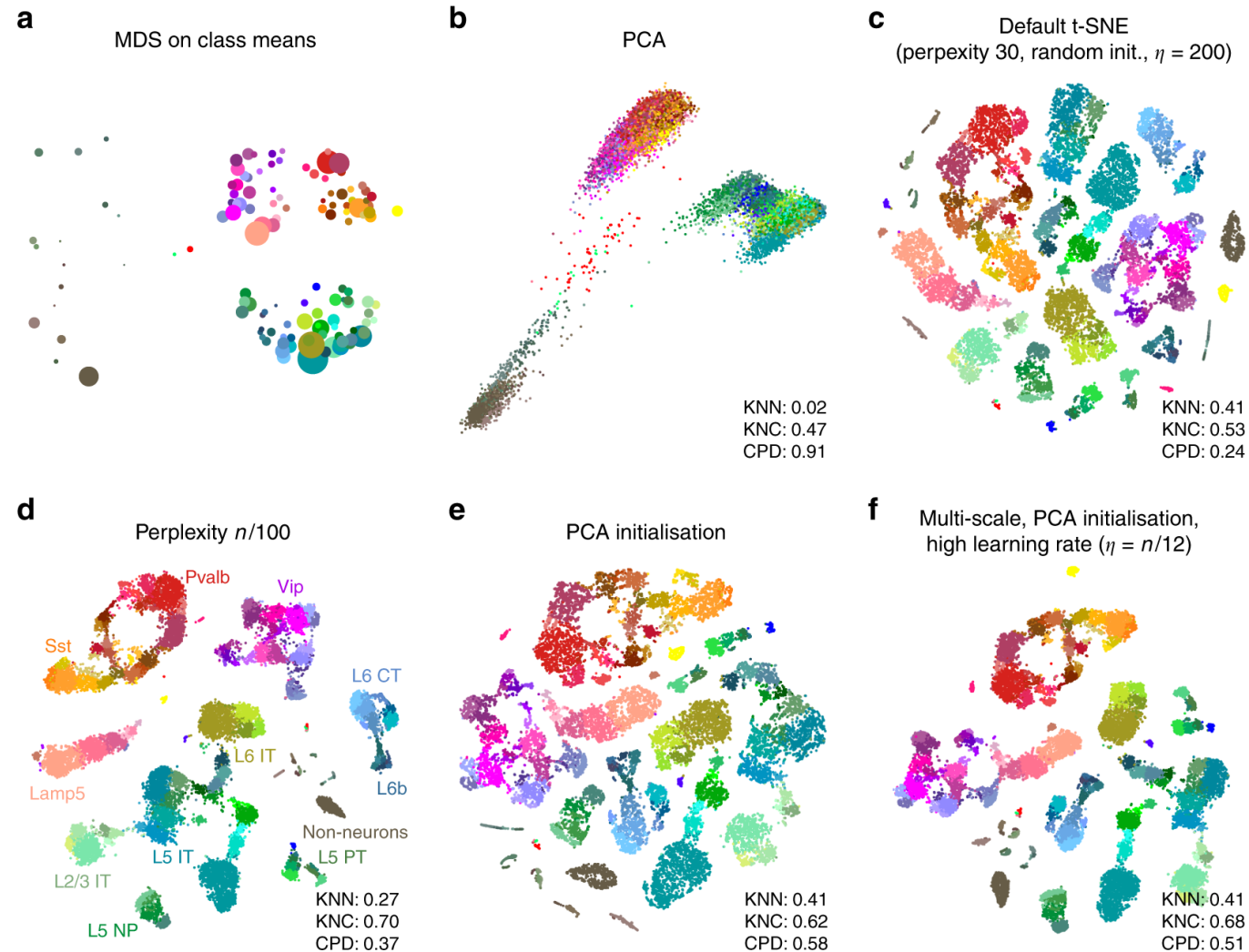
The larger the perplexity, the more neighbours are considered and the more the global structure is preserved

t-distributed Stochastic Neighbor Embedding (t-SNE)



<https://towardsdatascience.com/how-to-tune-hyperparameters-of-tsne-7c0596a18868>

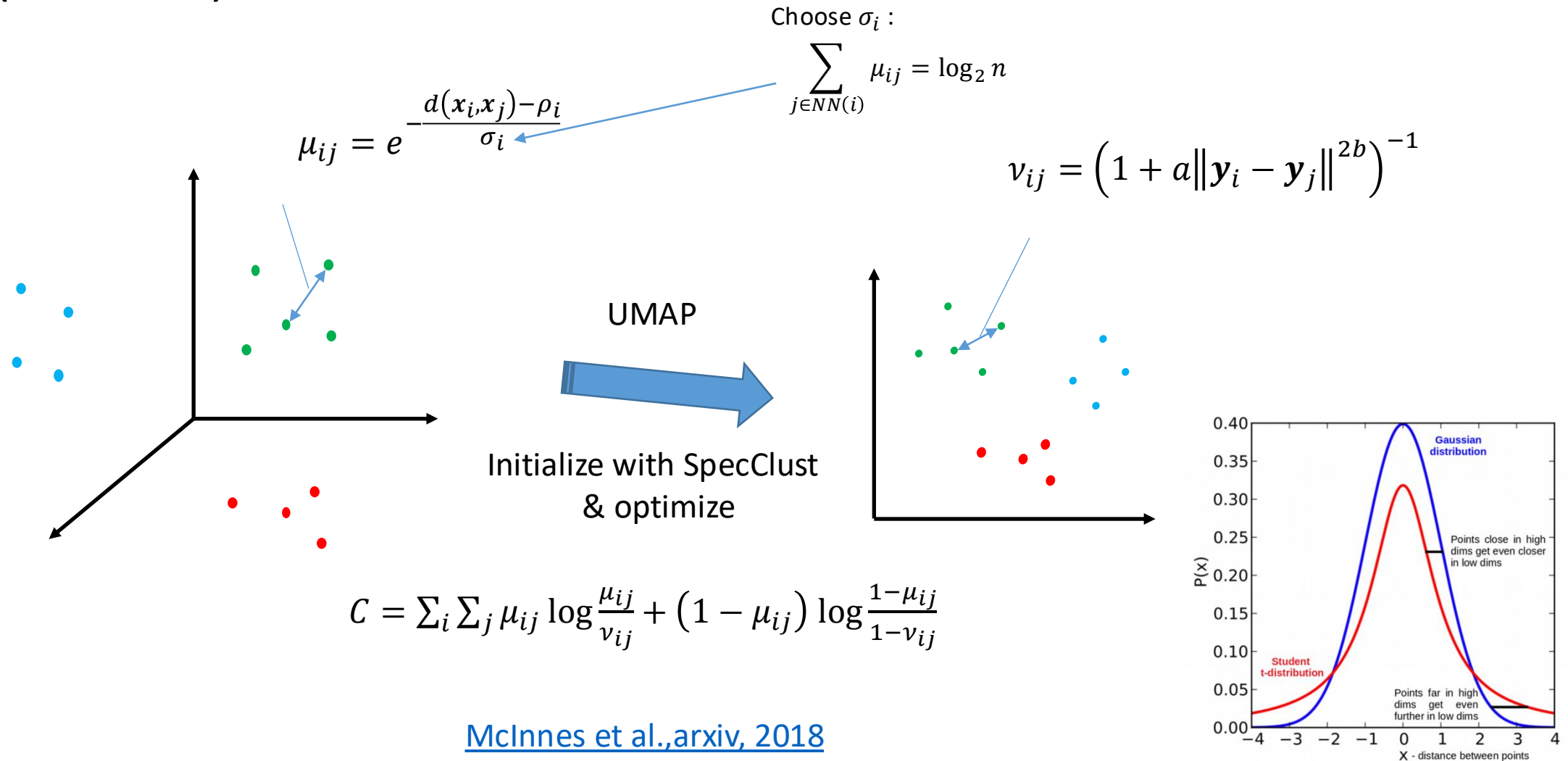
t-distributed Stochastic Neighborhood Embedding (t-SNE)



Single cell analysis

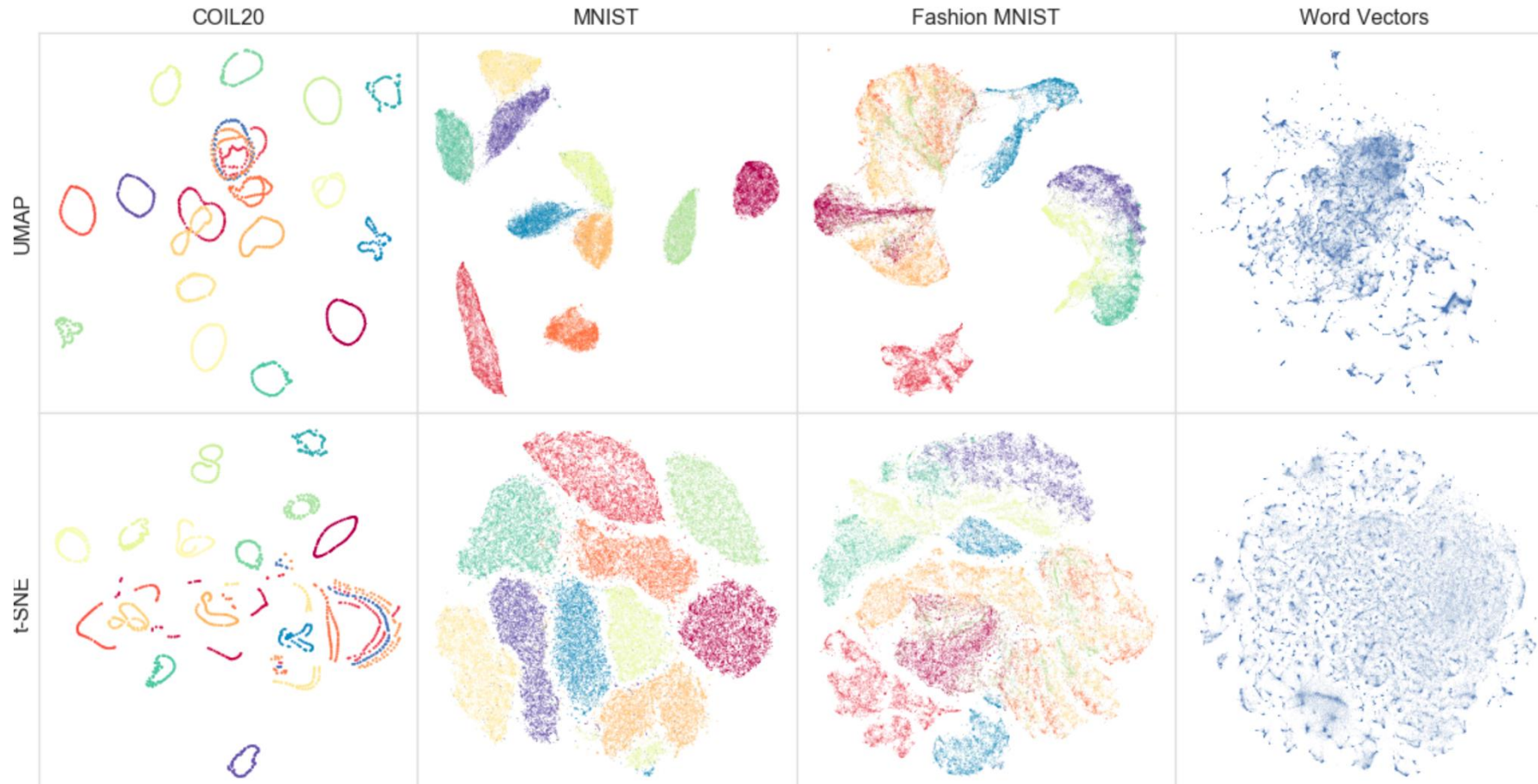
[Kobab & Berens, Nature Comm., 2019](#)

Uniform manifold approximation and projection (UMAP)



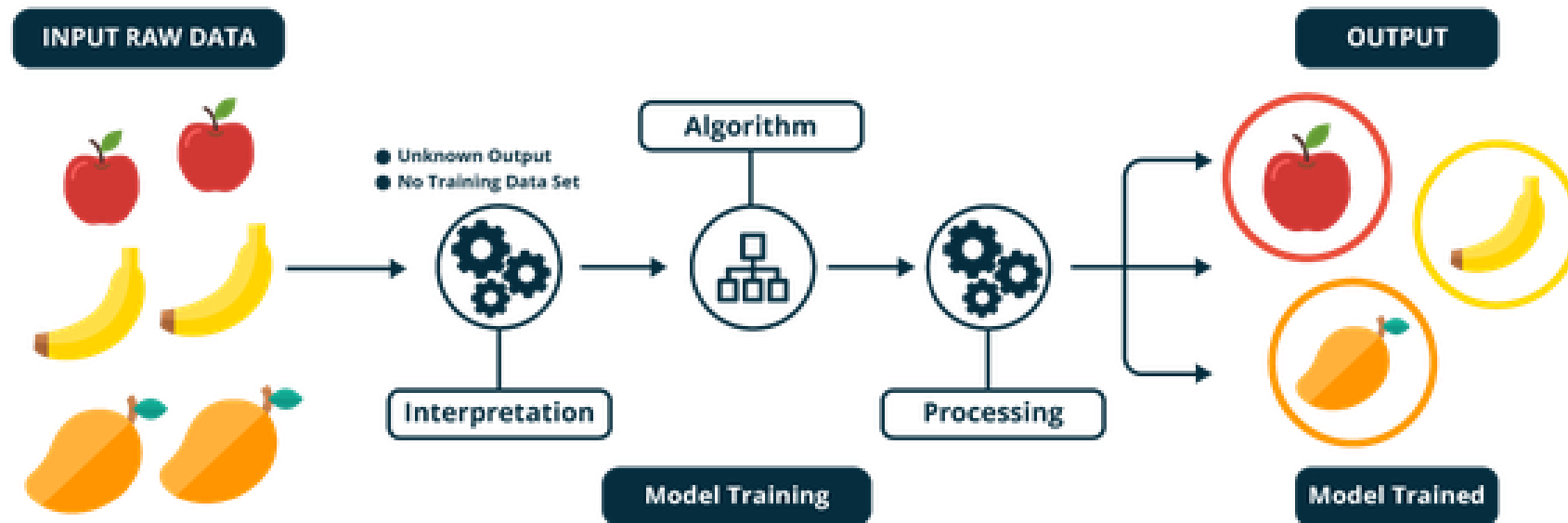
T-SNE versus UMAP

McInnes et al., 2018



[Nikolay Oskolkov's blog](#)

Unsupervised learning



Unsupervised machine learning algorithms

- In contrast to supervised machine learning algorithms, they:
 - are applied when the information used to train is neither classified nor labeled.
 - can infer a function to describe a hidden structure from unlabeled data.
 - do not figure out the right output, but explore the data and can draw inferences from datasets to describe hidden structures from unlabeled data.
- The goal for unsupervised learning is to model the underlying structure or distribution in the data in order to learn more about the data.

Clustering

- Pattern detection: discover hidden patterns in your data
- Compression: replace all cluster members by one representative
- Data augmentation: replace noisy feature vectors by the cluster consensus vector
- Dimensionality reduction: replace a feature vector by cluster similarities
- Outlier detection: detect feature vectors far away from cluster centers
- Semisupervised learning: assign cluster labels to unlabeled cluster members

K-means Clustering

1. Choose K points from $(x_1, x_2 \dots x_N)$ as initial values for the cluster centers μ_k .
These initial points should be spread out over the dataset.
2. Assignment step: assign each $x_i, i = 1 \dots N$, to the cluster with the closest center:

$$c_i = \underset{k}{\operatorname{argmin}} \|x_i - \mu_k\|^2$$

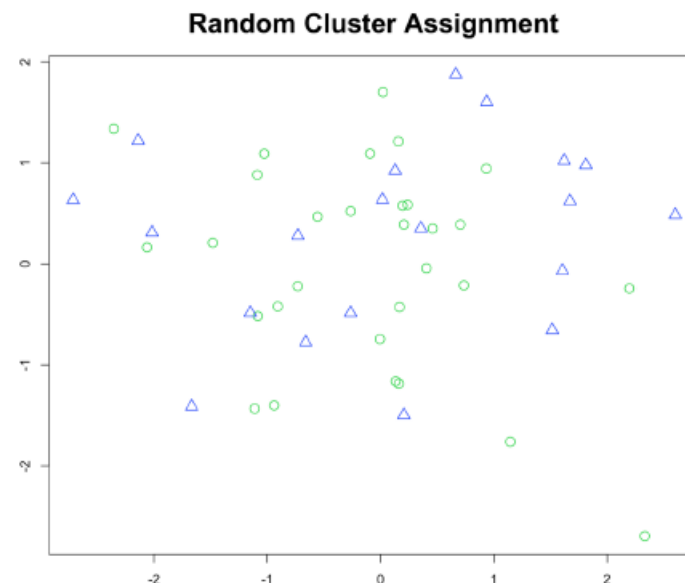
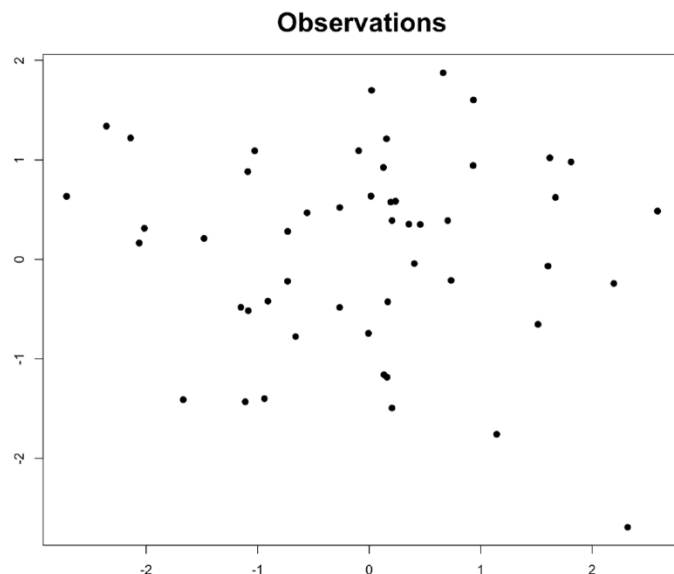
3. Update step: recalculate new μ_k based on cluster assignments $C_k = \{i | c_i = k\}$:

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$$

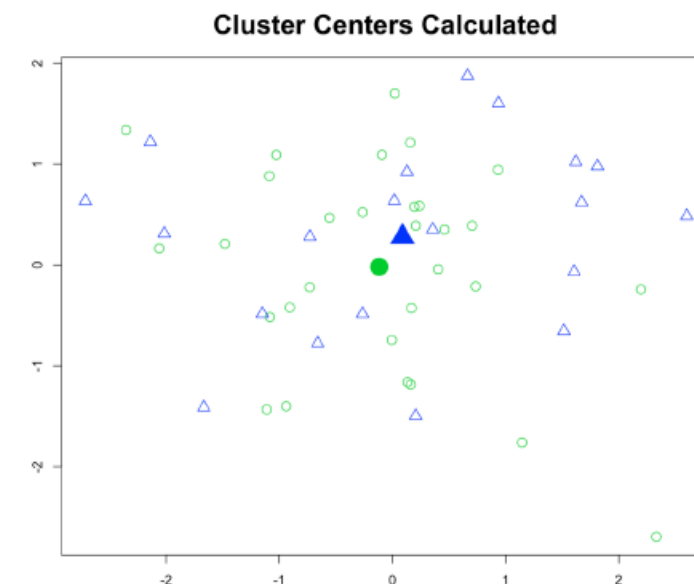
4. Iterate steps 2 & 3 until convergence

[K-means at work](#)

K-means

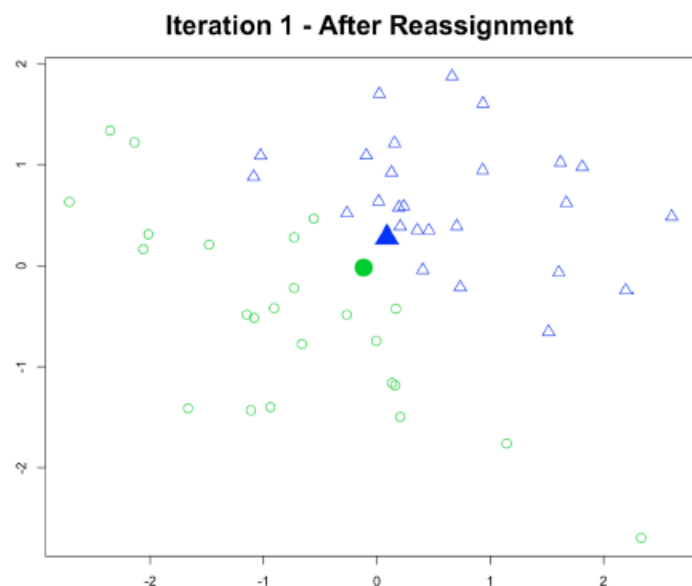


K-means initially assigns each point to one of two clusters.

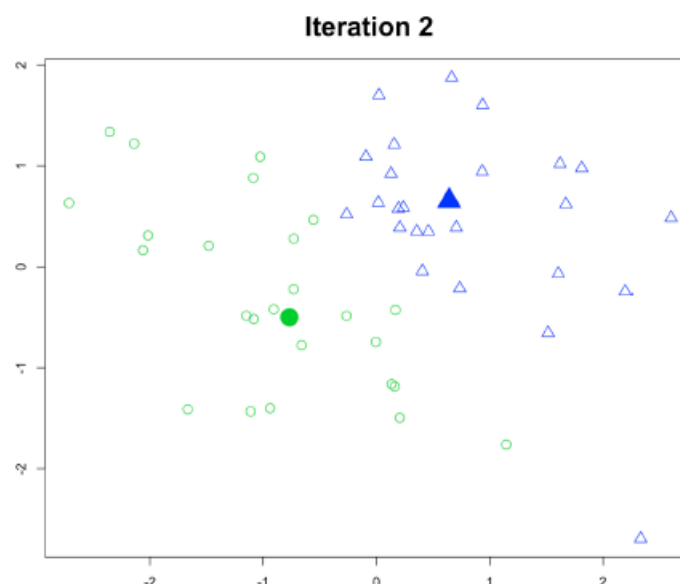


The centers of each subgroup is calculated as the average position of all the points in that group.

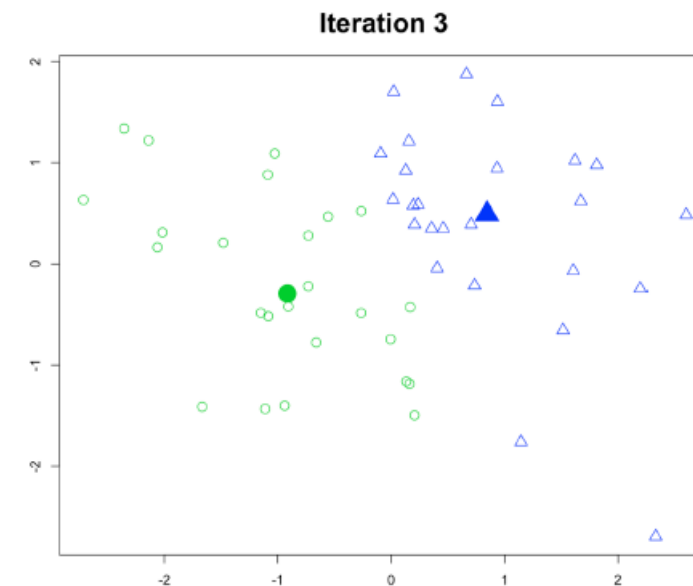
K-means



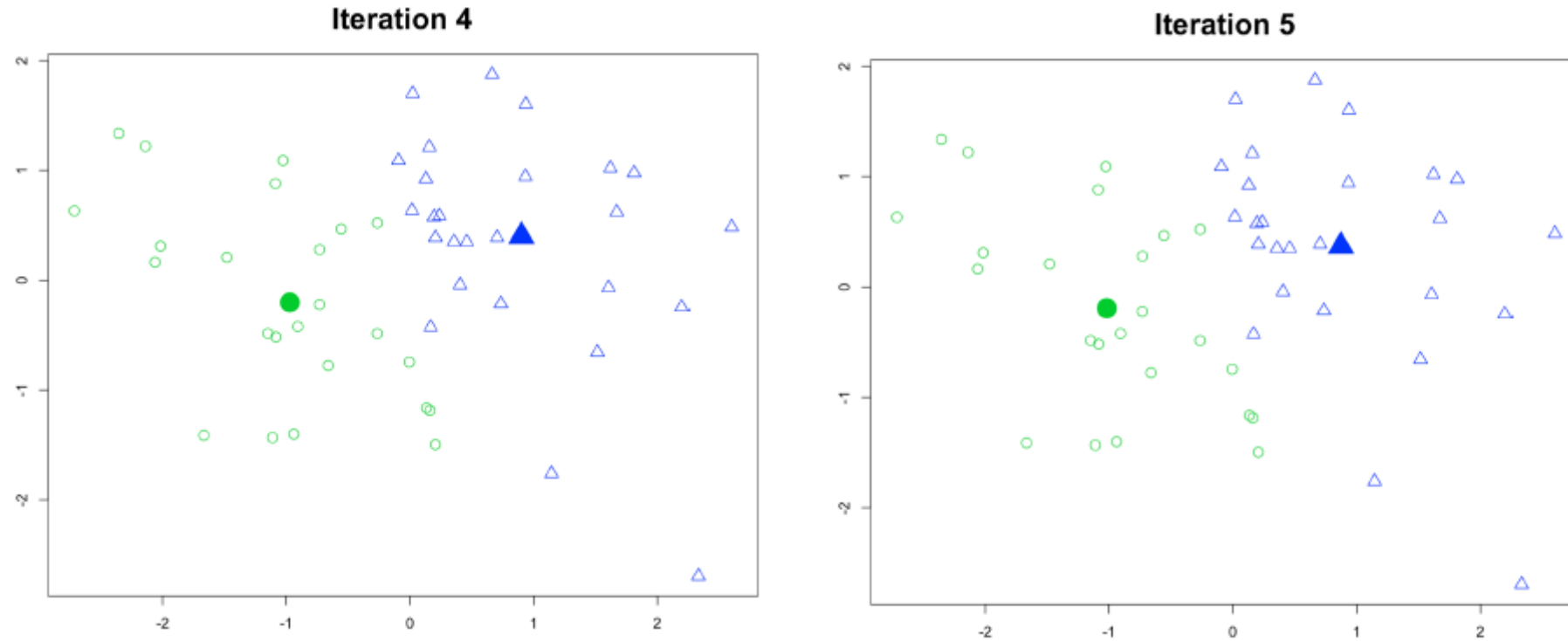
All the points closest to the solid blue triangle are assigned to the corresponding cluster while those close to the solid green circle are assigned to the other cluster.



New cluster centers are calculated and each observation reassigned to the cluster of the nearest center.



K-means

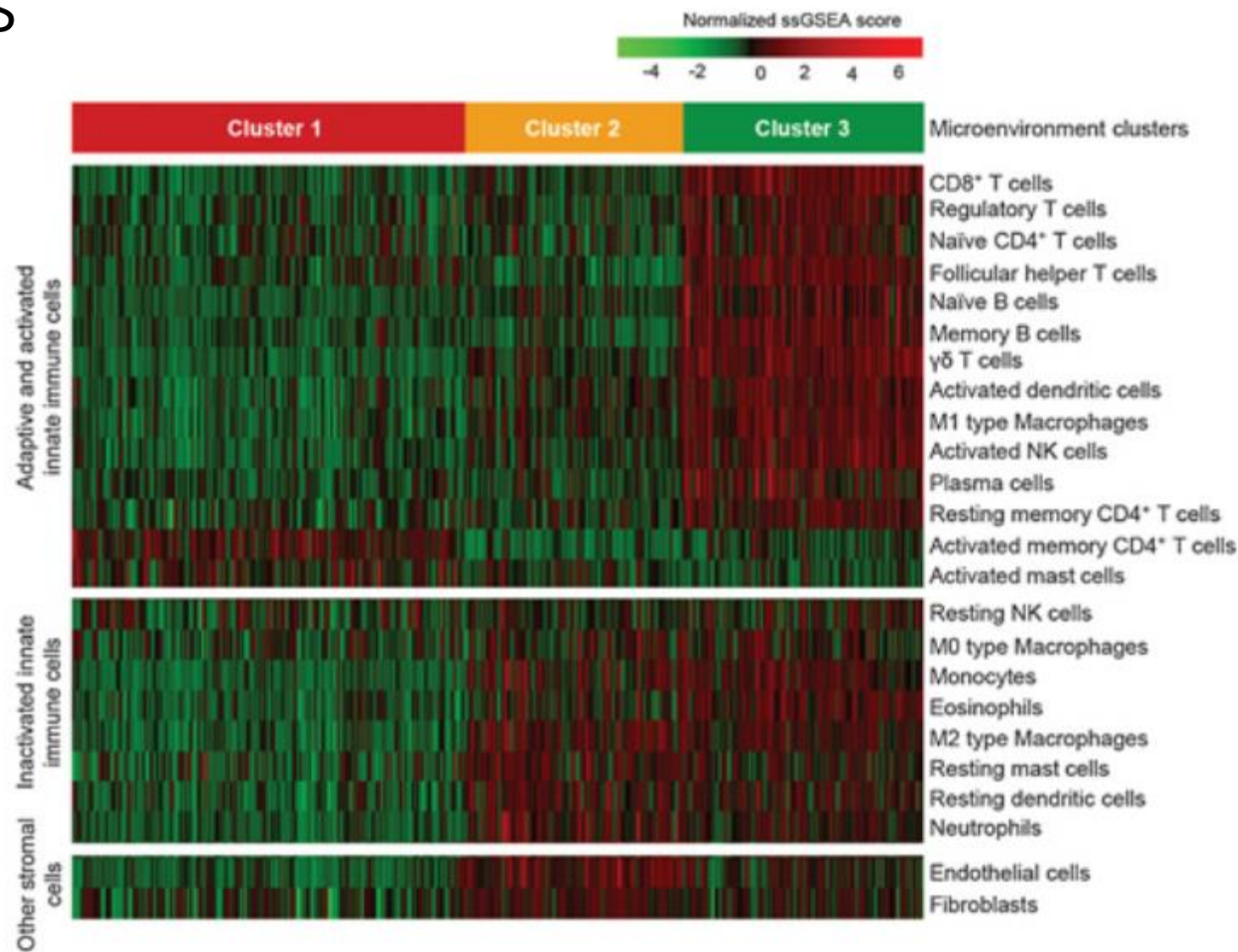


The iterative process stops after a given number of iterations or when the assignment of data points to clusters isn't changing.

K-means

- Advantage: easy to implement and fast and efficient in terms of computational cost
- Disadvantages:
 - Choice of K
 - Initial seeds have a strong impact on final results
 - Clustering data of varying sizes and density
 - Clustering outliers

K-means



Xiao et al. (2019) Precision Medicine and Imaging

Isabelle Dupanloup, BCF

Gaussian Mixtures

- The data is modelled as a mixtures of n Gaussian distributions:

$$p(\mathbf{x}|\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, k = 1 \dots K) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- The parameters $\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ are determined by the EM algorithm:

- Expectation step: calculate for each $\mathbf{x}_i$ the probability z_{ik} to belong to cluster k

$$z_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}$$

- Maximization step: recalculate new $\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ based on z_{ik} :

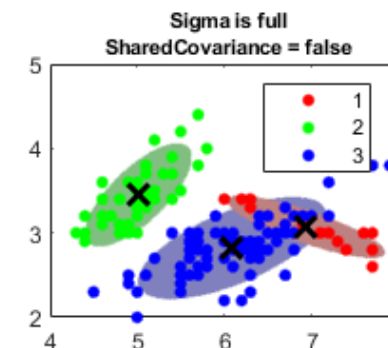
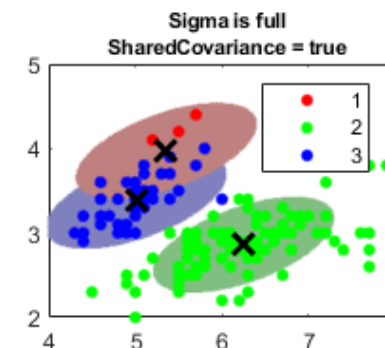
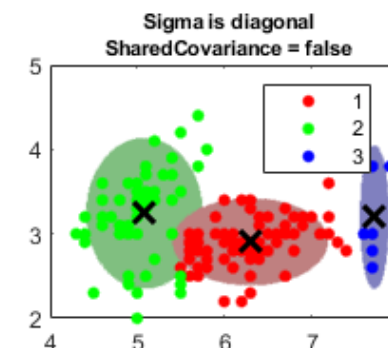
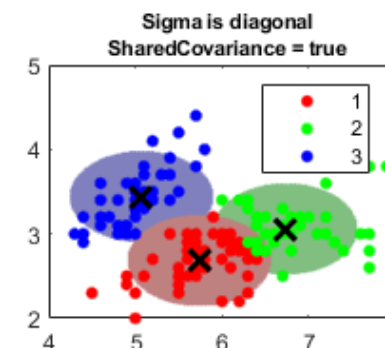
$$n_k = \sum_{i=1}^N z_{ik}, \pi_k = \frac{n_k}{N}, \boldsymbol{\mu}_k = \frac{1}{n_k} \sum_{i=1}^N z_{ik} \mathbf{x}_i,$$

$$\boldsymbol{\Sigma}_k = \frac{1}{n_k} \sum_{i=1}^N z_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T$$

- Iterate steps 2 & 3 until convergence
- $Z_i = \underset{k}{\operatorname{argmax}} z_{ik}$: cluster assignment for $\mathbf{x}_i$

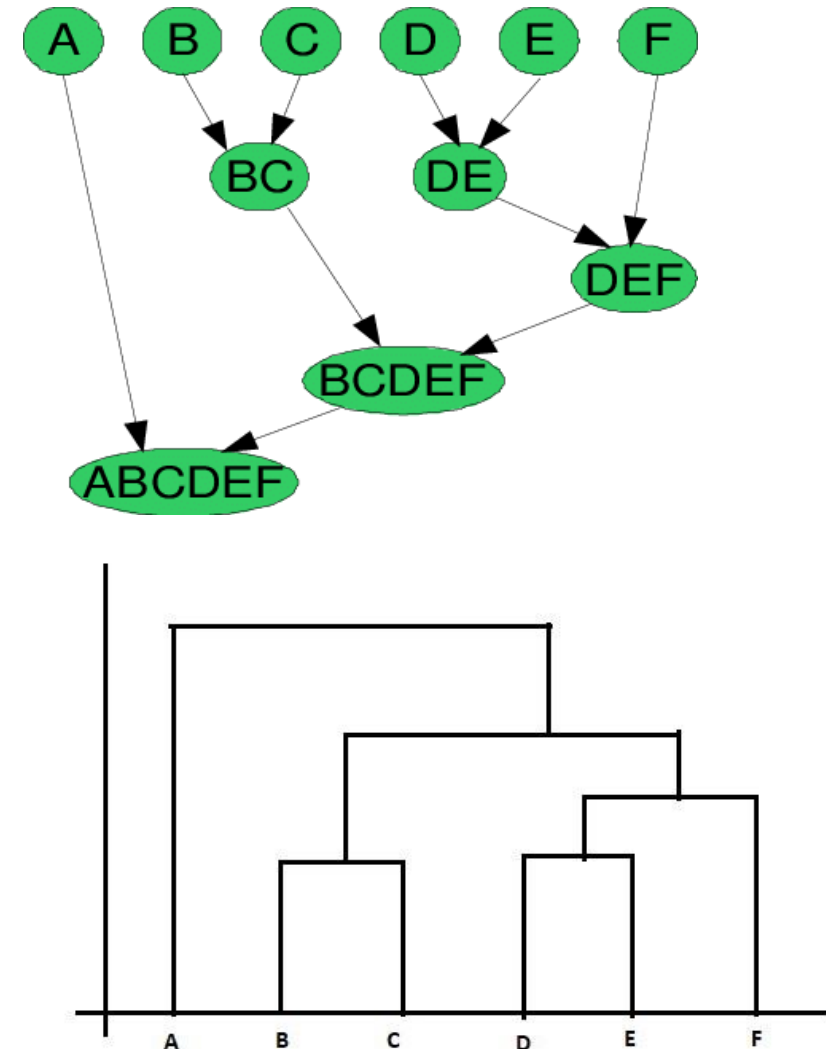
[Frailey & Raftery, The Computer Journal, 1998](#)

Models $\mathcal{M}$

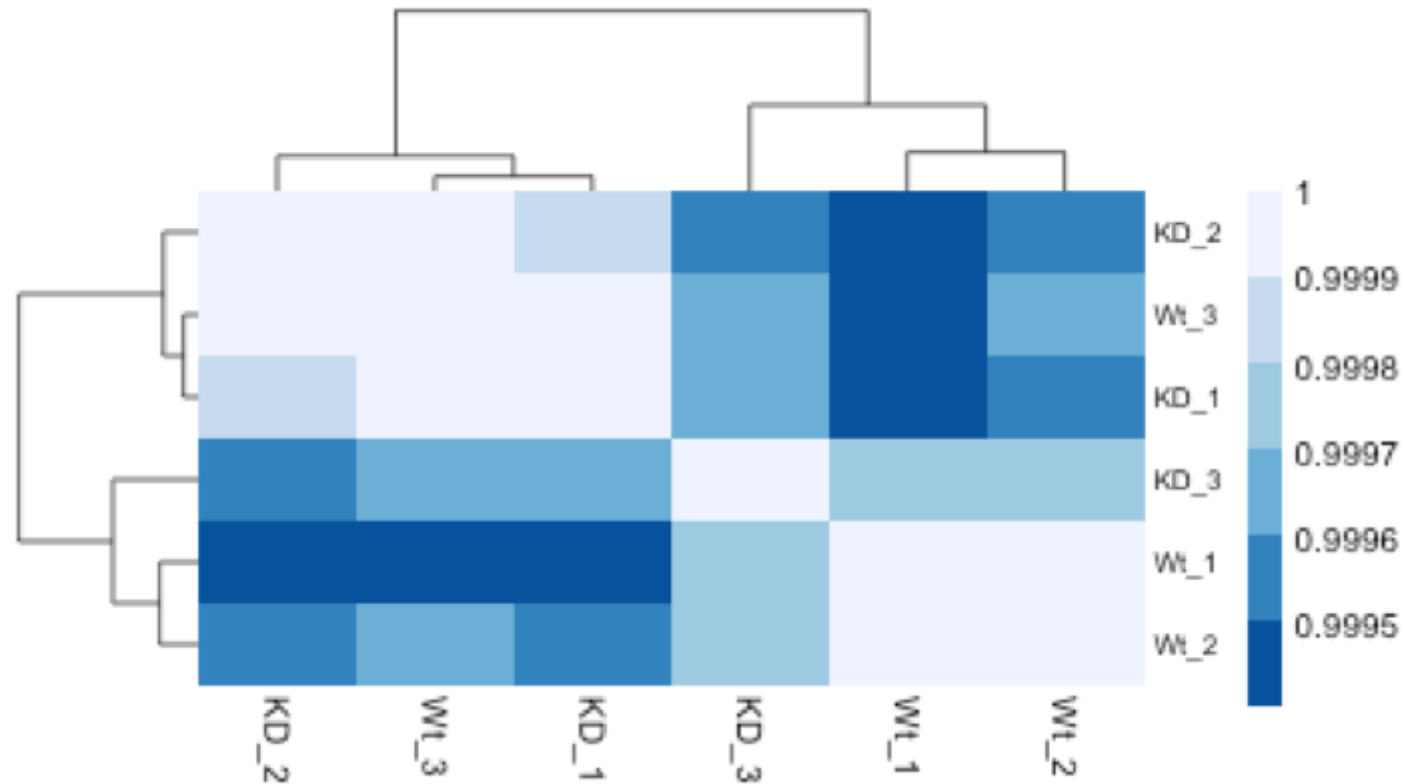


Hierarchical clustering

- 2 types: agglomerative or divisive
- Agglomerative:
 1. Compute the proximity matrix
 2. Let each data point be a cluster
 3. Repeat: Merge the two closest clusters and update the proximity matrix
 4. Until only a single cluster remains
- Results can be visualized using a dendrogram



Hierarchical clustering



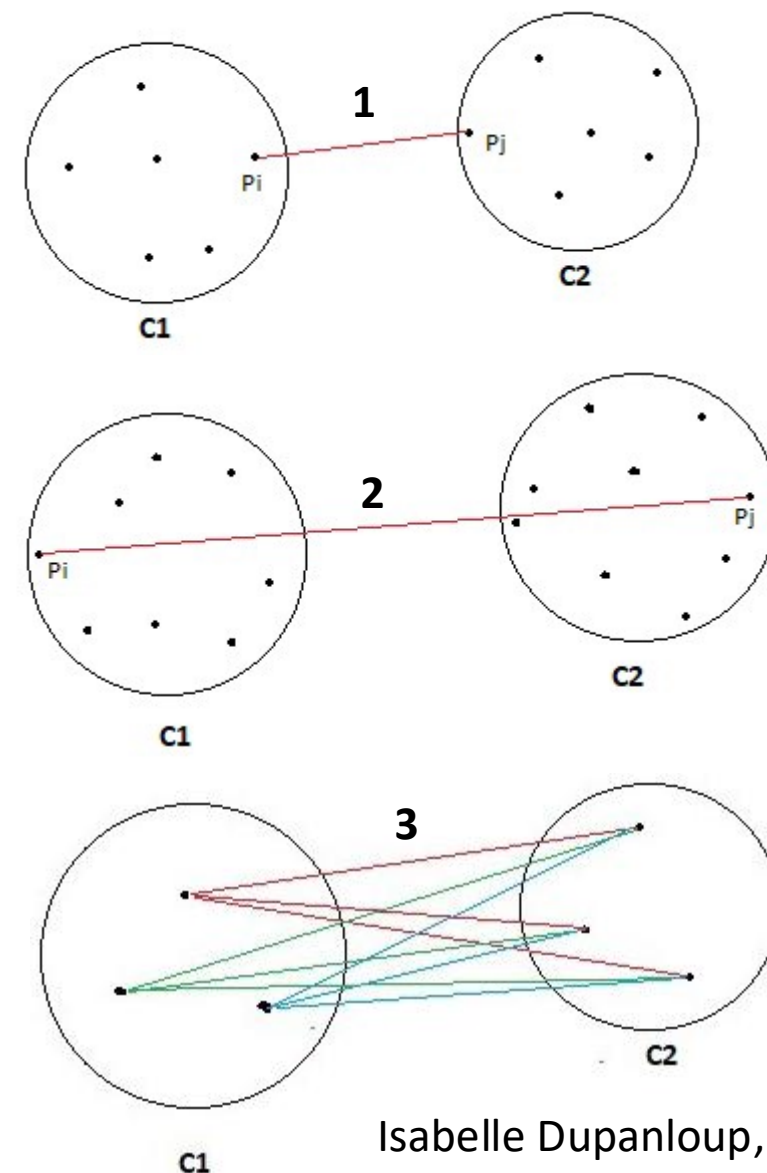
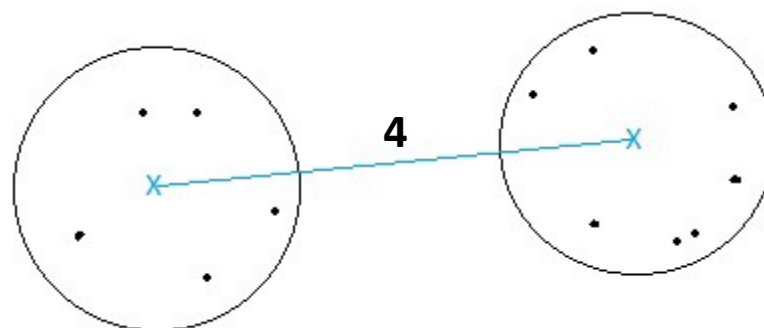
https://hbctraining.github.io/DGE_workshop_salmon/lessons/03_DGE_QC_analysis.html

Agglomerative Hierarchical Clustering

1. Initialize clusters $C_i = \{x_i\}$, $i = 1 \dots N$, $id = \{1, 2, \dots N\}$ and calculate distances $d_{ij} = d(C_i, C_j)$
2. Merge closest clusters: $C_i = C_i \cup C_j$, if $d_{ij} = \min_{(l,k)} d_{lk}$, drop j : $id = id \setminus j$ and recalculate $d_{ik} = d(C_i, C_k)$, $k \in id \setminus i$
3. Repeat 2 until $|id| = K$ or $d_{ik} \geq \varepsilon, \forall i, k \in id$

Hierarchical clustering

- **Divisive:** start with all the data points as a single cluster and in each iteration, we separate the data points from the cluster which are not similar
- **Similarity between clusters (choice of linkage)**
 1. Min
 2. Max
 3. Group Average
 4. Distance Between Centroids
 5. Ward's Method (similar to Group Average)



Agglomerative Hierarchical Clustering

Names	Formula
Maximum or complete-linkage clustering	$\max \{ d(a, b) : a \in A, b \in B \}.$
Minimum or single-linkage clustering	$\min \{ d(a, b) : a \in A, b \in B \}.$
Unweighted average linkage clustering (or UPGMA)	$\frac{1}{ A \cdot B } \sum_{a \in A} \sum_{b \in B} d(a, b).$
Weighted average linkage clustering (or WPGMA)	$d(i \cup j, k) = \frac{d(i, k) + d(j, k)}{2}.$
Centroid linkage clustering, or UPGMC	$\ c_s - c_t\ $ where c_s and c_t are the centroids of clusters s and t , respectively.
Minimum energy clustering	$\frac{2}{nm} \sum_{i,j=1}^{n,m} \ a_i - b_j\ _2 - \frac{1}{n^2} \sum_{i,j=1}^n \ a_i - a_j\ _2 - \frac{1}{m^2} \sum_{i,j=1}^m \ b_i - b_j\ _2$

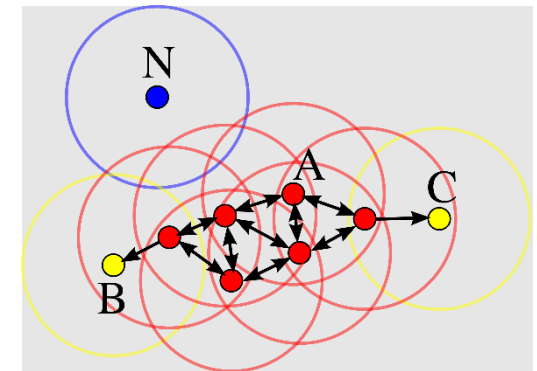
where d is the chosen metric. Other linkage criteria include:

- The sum of all intra-cluster variance.
- The increase in variance for the cluster being merged ([Ward's criterion](#)).^[8]
- The probability that candidate clusters spawn from the same distribution function (V-linkage).
- The product of in-degree and out-degree on a k-nearest-neighbour graph (graph degree linkage).^[9]
- The increment of some cluster descriptor (i.e., a quantity defined for measuring the quality of a cluster) after merging two clusters.[!]

https://en.wikipedia.org/wiki/Hierarchical_clustering

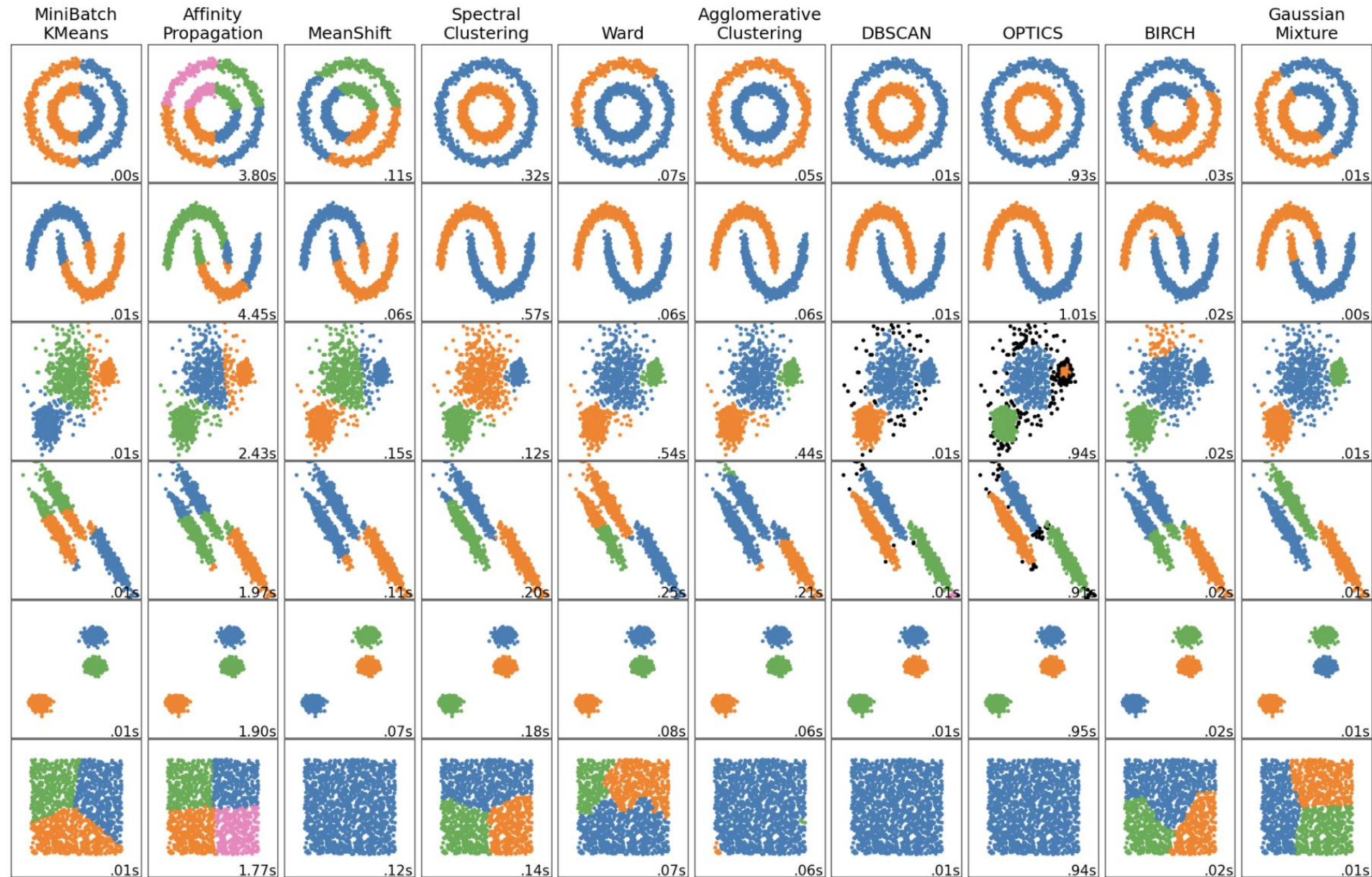
DBSCAN

- DBSCAN uses a minimal distance ε and a minimal number of cluster seed points n as input
- It finds dense regions in the data and uses them as cluster seeds. Then it expands these clusters and adds points if they are within ε distance to a cluster. All points not within reach of a cluster are marked as outliers.
- DBSCAN can deal with clusters of arbitrary shapes.
- DBSCAN requires clusters of similar density
- Distance based clustering are difficult in high dimensions (curse of dimensionality)



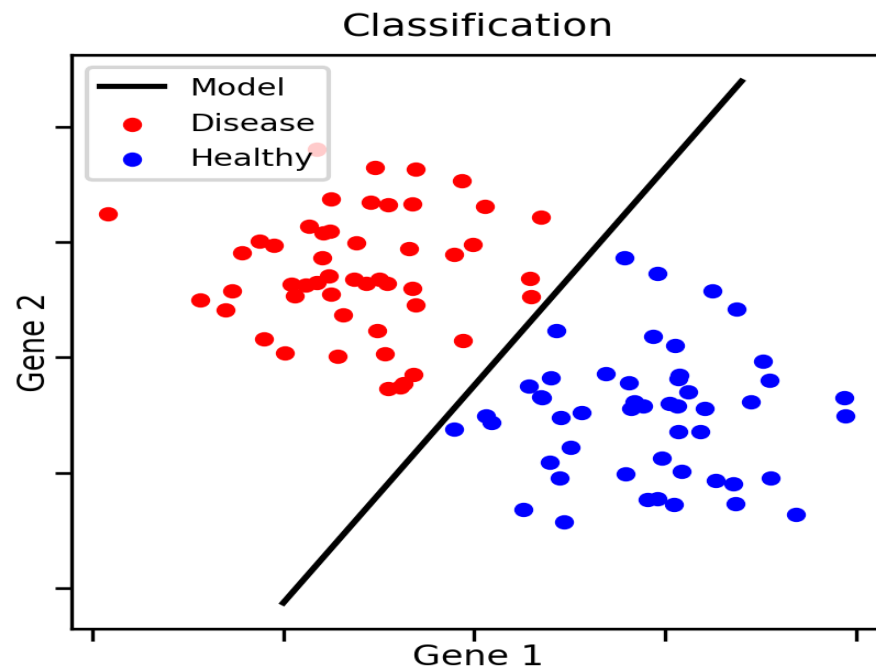
DBSCAN

[sklearn docu](https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html)

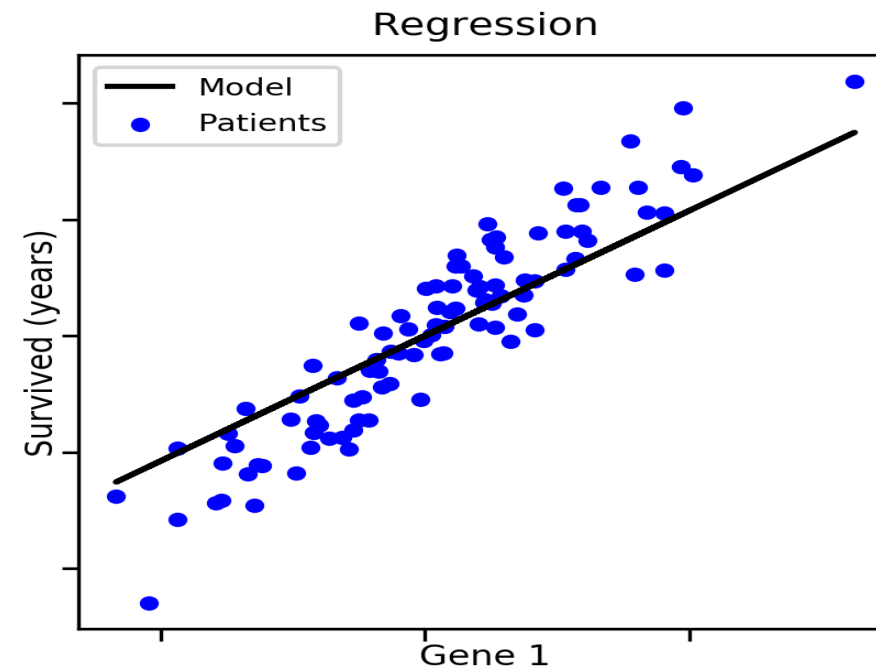


Supervised learning

Fundamentally, classification is about predicting a label and regression is about predicting a quantity.

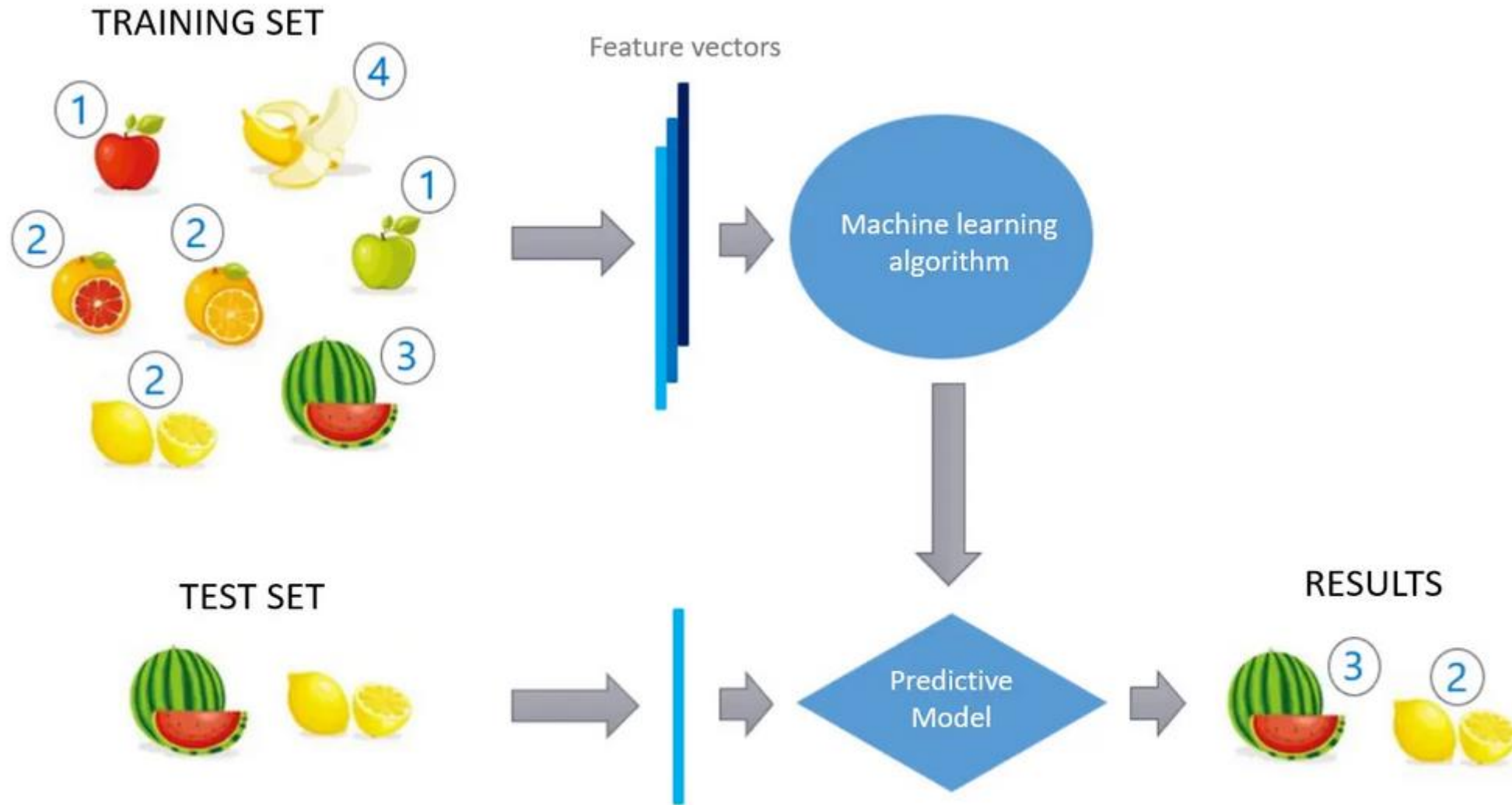


Classification predictive modeling is the task of approximating a mapping function (f) from input variables (X) to discrete output variables (y).

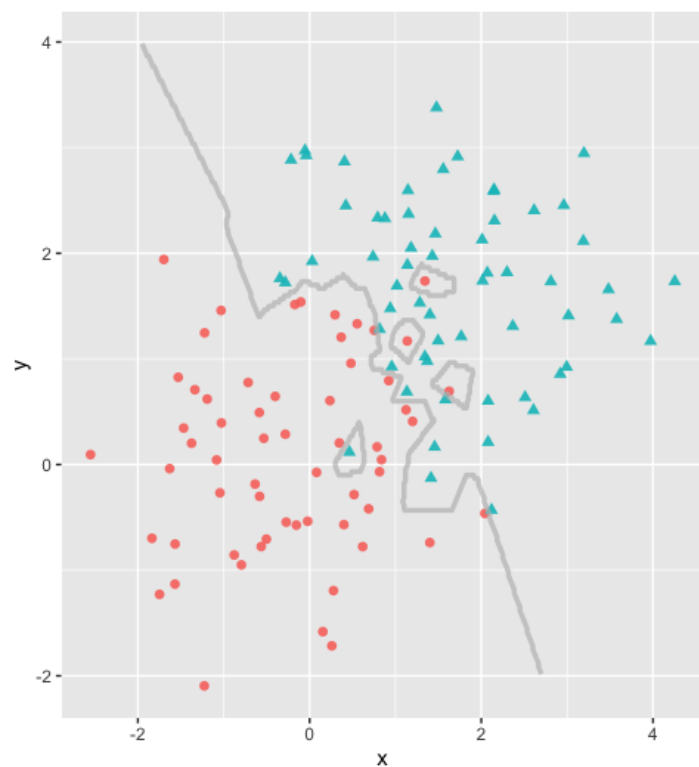


Regression predictive modeling is the task of approximating a mapping function (f) from input variables (X) to a continuous output variable (y).

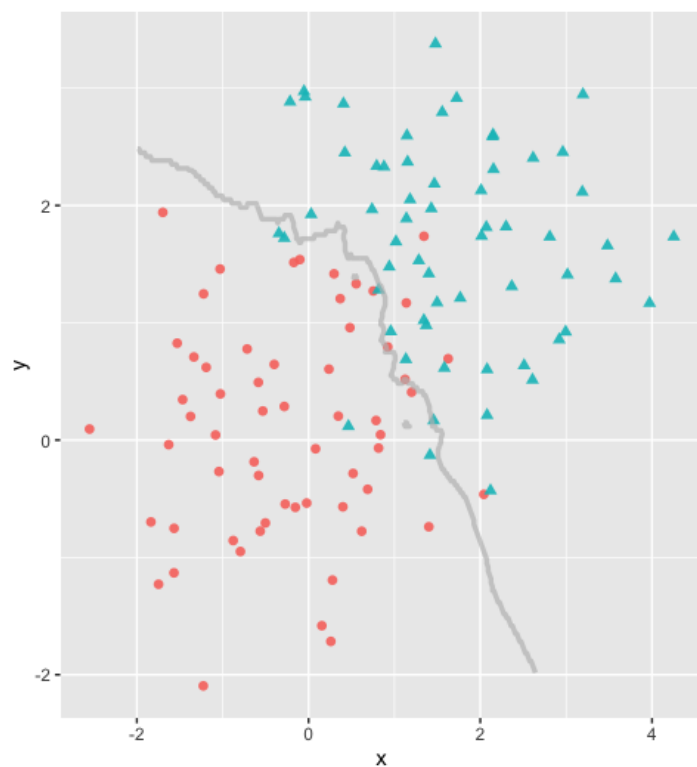
Supervised learning



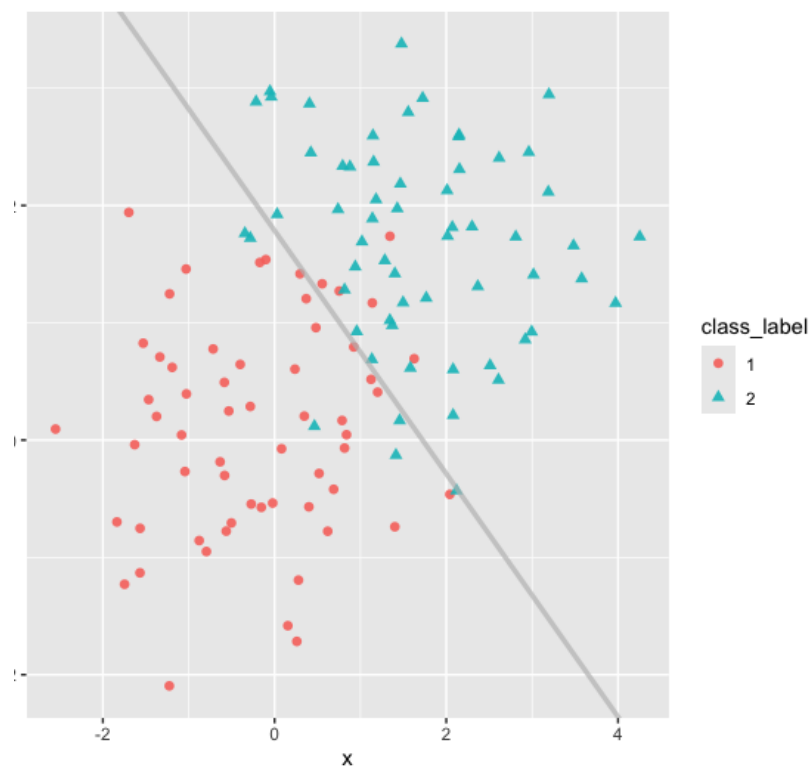
Building a classifier from data: which one?



A



B



C

What is a good classifier?

- Infinite number of models that fit training data equally well
- Model is not defined by training data alone!
- Additional criteria needed:
 - Smoothness
 - Low complexity model (especially for small training data set)
 - A model must accurately predict independent test data, if training and test data are identically distributed

Training, Validation and Test Sets

- Given a dataset, we can always find a classifier that perfectly predicts the outcome (for example the 1-NN classifier) for this dataset.
- The performance on the training data may be too optimistic and cannot be reproduced on the test data. We call this “*overfitting the training data*”.
- However, the purpose of training a classifier is to give us accurate predictions on independent test datasets (generalization).

Supervised learning datasets

- **Training dataset**
 - The subset of the data provided to train the algorithm.
- **Validation dataset**
 - Another disjunct subset of the data used to tune the meta-parameters of a classifier
- **Test dataset**
 - Another disjunct subset or an independent dataset used to test the performance of the classifier



Some More Notations:



Swiss Institute of
Bioinformatics

- In the classification task, we would like to predict the outcome y from the features X .
- We choose a classifier C_ϕ with hyper parameters ϕ and internal parameters θ .
- Hyper parameters (e.g k in $k - NN$, kernel in SVM , learning rate in LR) have to be specified by the user or learned in CV loop outside the classifier.
- The internal parameters θ are learned by the classifier (e.g. the coefficients in LR , or support vector coefficients in SVM).
- The classifier C_ϕ learns those internal parameters θ that are able to predict y from the features X with the lowest error:

$$\theta = \underset{\theta}{\operatorname{argmin}} \mathcal{L}(f_{C,\phi}(X, \theta), y)$$

where the loss function $\mathcal{L}$ measures the error between predicted ($f_{C,\phi}(X, \theta)$) and known (y) outcomes.

Loss or cost functions

$$f_{\mathcal{C},\phi}(\mathbf{X}, \boldsymbol{\theta})$$

Empirical loss:

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\mathcal{C},\phi}(\mathbf{x}_i; \boldsymbol{\theta}), y_i)$$

Square loss:
Regression or
classification

$$\mathcal{L}(f_{\mathcal{C},\phi}(\mathbf{x}_i; \boldsymbol{\theta}), y_i) = (f_{\mathcal{C},\phi}(\mathbf{x}_i; \boldsymbol{\theta}) - y_i)^2$$

Cross entropy loss:
Classification K classes

$$\mathcal{L}(f_{\mathcal{C},\phi}(\mathbf{x}_i; \boldsymbol{\theta}), y_i) = - \sum_{k=1}^K y_{ik} \log(p_k(\mathbf{x}_i; \boldsymbol{\theta}))$$

$$y_{ik} = \begin{cases} 1 & \text{if } y_i = k \\ 0 & \text{if } y_i \neq k \end{cases}; \quad \sum_{k=1}^K p_k(\mathbf{x}_i; \boldsymbol{\theta}) = 1$$

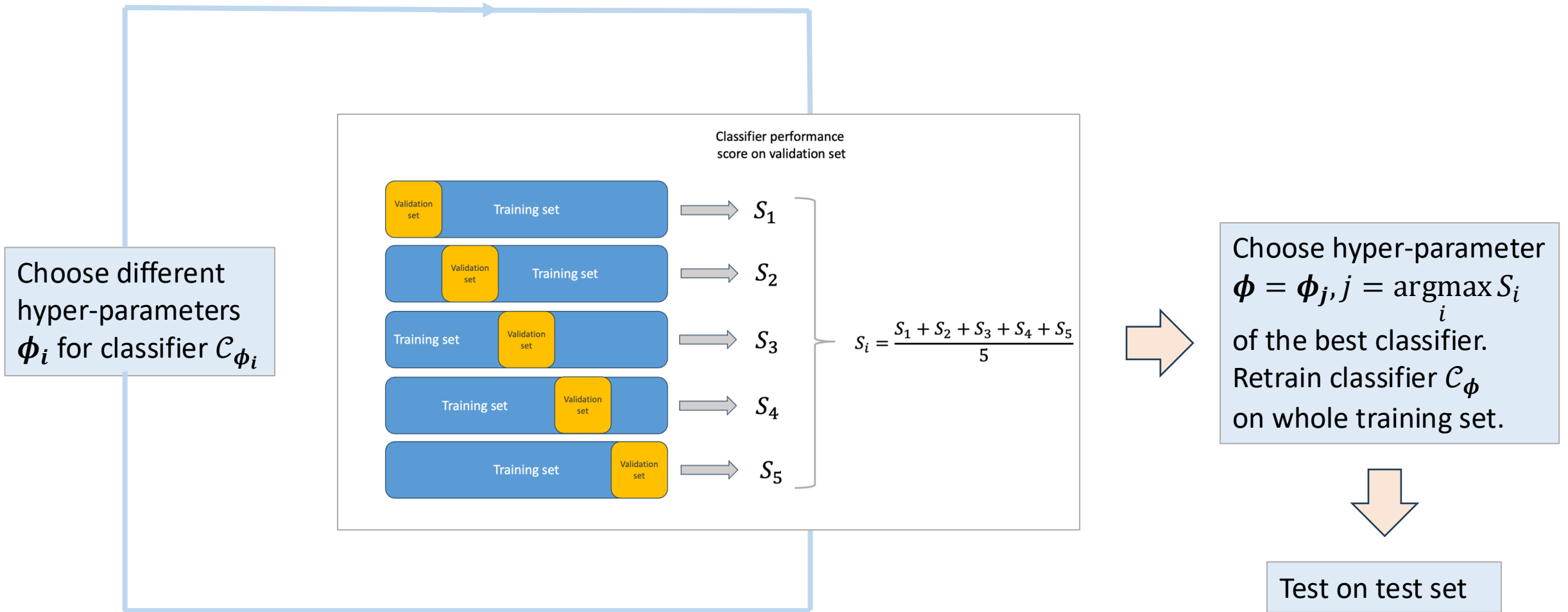
Normalization: $f_k > 0$

$$p_k = f_k / \sum_{k=1}^K f_k$$

Logistic or softmax:

$$p_k = e^{f_k} / \sum_{k=1}^K e^{f_k}$$

Cross validation



Assessing classifier performance

- **TP (True Positives)**: number of positive predictions which are correct
- **FP (False Positives)**: number of positive predictions which are incorrect
- **TN (True Negatives)**: number of negative predictions which are correct
- **FN (False Negatives)**: number of negative predictions which are incorrect

confusion matrix	Label predicted by classifier		P	N
	True Label			
	P		TP	FN
	N		FP	TN

Assessing classifier performance



Swiss Institute of
Bioinformatics

- **Precision or TPR:**

- Probability that positive prediction is correct

$$Prec = TPR = \frac{TP}{TP + FP}$$

- **Specificity:**

- Probability that negative sample is correctly predicted

$$Specificity = \frac{TN}{TN + FP}$$

- **Recall or TPR or Sensitivity:**

- Probability that positive sample is correctly predicted

$$Recall = TPR = Sensitivity = \frac{TP}{TP + FN}$$

- **FPR or FDR:**

- Probability that positive prediction is incorrect

$$FPR = 1 - TPR = \frac{FP}{TN + FP}$$

- **F-measure:**

- Harmonic mean of precision and recall

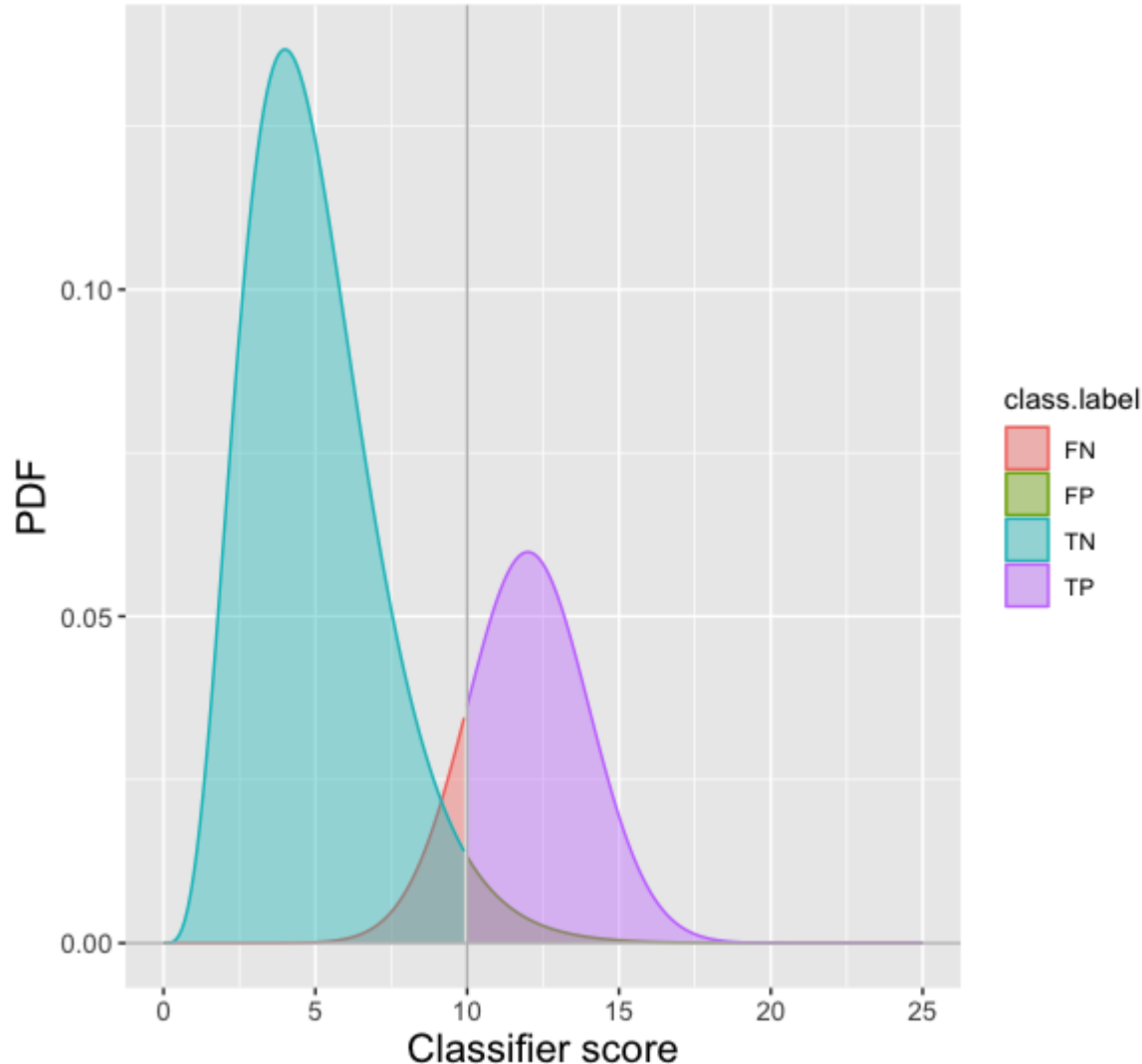
$$F = 2 * \frac{Prec * Recall}{Prec + Recall}$$

- **Accuracy:**

- Fraction of the total predictions that are correct

$$Acc = \frac{TP + TN}{TN + TN + FP + FN}$$

Assessing classifier performance



$$p(\text{score}) = \pi_1 p(\text{score}|1) + \pi_2 p(\text{score}|2)$$

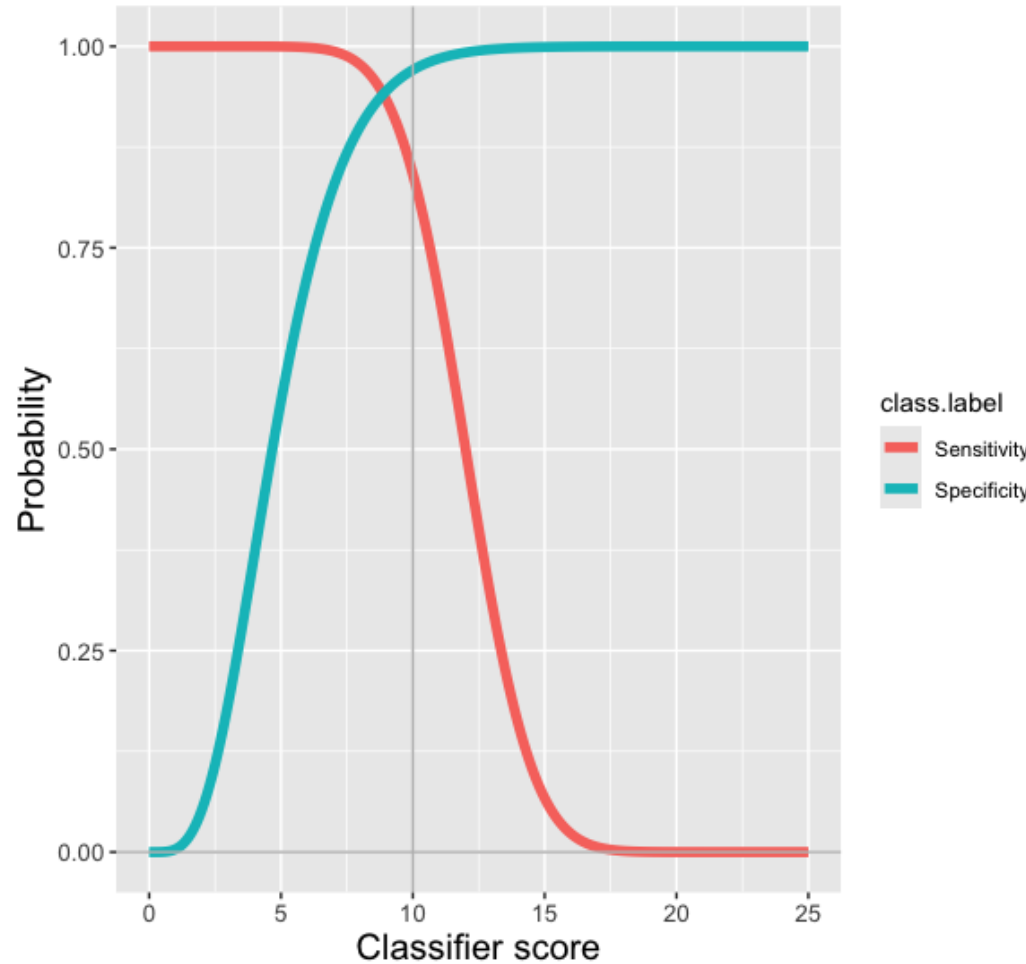
$$\pi_2 = \text{Prevalence} = \frac{P}{P + N}$$

Class imbalance: $\pi_1 \gg \pi_2$ or $\pi_2 \gg \pi_1$

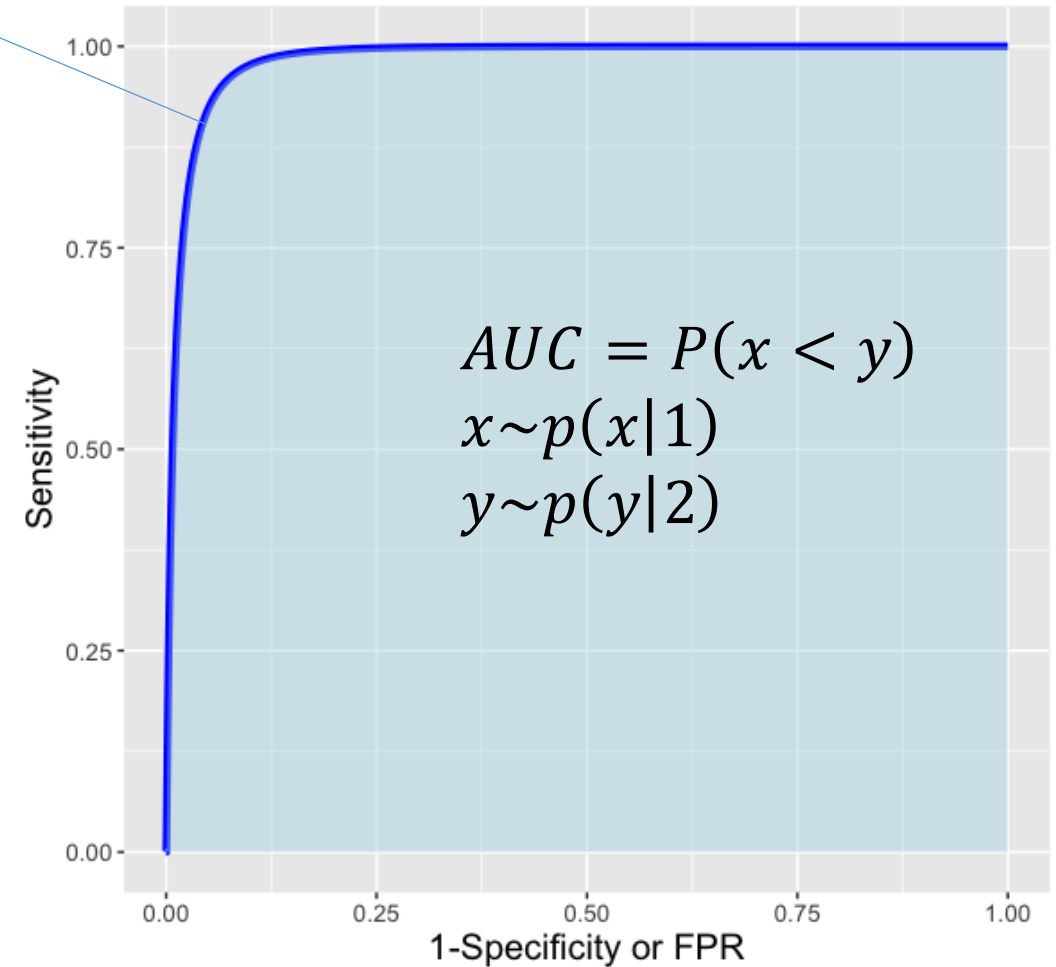
Specificity, Sensitivity are independent of π_i

TPR, FPR, Acc, F depend on π_i

Assessing performance: AUC - ROC curve

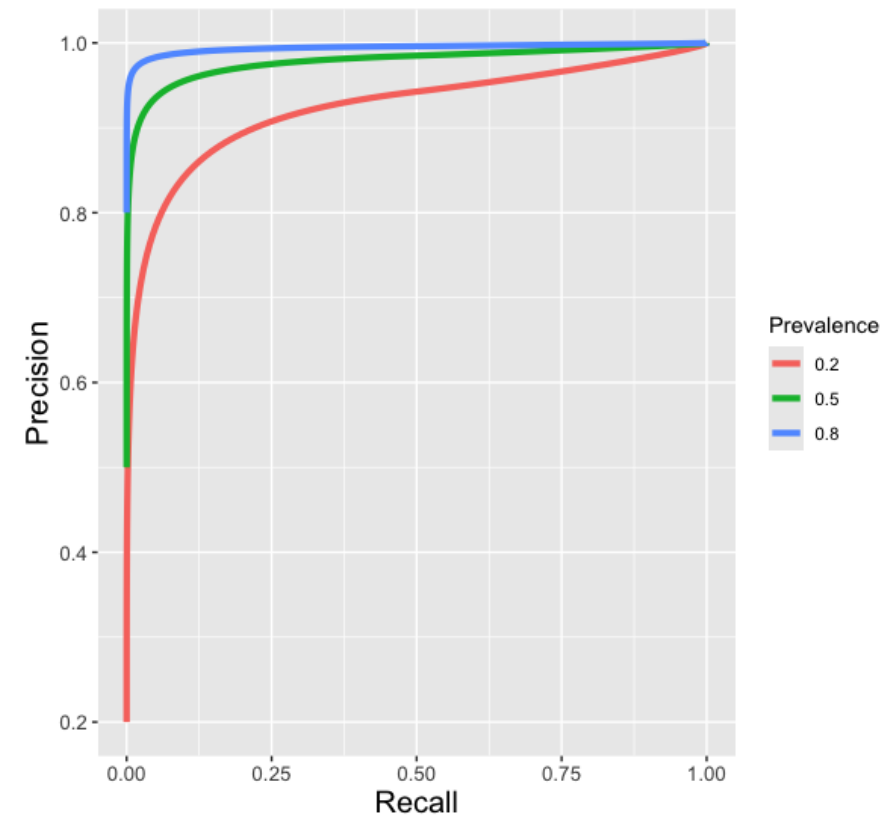
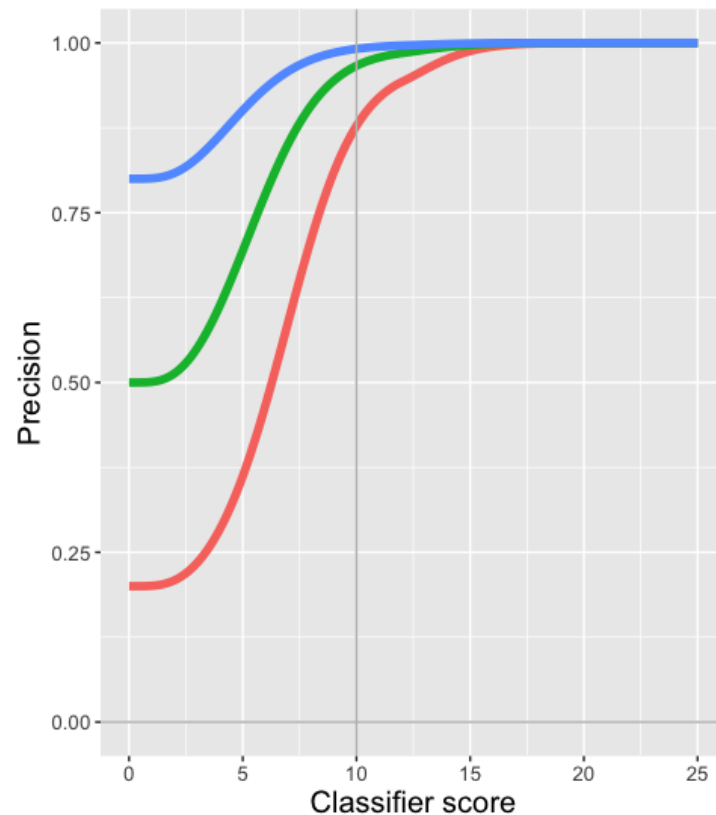


ROC



Assessing performance: Precision-Recall curve

$$p(\text{score}) = \pi_1 p(\text{score}|1) + \pi_2 p(\text{score}|2)$$



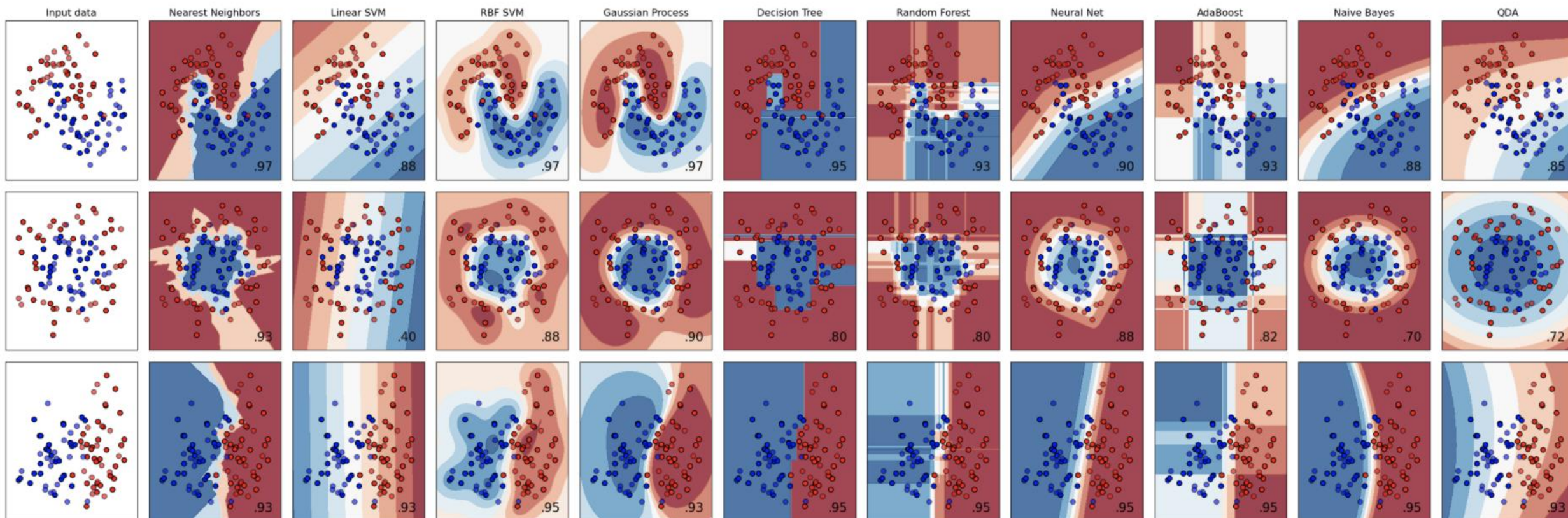
Assessing classifier performance

- You measure single cell gene expression profiles of tumor and healthy cells and you train a classifier to categorize unknown cells as tumoral or healthy. In training and test sets you have about equal amounts of tumoral and healthy cells. Which performance measure could you use?
- You have a blood plasma biomarker pannel for screening a rare cancer. Both training and test sets a highly unbalanced (prevalence of cancer patients 0.1%). Which performance measure could you use?

Which Classifier Should We Choose?

- There is a mathematical theorem that says that there is no universally best classifier. It really depends on the data!
- One needs to test several classifiers for each dataset and compare the results.
- Best is to have a few favorite classifiers (e.g. kNN, LR, SVM), which you know well, and which are somehow complementary (local density based, linear, or inbetween)

Which Classifier Should We Choose?



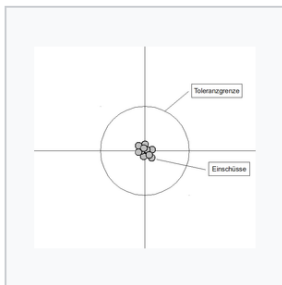
[sklearn classifier comparison](#)

Bias versus Variance Tradeoff

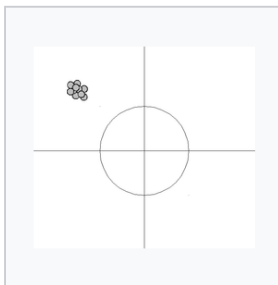
- All classifiers make errors and wrongly predict class labels for certain instances. The error can be split into two part: *bias* and *variance*.
- Variance: the classifier will depend on the subset of the data we choose for training. The more a classifier depends on the training data selection, the higher its variance.
- Bias: the variance usually decreases with the size of the training set. However, even for very large training sets, there will be an error left due to the bias of the classifier. For example, a linear classifier will never be perfectly able to separate classes with non-linear boundaries even if large datasets are available.
- Classifiers with low bias fit the training data well, but may have a large variance and low performance on the test set. Others have a larger bias, but are more stable and predictive for the test set.

Bias versus Variance Tradeoff

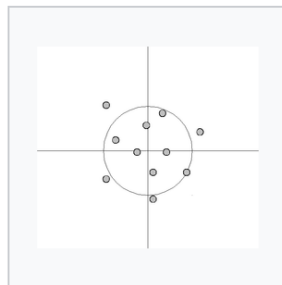
Deep learning Linear SVM
Regularized
Logistic Regr. 1-NN
RBF SVM Linear
Perceptron



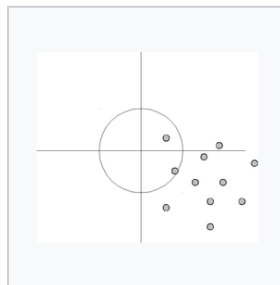
bias low,
variance low:
precision high



bias high,
variance low:
precision high

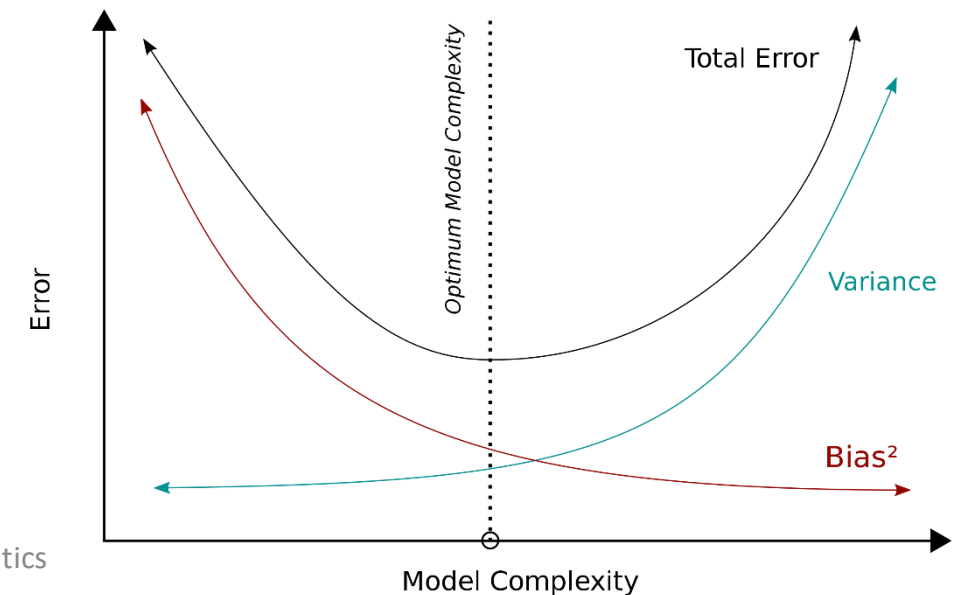
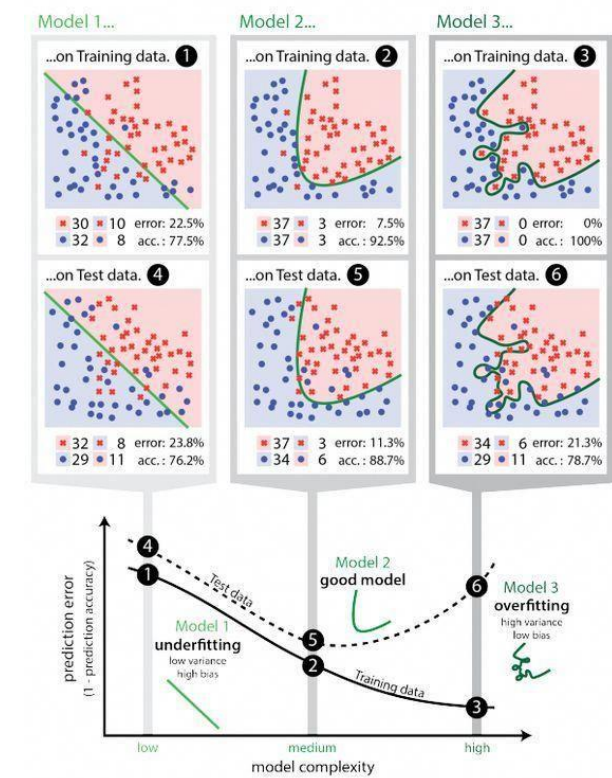


bias low,
variance high:
precision low



bias high,
variance high:
precision low

https://en.wikipedia.org/wiki/Bias-variance_tradeoff



Some Notations:

$$\text{Data matrix: } \mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_N)^T = \begin{pmatrix} x_{11} & \cdots & x_{1D} \\ \vdots & \ddots & \vdots \\ x_{N1} & \cdots & x_{ND} \end{pmatrix}$$

$\longleftrightarrow D \text{ features}$

$\updownarrow N \text{ samples}$

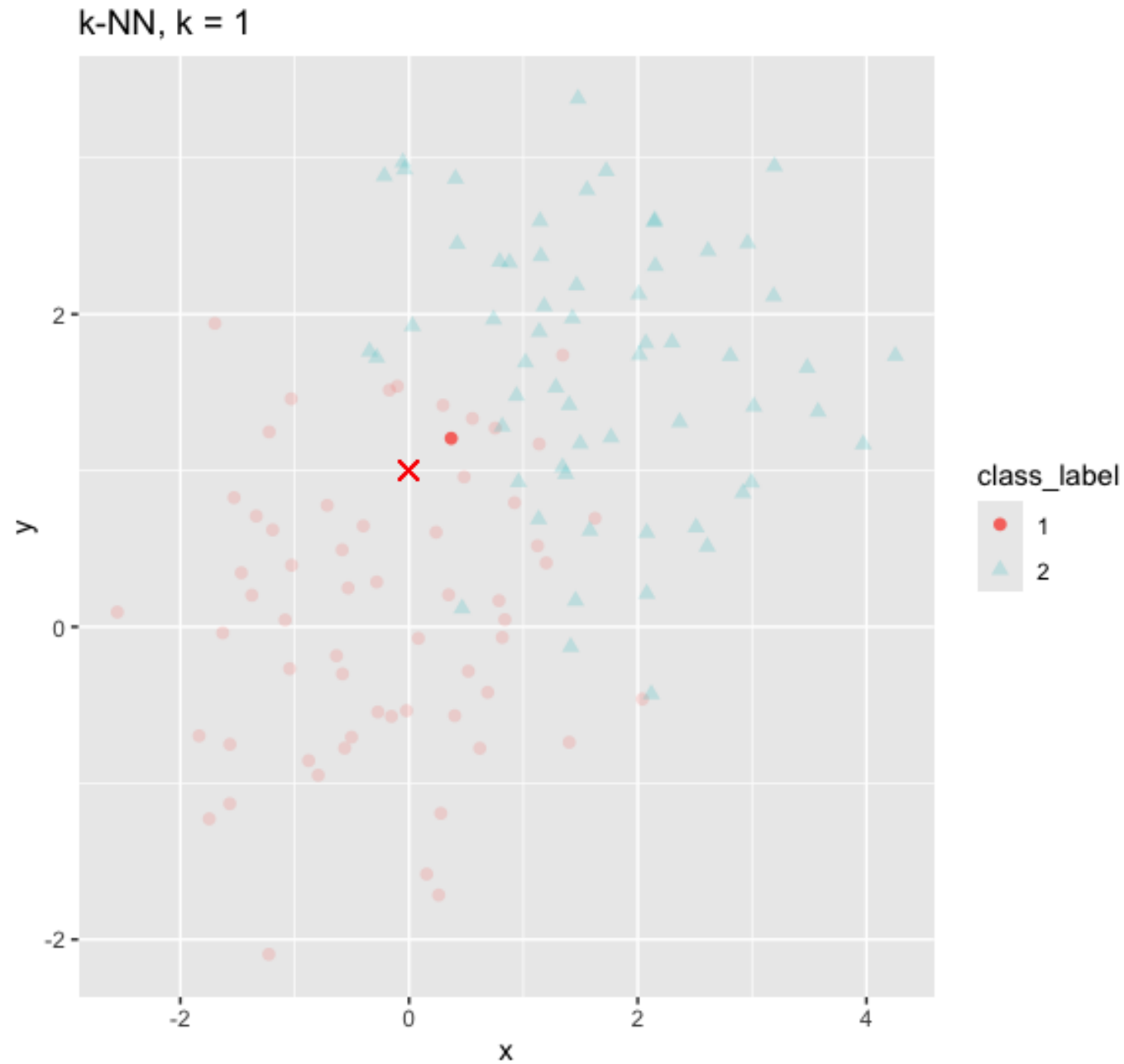
$$\text{Outcome: } \mathbf{y} = (y_1, \dots, y_N)^T$$

Outcome is a known attribute of the items (rows). It can be a binary factor (e.g. healthy/diseased) or a class label (book, glasses, bike, ...).

K-Nearest Neighbor Classifier (k-NN)

- Let's assume that our data is continuous and smooth, i.e. the class labels do not change abruptly within the class boundaries
- Then it is reasonable to assume that the unknown class label of a new data sample is similar to the class label of its nearest neighbours.
- Given a training dataset $\mathbf{X}^{train}, \mathbf{y}^{train}$ we predict the class of an instance $\mathbf{x}$ by taking the k nearest instances $\mathbf{x}_i^{train}, i \in kNN(\mathbf{x})$ according to some distance metric $d(\mathbf{x}, \mathbf{x}_i^{train})$ and predict the majority class in $\mathbf{y}_i^{train}, i \in kNN(\mathbf{x})$
- Instead of simply predicting the majority class we can also weight the classes by $1/d(\mathbf{x}, \mathbf{x}_i^{train})$.
- The number of neighbors k we will determine with cross validation.

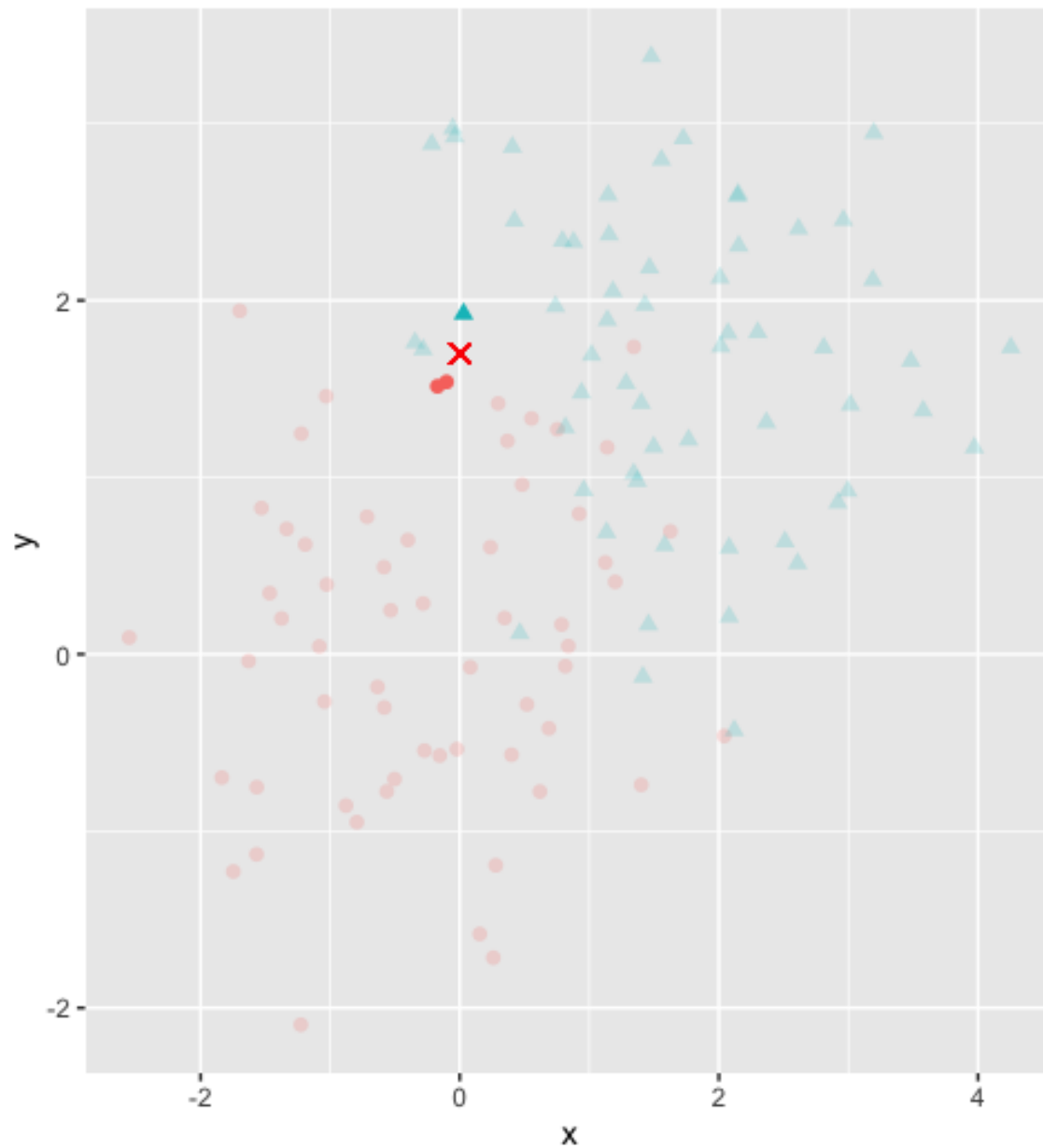
k-NN



✗ class label = ●

k-NN

k-NN, k = 3



class_label



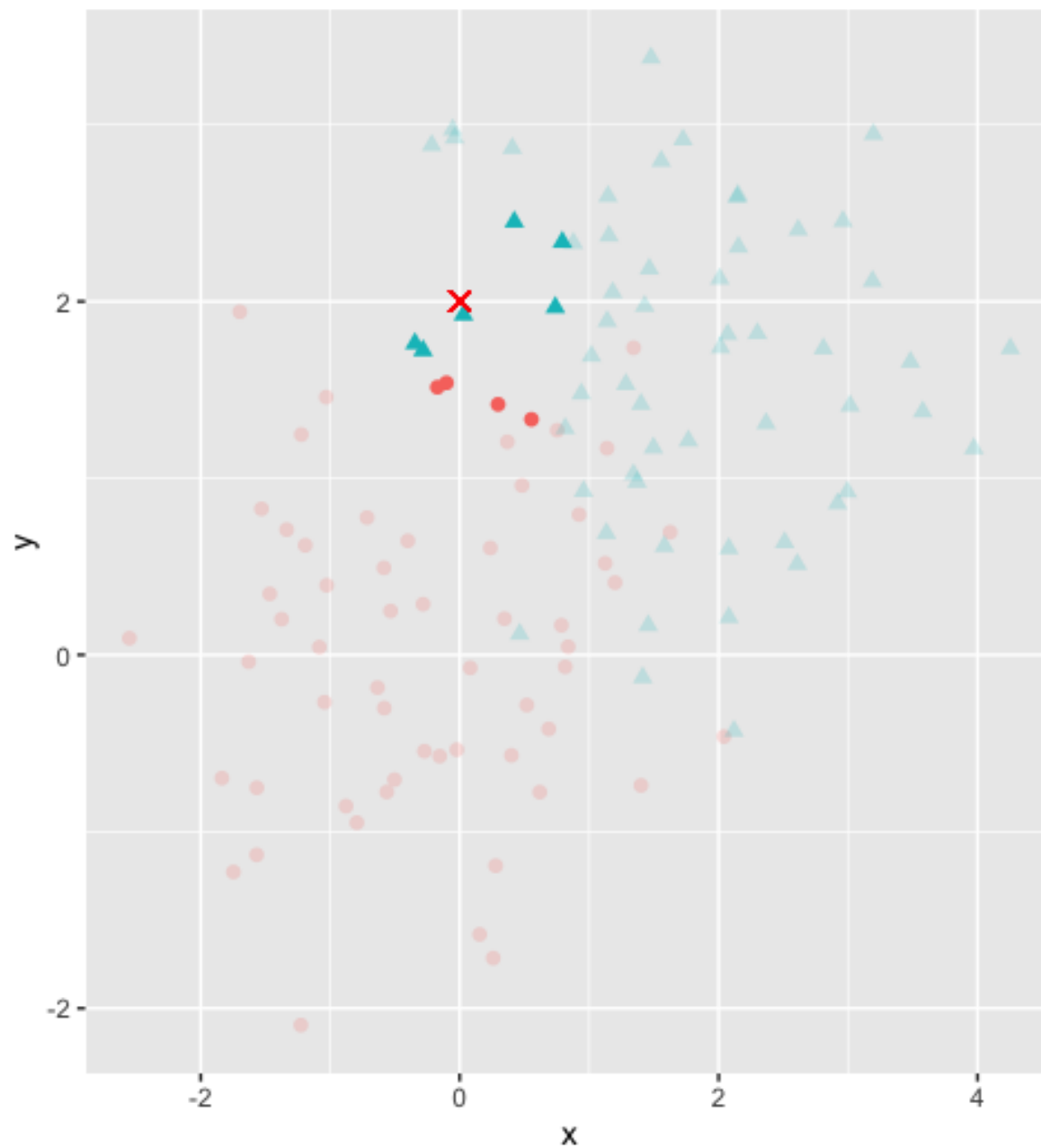
✗ class label = ●



Swiss Institute of
Bioinformatics

k-NN

k-NN, k = 10



class_label

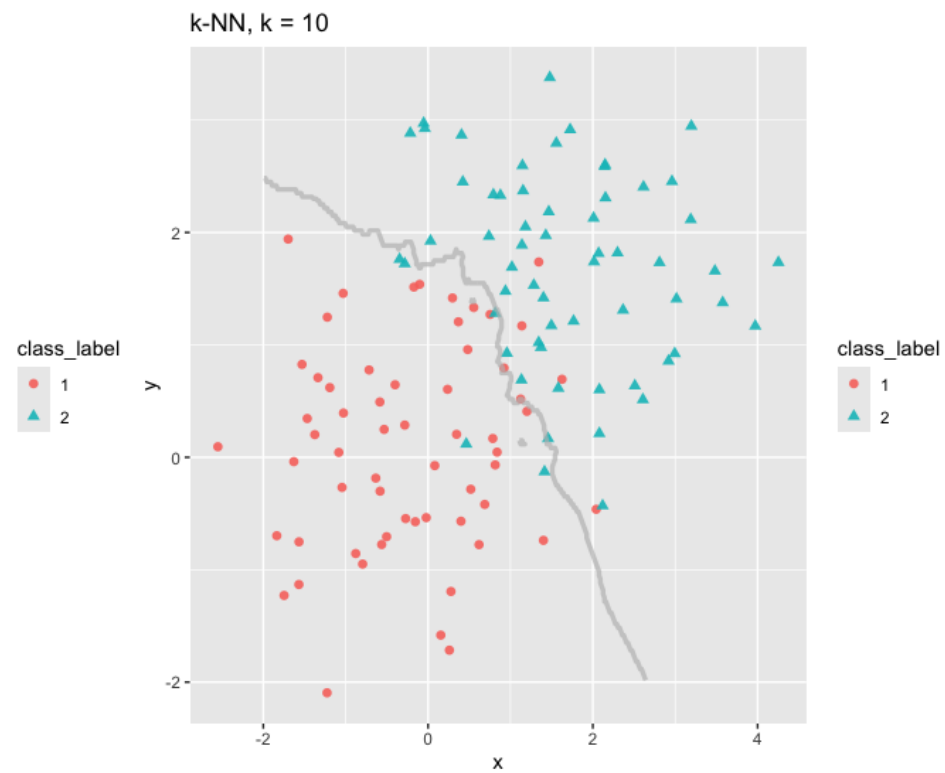
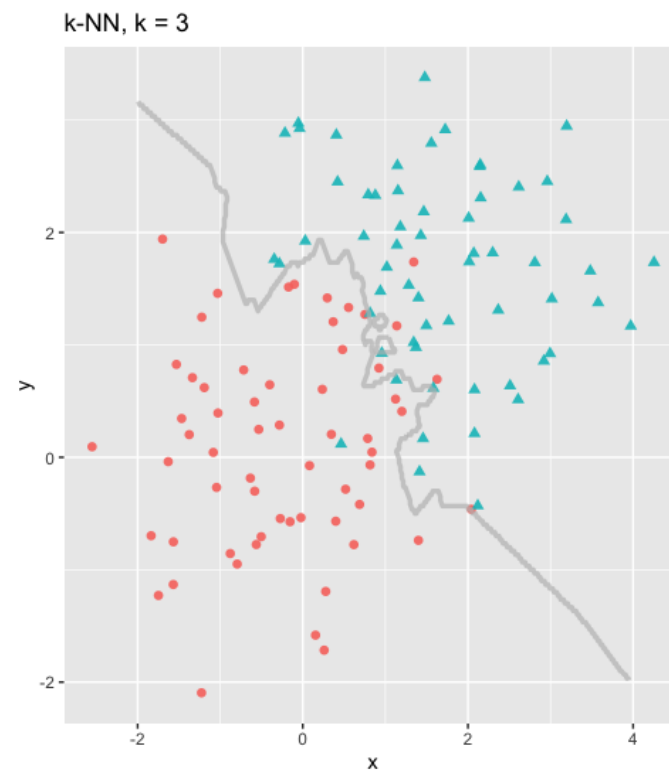
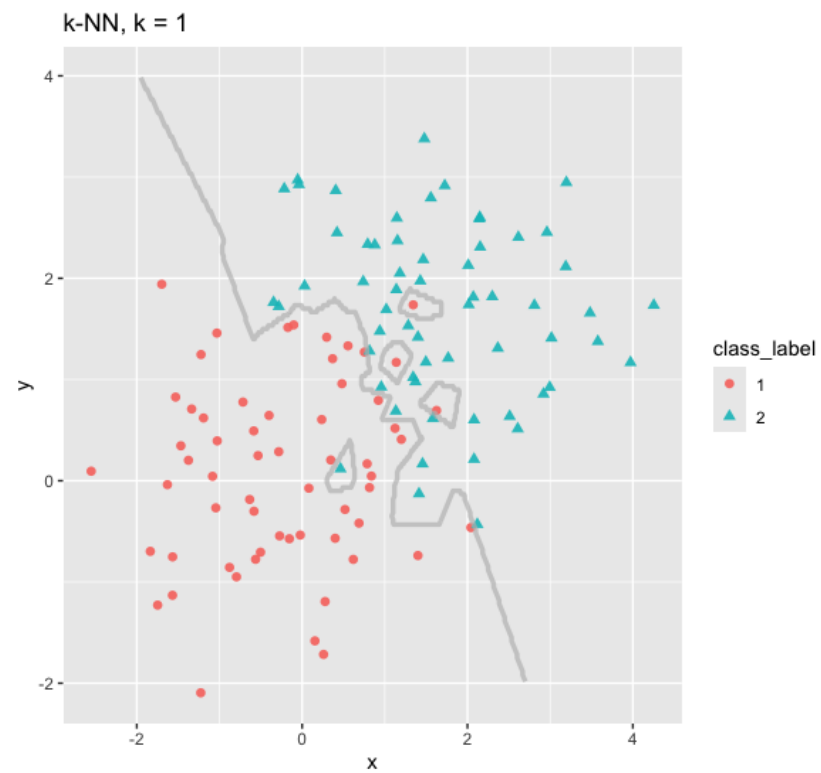


✗ class label = ▲



Swiss Institute of
Bioinformatics

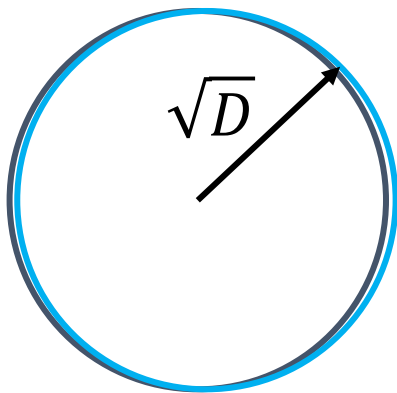
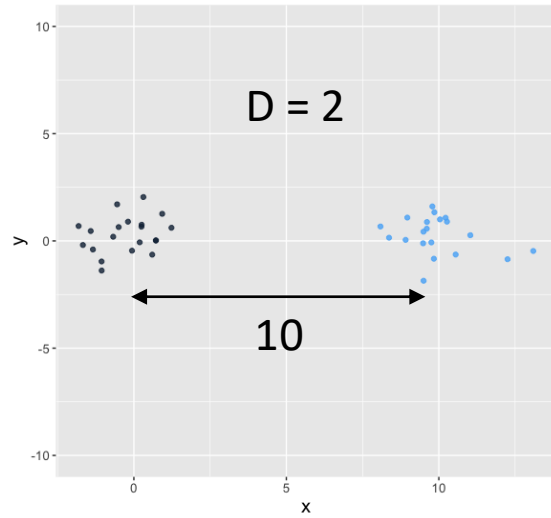
K-NN



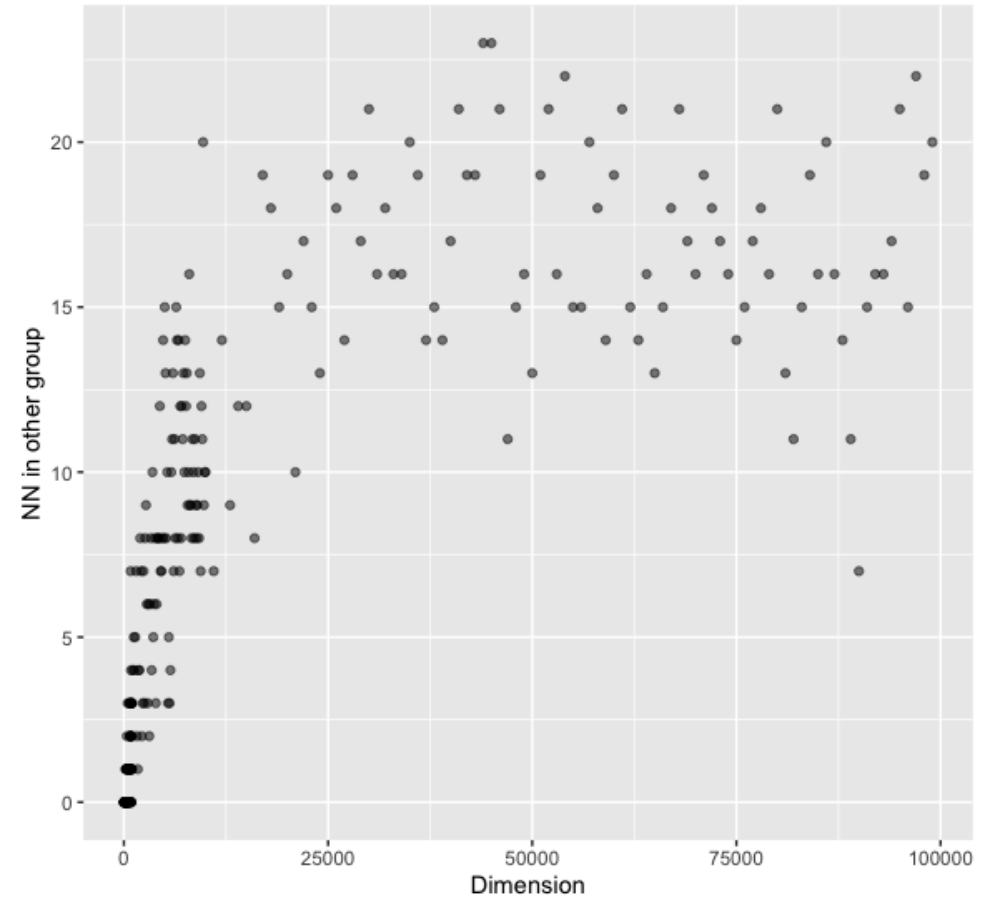
K-NN

- Pros
 - No need to train the model (model = data)
 - Works for any data for which distance can be defined
 - Works for multiply class problems
 - Can be used for regression: the output value for the object is computed by the average of k closest neighbors value.
 - Calculates probability
- Cons
 - Class boundaries not smooth
 - For large data, all data needs to be kept in memory
 - Search NN can be slow for large data and dimensions
 - Features should be scaled, otherwise distance is biased by large feature values
 - It is not suitable for very large dimensional data (curse of dimensionality)

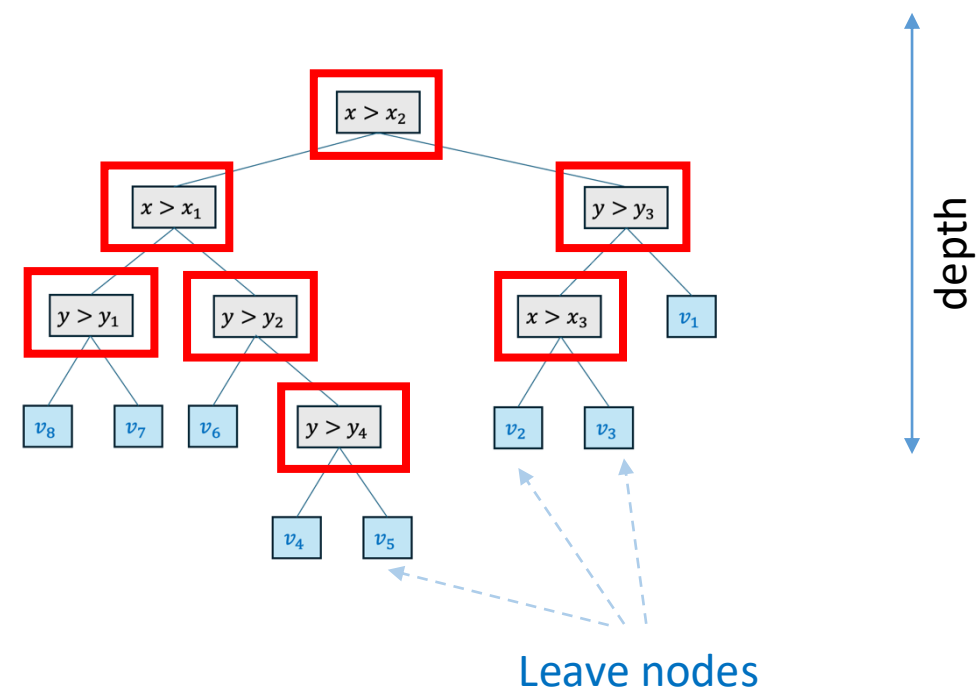
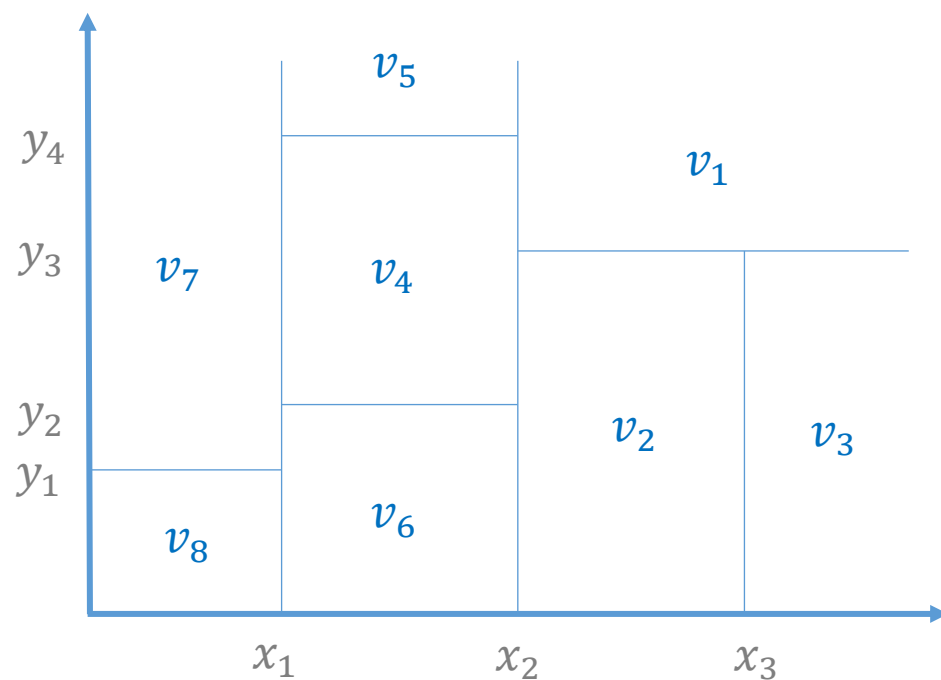
K-NN in high dimensions



$D \gg 10$



Trees



Decision trees

- Algorithm:
 - hierarchical sequence of questions on your features that can be answered by yes or no and that subdivides the data into subgroups on which a new question is asked, and so on and so on
 - tests all the features and chooses the most discriminative feature (in terms of the labels) to divide the data into subsets (impurity: Shannon entropy, Gini index)

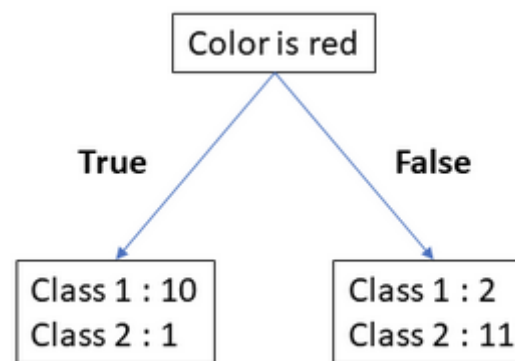
Class 1 

Class 2 

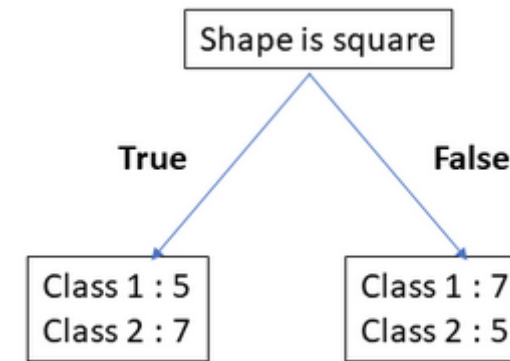
Features:

Color: blue or red

Shape: square or circle



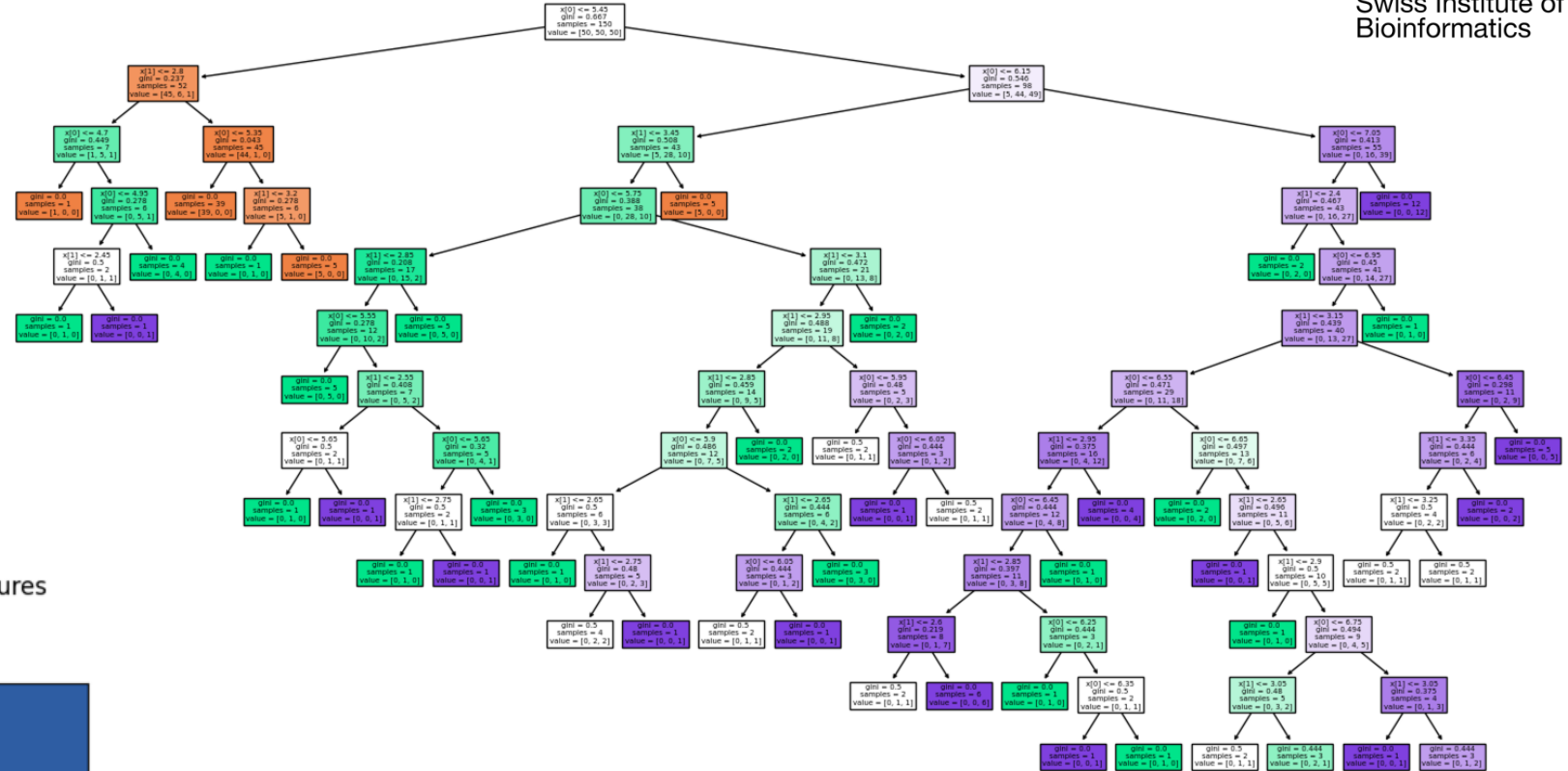
Those groups are
highly skewed toward
certain class



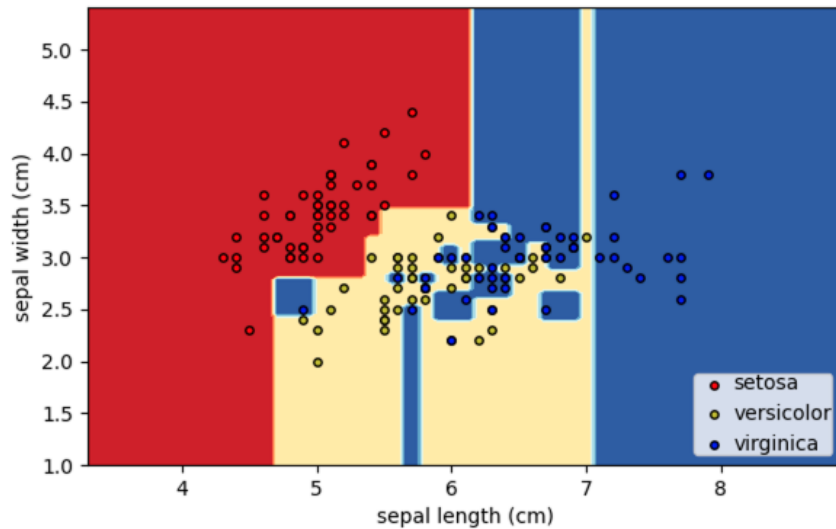
Those groups are well
mixed

Trees

Decision tree trained on all the iris sepal length and width



Decision surface of decision trees trained on pairs of features



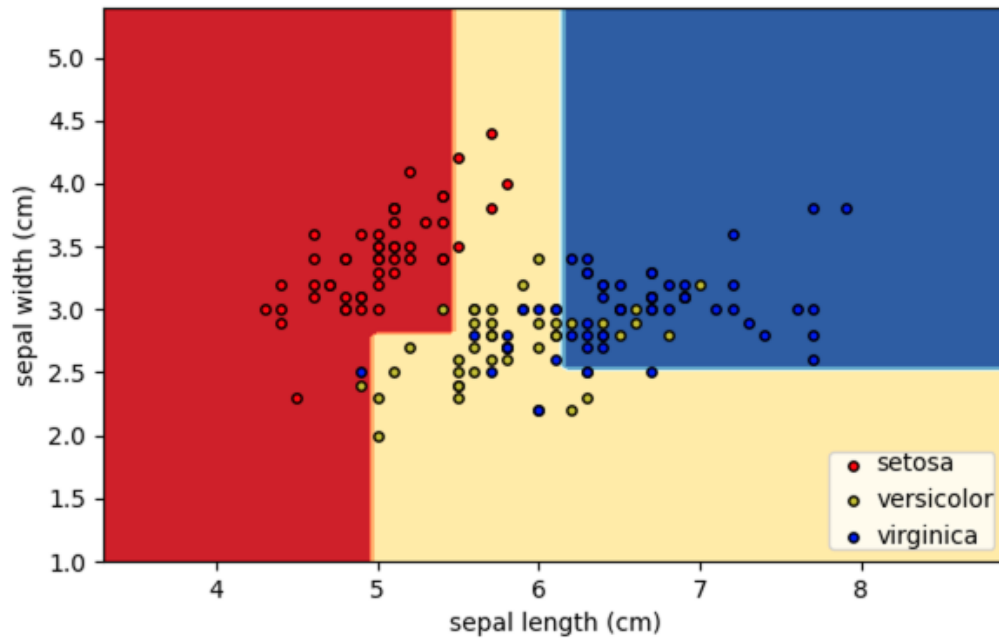
[Code on sklearn page !](#)

Trees

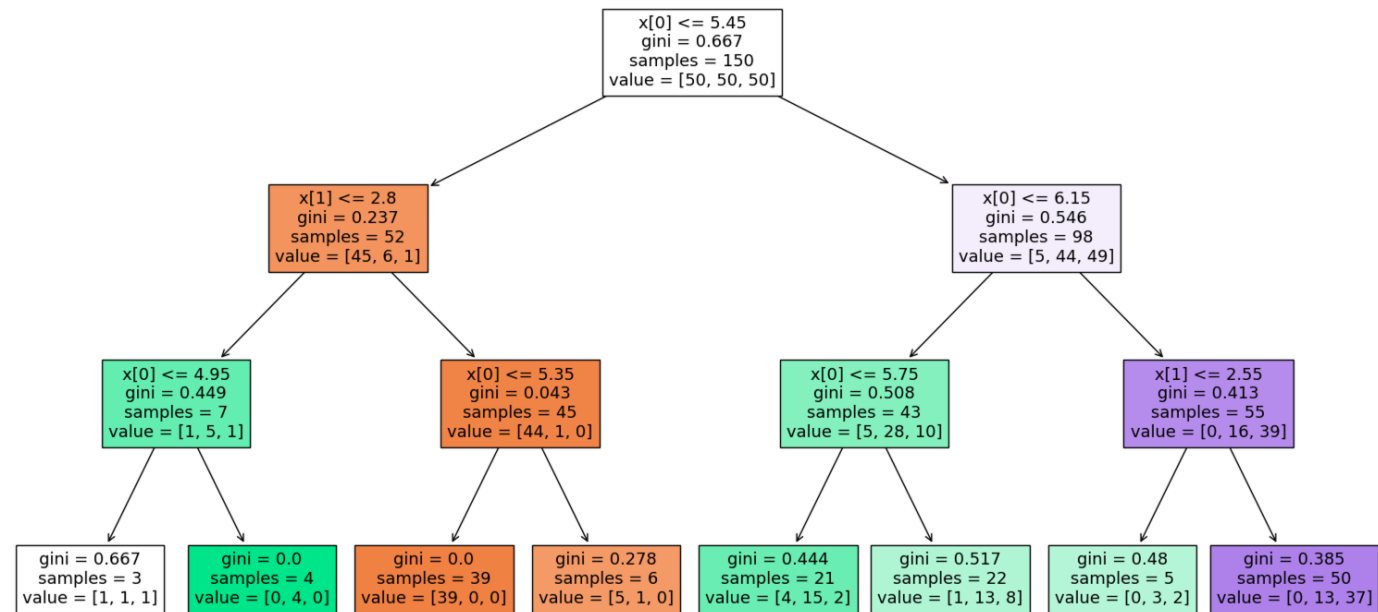
- Regression and classification trees
- Trees are universal approximators (no bias) if the size is not restricted
- Tree's are naturally able to deal with categorical features, and are not sensitive to the distribution of features values
- Parameters:
 - For each node k we have a feature F_k and threshold θ_k
 - For each leaf node l we have a weight v_l for regression, and a class label c_l for classification
- Tree hyperparameters add constraints to its growth and weights:
 - max_depth: maximal number of internal nodes before leaf
 - max_leaf_nodes: maximal number of leaf nodes in the tree
 - min_samples_leaf: minimal number of data points in the training set within leaf
 - L1 or L2 regularization of weights

Tree hyperparameters

DecisionTreeClassifier(min_samples_leaf=3, max_depth=3, max_features=1)



DecisionTreeClassifier(min_samples_leaf=3, max_depth=3, max_features=1)



Tree learning

Algorithm 1. Decision tree induction.

LearnDT (*TrainSet*)

if all examples in *TrainSet* have the same class y_* **then**

return MakeLeaf(y_*)

if no feature x_j has $\text{InfoGain}(x_j, y) > 0$ **then**

$y_* \leftarrow$ Most frequent class in *TrainSet*

return MakeLeaf(y_*)

$x_* \leftarrow \text{argmax}_{x_j} \text{InfoGain}(x_j, y)$

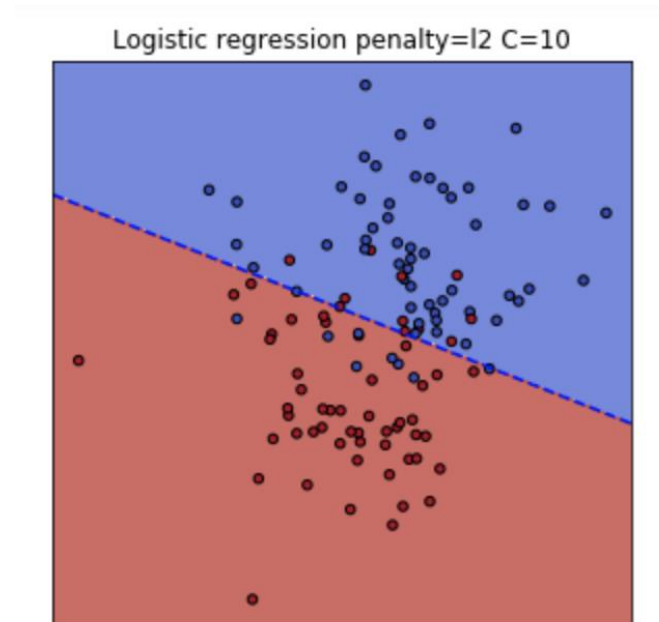
$TS_0 \leftarrow$ Examples in *TrainSet* with $x_* = 0$

$TS_1 \leftarrow$ Examples in *TrainSet* with $x_* = 1$

return MakeNode(x_* , LearnDT(TS_0), LearnDT(TS_1))

Logistic Regression

- In logistic regression we assume that our data is “linearly separable” by a hyperplane $h : w_0 + \sum_{i=1}^D w_i x_i = w_0 + \mathbf{w} \cdot \mathbf{x} = 0$.

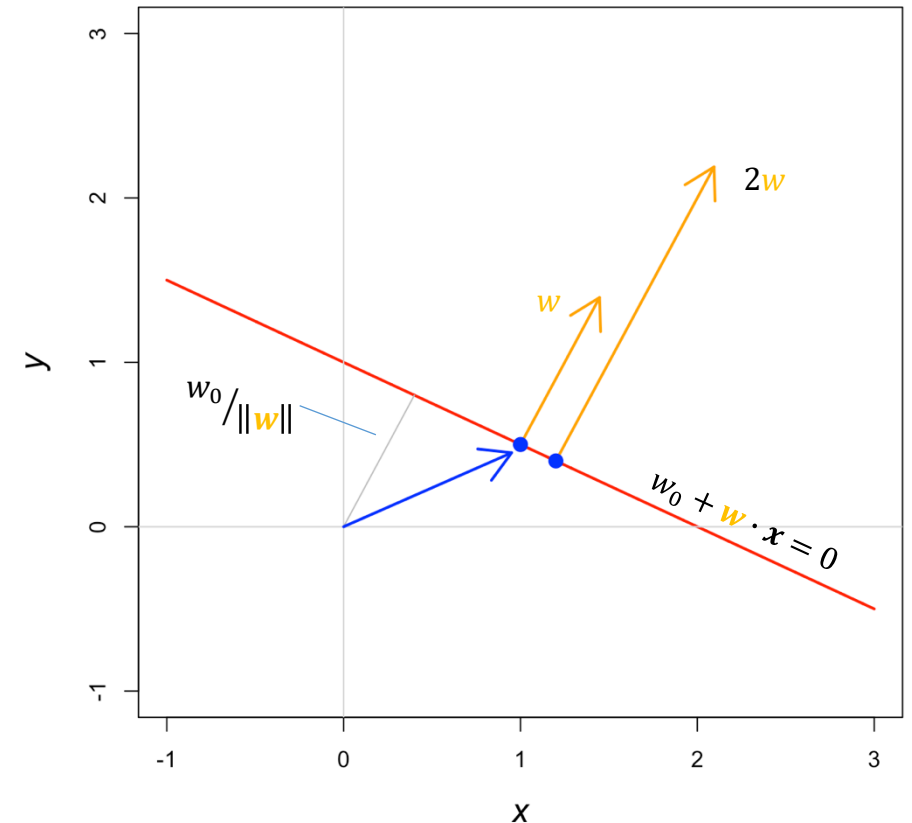


Logistic Regression

- Hyperplane $h : w_0 + \sum_{i=1}^D w_i x_i = w_0 + \mathbf{w} \cdot \mathbf{x} = 0$
- We can choose any $\lambda \mathbf{w}$ and λw_0 without changing h
- Distance d of a point $\mathbf{x}$ to h :

$$d = \frac{w_0 + \mathbf{w} \cdot \mathbf{x}}{\|\mathbf{w}\|}$$

- If $\|\mathbf{w}\|$ is large then $h(\mathbf{x}) = w_0 + \mathbf{w} \cdot \mathbf{x}$ changes rapidly around h .

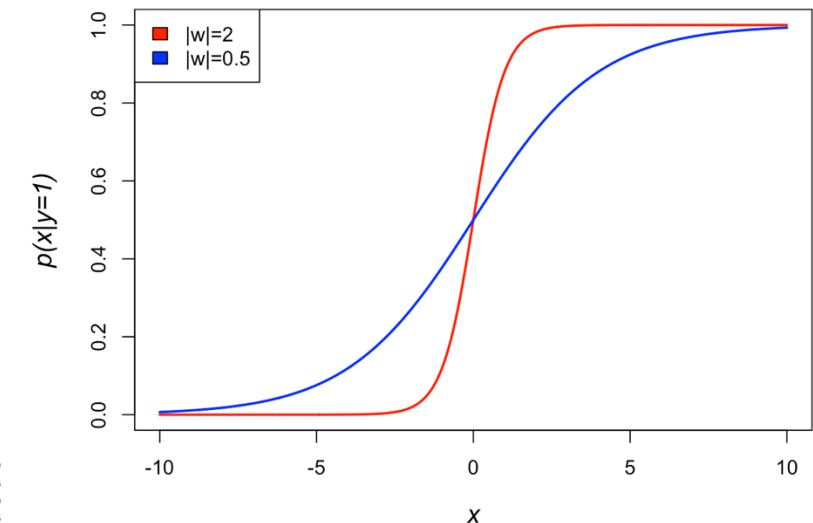
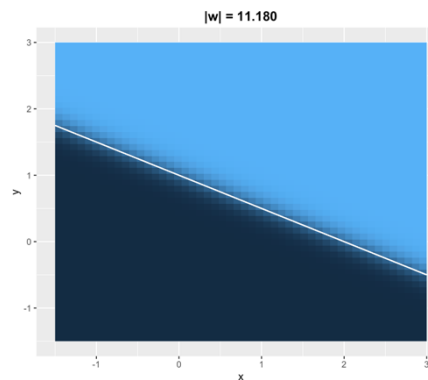
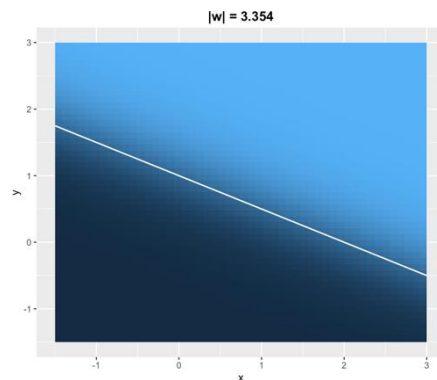
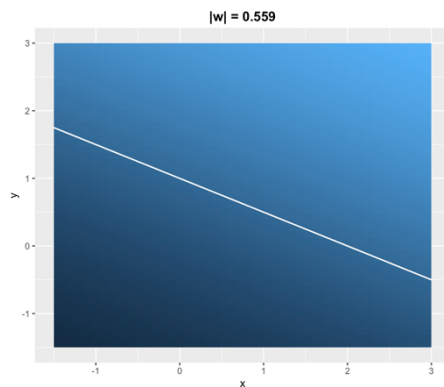


Logistic Regression

- We assume we have two classes labelled 0 and 1.
- We assume a sigmoid shaped class probability distribution p , which only depends on the distance d to h :

$$p(\mathbf{x} | y = 1) = \frac{1}{1 + e^{-(w_0 + \mathbf{w} \cdot \mathbf{x})}} = \frac{1}{1 + e^{-d \|\mathbf{w}\|}}$$

$$p(\mathbf{x} | y = 0) = 1 - p(\mathbf{x} | y = 1)$$



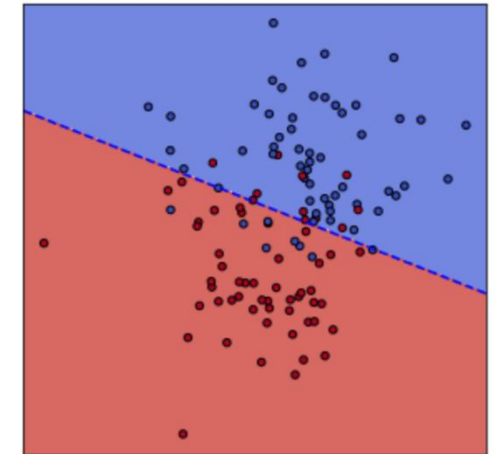
Logistic Regression

- To train the LR model, we have to find $w_0, \mathbf{w}$ that fit best to the training data. To evaluate the quality of a fit, we need a loss function $\mathcal{L}$.
- Logistic loss function:

$$\mathcal{L}(w_0, \mathbf{w}, \mathbf{X}, \mathbf{y}) = -\frac{1}{N} \left\{ \sum_{i, y_i=1} \log p(\mathbf{x}_i | y = 1) + \sum_{i, y_i=0} \log p(\mathbf{x}_i | y = 0) \right\} \geq 0$$

Class 1
Class 0

$p(\mathbf{x}_i | y = 1) = 1: \log\{p(\mathbf{x}_i | y = 1)\} = 0$
 $p(\mathbf{x}_i | y = 1) < 1: \log\{p(\mathbf{x}_i | y = 1)\} < 0$



Logistic Regression

- Logistic loss function:

$$\mathcal{L}(w_0, \mathbf{w}, \mathbf{X}, \mathbf{y}) = -\frac{1}{\sum_{i=1}^N s_i} \left\{ c_1 \sum_{i, y_i=1} s_i \cdot \log p(\mathbf{x}_i | y = 1) + c_0 \sum_{i, y_i=0} s_i \cdot \log p(\mathbf{x}_i | y = 0) \right\} \geq 0$$

Diagram annotations:

- sample_weight* (orange text) with arrows pointing to s_i in both summations.
- class_weight* (blue text) with arrows pointing to c_1 and c_0 .

- LR finds the optimal $w_0, \mathbf{w}$ by minimizing $\mathcal{L}$. LR in sklearn offers several minimization algorithms.

Logistic Regression

- Regularization: keep $\mathbf{w}$ small and boundary wide and smooth:

- $\mathcal{L}(w_0, \mathbf{w}, \mathbf{X}, \mathbf{y}) + \frac{1}{c} \sum_{i=1}^D |w_i|$: lasso or 'l1'

- $\mathcal{L}(w_0, \mathbf{w}, \mathbf{X}, \mathbf{y}) + \frac{1}{2c} \sum_{i=1}^D w_i^2$: ridge or 'l2'

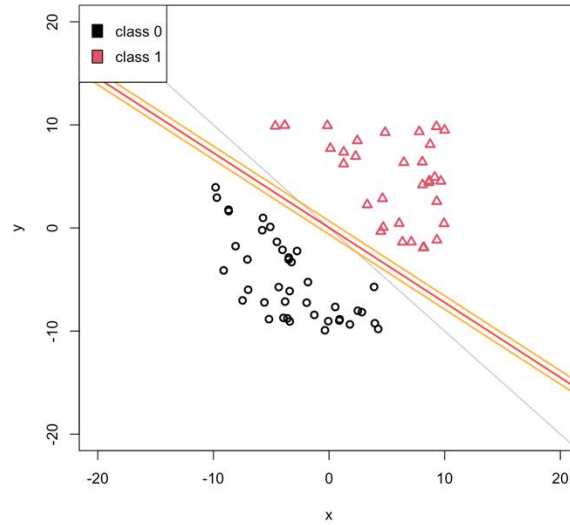
- $\mathcal{L}(w_0, \mathbf{w}, \mathbf{X}, \mathbf{y}) + \frac{1}{c} \left\{ \sum_{i=1}^D \alpha |w_i| + \frac{(1-\alpha)}{2} w_i^2 \right\}$
 - α : l1_ratio : elastic net

Logistic Regression

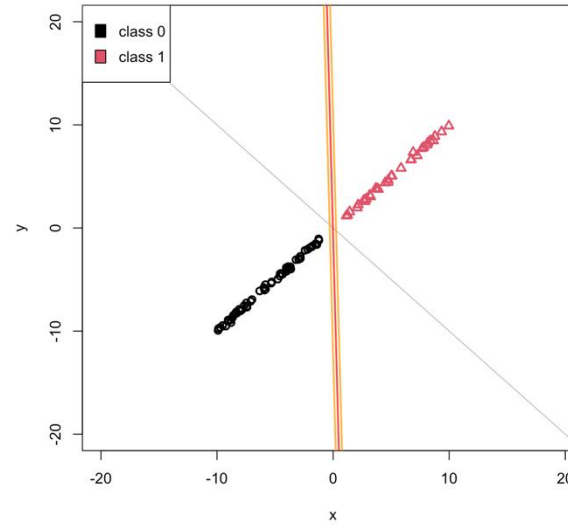


Swiss Institute of
Bioinformatics

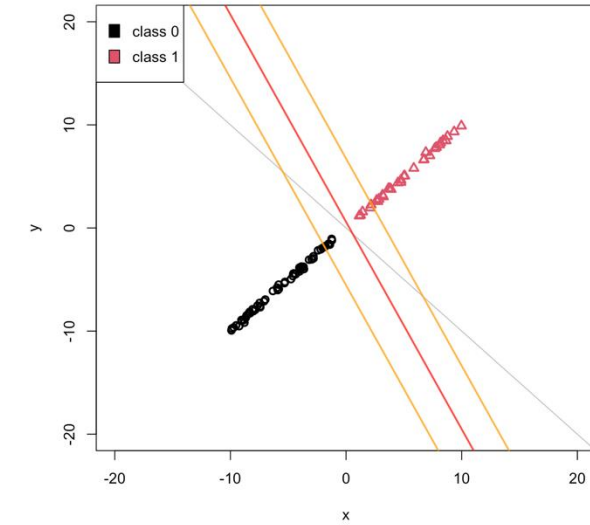
Ridge, $C = \text{Inf}$, $|w| = 5.600$



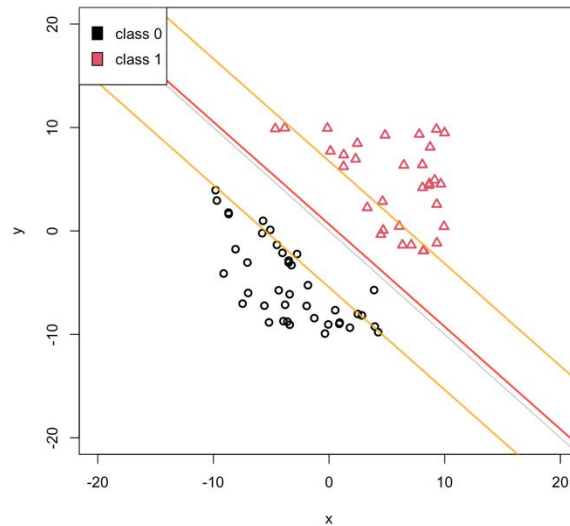
Ridge, $C = \text{Inf}$, $|w| = 10.826$



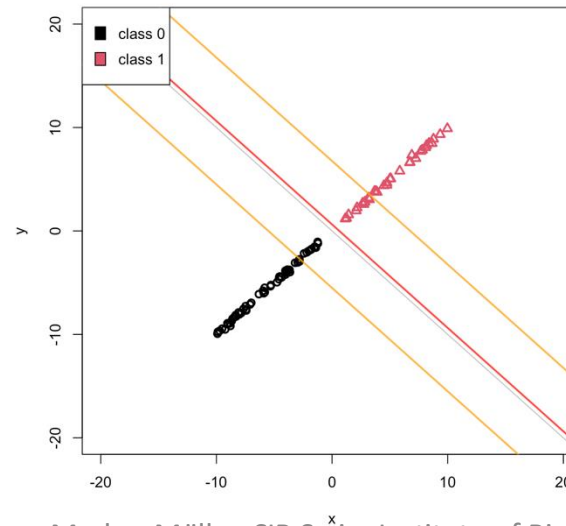
Lasso, $C = 2.000\text{e}+02$, $|w| = 1.076$



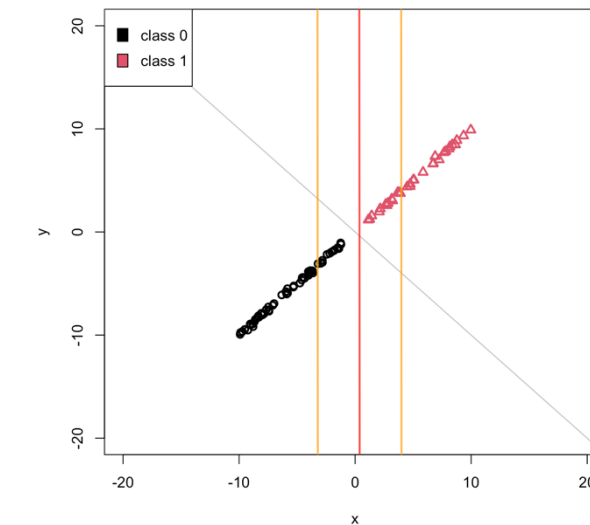
Ridge, $C = 2.000\text{e}+02$, $|w| = 0.679$



Ridge, $C = 2.000\text{e}+02$, $|w| = 0.679$



Lasso, $C = 5.000\text{e}+01$, $|w| = 0.817$

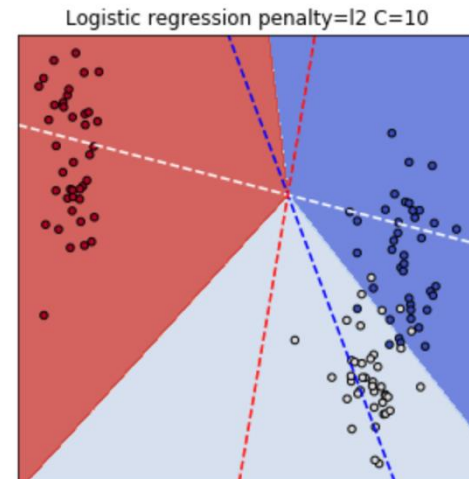


Multinomial Logistic Regression (K classes)

- $p(\mathbf{x}|y = k) = \frac{e^{w_{0k} + \mathbf{w}_k \cdot \mathbf{x}}}{\sum_{i=1}^K e^{w_{0i} + \mathbf{w}_i \cdot \mathbf{x}}}$
- **Class** $y(\mathbf{x}) = \underset{k}{\operatorname{argmax}} p(\mathbf{x}|y = k) = \underset{k}{\operatorname{argmax}} (w_{0k} + \mathbf{w}_k \cdot \mathbf{x})$
- **Loss function:**

$$\mathcal{L}(w_{0k}, \mathbf{w}_k, \mathbf{X}, \mathbf{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_i^k \log p(\mathbf{x}_i|y = k);$$

$$y_i^k = \begin{cases} 1: y(\mathbf{x}_i) = k \\ 0: y(\mathbf{x}_i) \neq k \end{cases}$$



Logistic Regression

- Can be implemented very efficiently
- Scales to large data
- Good generalization thanks to regularization
- Few hyperparameters
- Multiclass
- Interpretable

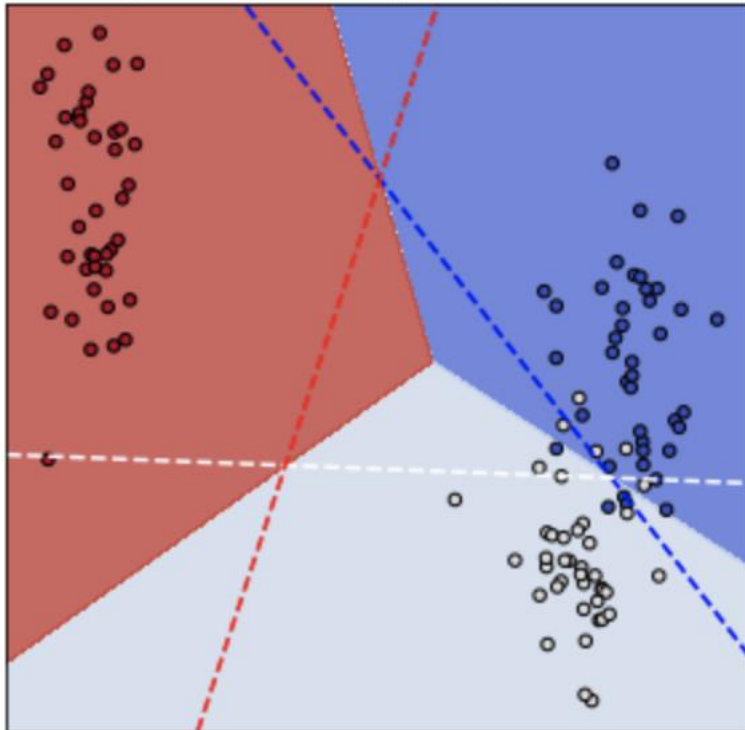
Multiclass Classification

- If more than two classes there are several options to deal with them:
 - **Ovo** (one-versus-one): split the problem into $K(K - 1)/2$ classification tasks. At prediction time we choose the class with the most votes. Implemented in SVC. See also `sklearn OneVsOneClassifier`.
 - **Ovr** (one-versus-rest): split the problem into K classification tasks. At prediction time we choose the class with the highest prediction score. Implemented in LogisticRegression. See also `OneVsRestClassifier`.
 - **Inherently multiclass**: Some classifiers have inherent support for multiclass problems (e.g. `k-NN`, `LogisticRegression`, `DecisionTree`, `RandomForest`)
- Check: <https://scikit-learn.org/stable/modules/multiclass.html>

Multiclass Classification

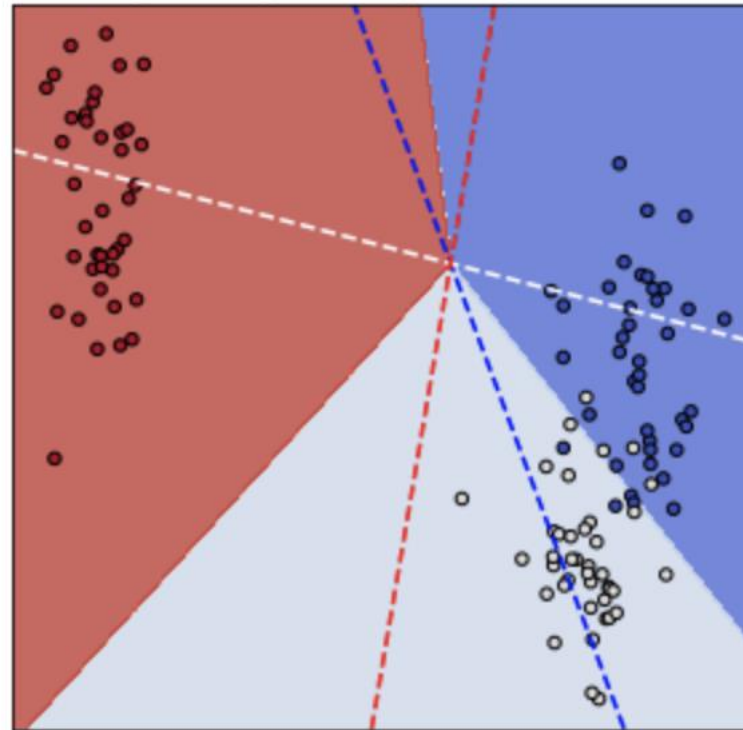
ovr

Logistic regression penalty=l2 C=10

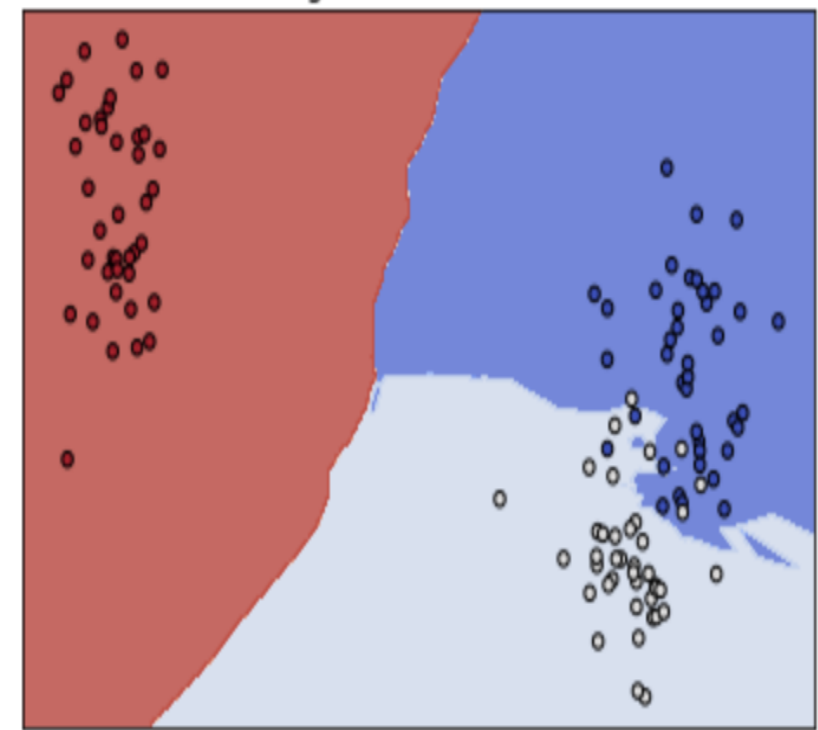


multinomial

Logistic regression penalty=l2 C=10

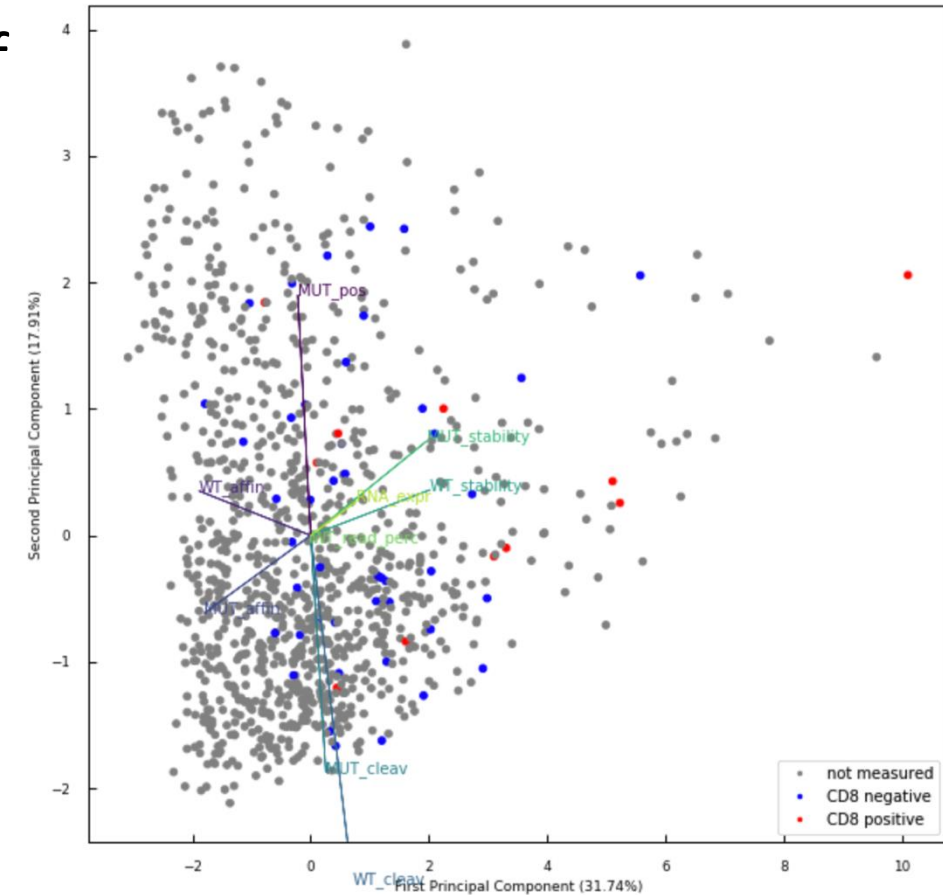


K neighbors k=5, uniform



Unbalanced Classes

- Very often in ML, there are different number of instances in each class
- This can be a problem for a ML classification method if classes are well mixed
- If '*accuracy*' is used in a CV loop a classifier will try to predict the abundant class correctly, and ignore the rare class.
- Solution: Use '*roc_auc*' or work with '*class_weight*' parameter.
 - *class_weight*='balanced' : $cw_k = \frac{N}{K \cdot \sum_{i=1}^N y_i^k}$
 - Set '*class_weight*' or learn in CV loop

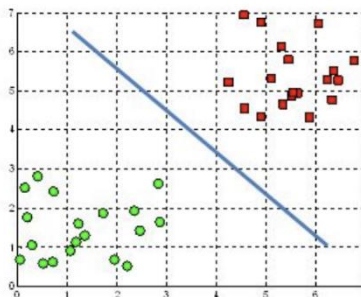


Support Vector Machines (SVM)

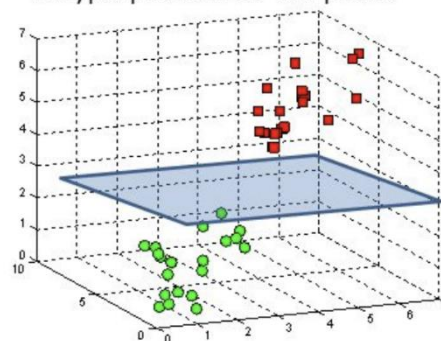
- Separating hyperplanes
- Soft margin classification
- Linear SVM
- Non-linear SVM & Kernels

Largest Margin Separating Hyperplane

A hyperplane in $\mathbb{R}^2$ is a line



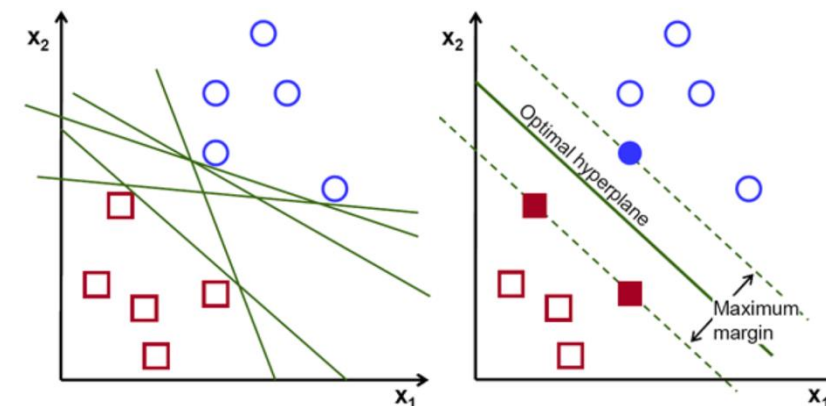
A hyperplane in $\mathbb{R}^3$ is a plane



Hyperplanes are linear decision boundaries that help classify the data points.

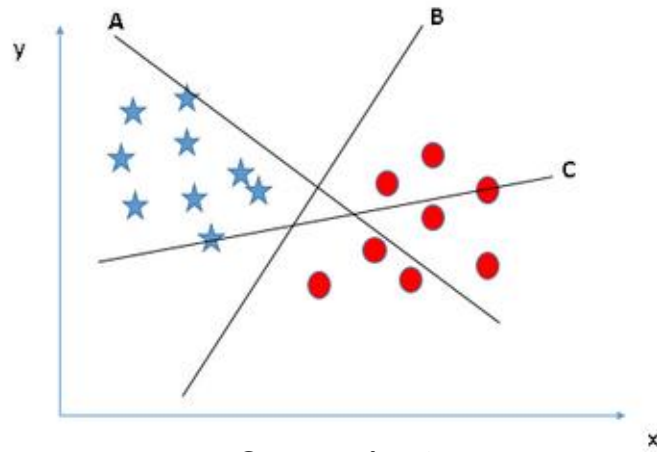
- Objective:

- find a hyperplane in an M-dimensional space (M = number of features) that distinctly classifies the data points
- find a plane that has the maximum margin, i.e the maximum distance between data points of both classes

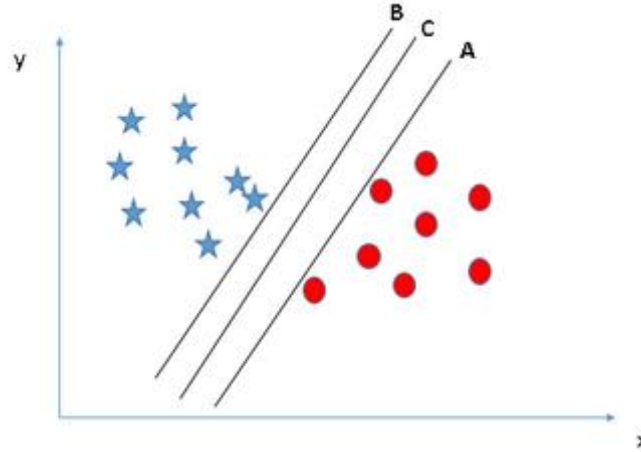


- Maximizing the margin distance provides some reinforcement so that future data points can be classified with more confidence

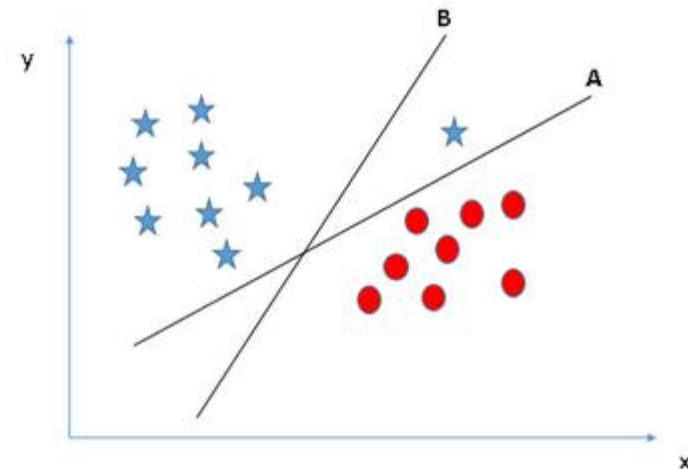
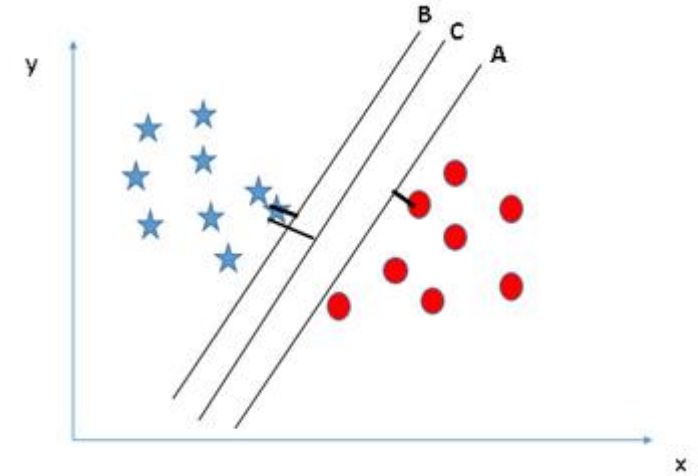
Support Vector Machines (SVM)



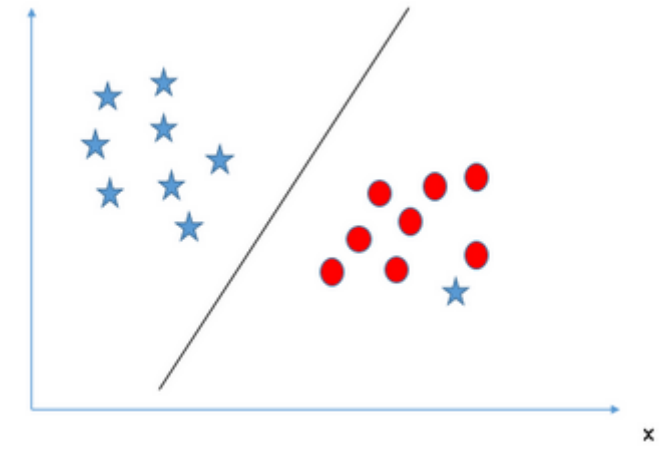
Scenario 1



Scenario 2



Scenario 3

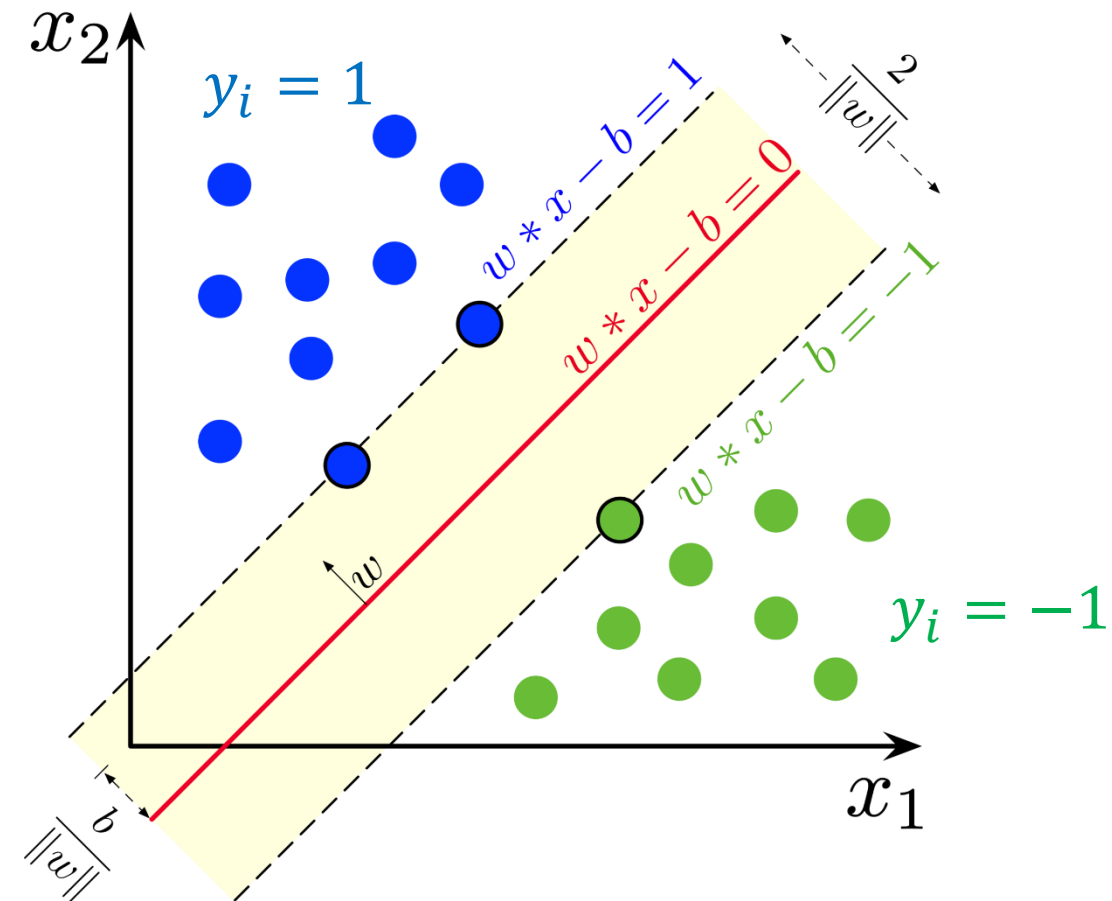


Scenario 4

Largest Margin Separating Hyperplane

https://en.wikipedia.org/wiki/Support-vector_machine

- Margin $m = \frac{2}{\|\mathbf{w}\|}$
- Maximal margin $m \Rightarrow \text{minimal } \|\mathbf{w}\|$
- $\min_{\mathbf{w}, b} \|\mathbf{w}\|^2$
subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1, i = 1..N$

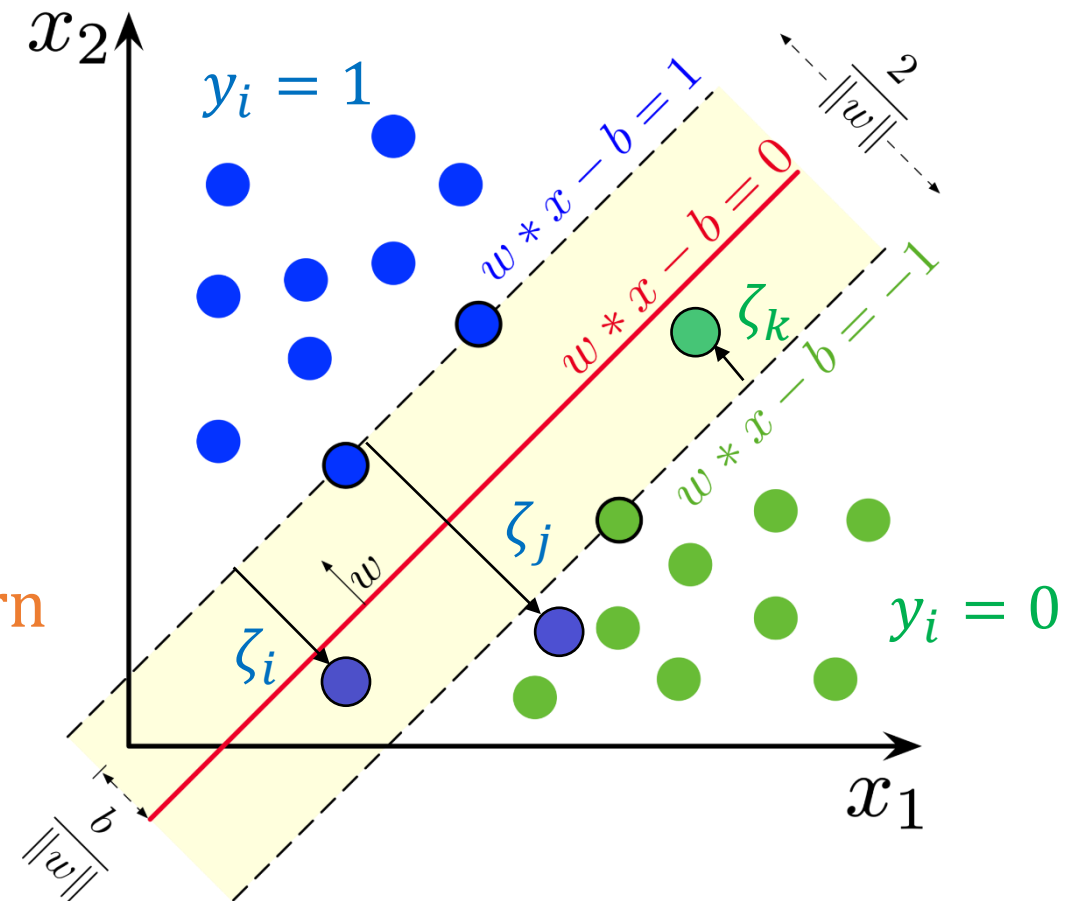


Soft Margin Separating Hyperplane

- $\zeta_i = \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b))$
- $\min_{\mathbf{w}} \{ \|\mathbf{w}\|^2 + C \sum_{i=1}^N \zeta_i \}$

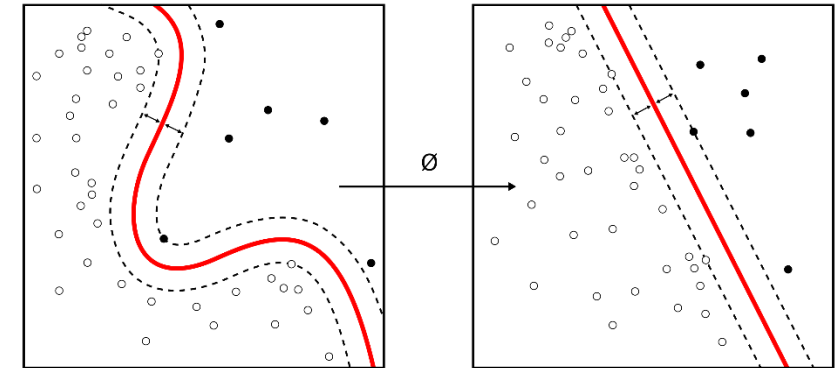
subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1 - \zeta_i, \zeta_i \geq 0$
 $i = 1 \dots N$

- This is a quadratic programming problem. Efficient algorithms implemented in sklearn LinearSVC can solve this in $O(NM)$
- Non-linear SVM (see below) requires $O(N^\alpha M)$, where $2 < \alpha < 3$



Non-Linear Support Vector Machines

- Dual form: $\min_{\alpha} \left\{ -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^N \alpha_i \right\}$
subject to:
 $\sum_{i=1}^N y_i \alpha_i = 0$ and $0 \leq \alpha_i \leq C, i = 1 \dots N.$
- $\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$; $S = \{i | \alpha_i > 0\}$; $b = \frac{1}{|S|} \sum_{i \in S} (y_i - \mathbf{w} \cdot \mathbf{x}_i)$
- Prediction: $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} - b = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} - b$
- Kernel trick: $\mathbf{x} \cdot \mathbf{y} \mapsto K(\mathbf{x}, \mathbf{y})$
- Boser, Guyon & Vapnik, Proc. 5th COLT, 1992

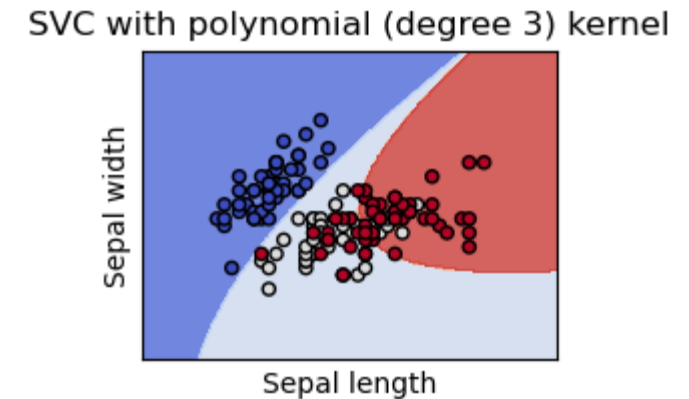
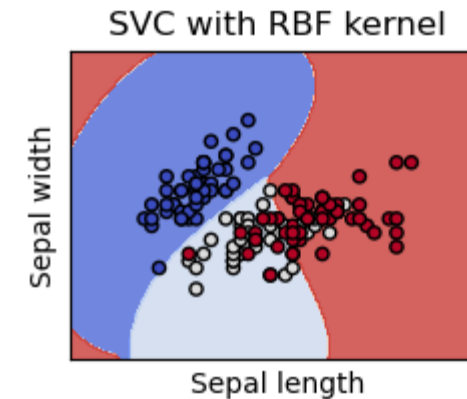
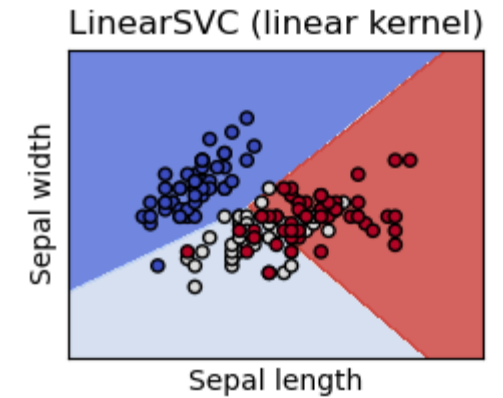
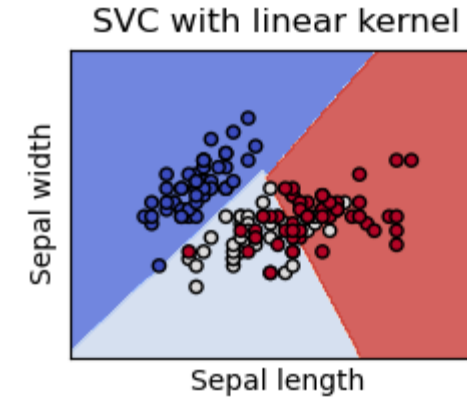


Non-Linear Support Vector Machines

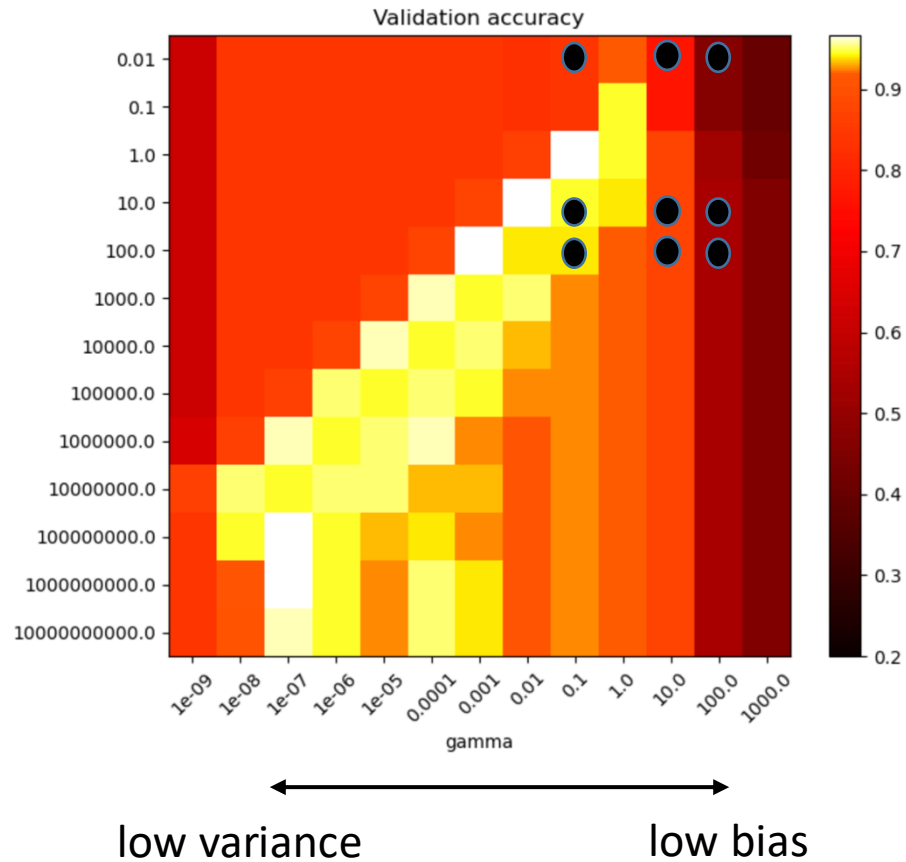
- **Kernel trick:** $\mathbf{x} \cdot \mathbf{y} \mapsto K(\mathbf{x}, \mathbf{y})$
 - $K(\mathbf{x}, \mathbf{y}) = K(\mathbf{y}, \mathbf{x}); \mathbf{c} K(\mathbf{x}_i, \mathbf{x}_j) \mathbf{c}^T \geq 0, \forall \mathbf{c} \in \mathbb{R}^n, \mathbf{x}_i, i = 1 \dots n \in \mathbb{R}^m$
 - **Linear:** $K(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y}$
 - **Polynomial:** $K(\mathbf{x}, \mathbf{y}) = (\gamma \mathbf{x} \cdot \mathbf{y} + r)^d$
 - **Radial:** $K(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}$
 - **Sigmoid:** $K(\mathbf{x}, \mathbf{y}) = \tanh(\gamma \mathbf{x} \cdot \mathbf{y} + r)$

Support vector machines (SVM)

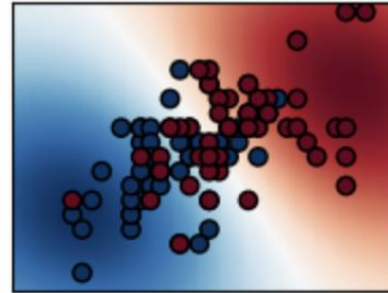
- Hyperparameters
 - Type of kernel (linear, polynomial, RBF; etc.)
 - Parameters of the kernel
 - Type of regularization
 - λ : regularization strength



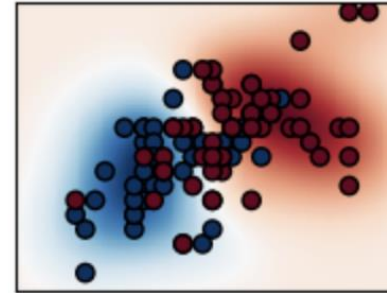
Radial Basis Kernel



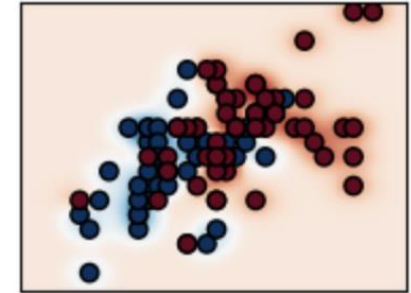
$\gamma=10^{-1}, C=10^{-2}$



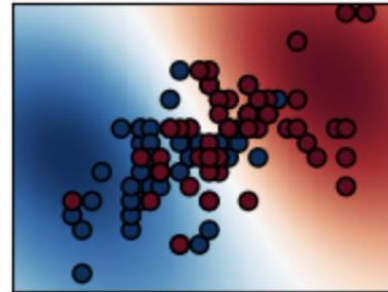
$\gamma=10^0, C=10^{-2}$



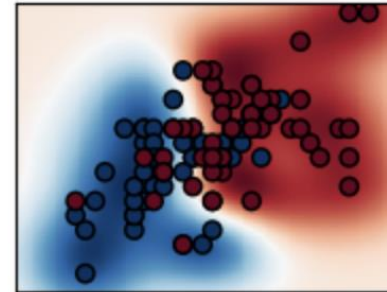
$\gamma=10^1, C=10^{-2}$



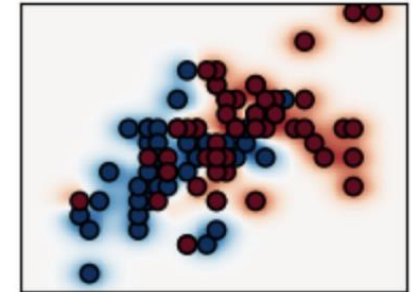
$\gamma=10^{-1}, C=10^0$



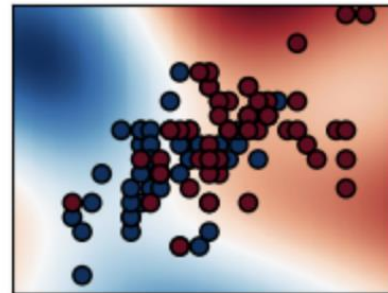
$\gamma=10^0, C=10^0$



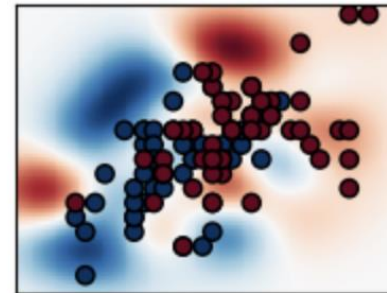
$\gamma=10^1, C=10^0$



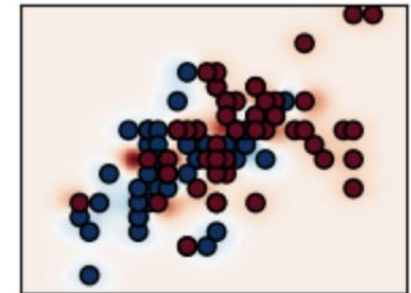
$\gamma=10^{-1}, C=10^2$



$\gamma=10^0, C=10^2$



$\gamma=10^1, C=10^2$



[sklearn SVC radial basis](#)

Kernels

- The beauty of SVM's or other kernel based approaches is that one does not need the data matrix $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2 \cdots \mathbf{x}_N)^T$ explicitly, but only the relation between the data points $\mathbf{x}_i$ given by the kernel $\mathbf{K}_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$
- This can be advantage when dealing with data items that cannot trivially be tranformed to vectors, but for which similarity kernels exist.
- For example, if $\mathbf{x}_i$ are sequences, then $K(\mathbf{x}_i, \mathbf{x}_j)$ can be defined by a sequence similarity score, without the need to turn sequences into vectors.

String Kernels (not included in sklearn.svm)

Example of string kernel from [Hilmarsson et al. BioRxiv, 2021](#)

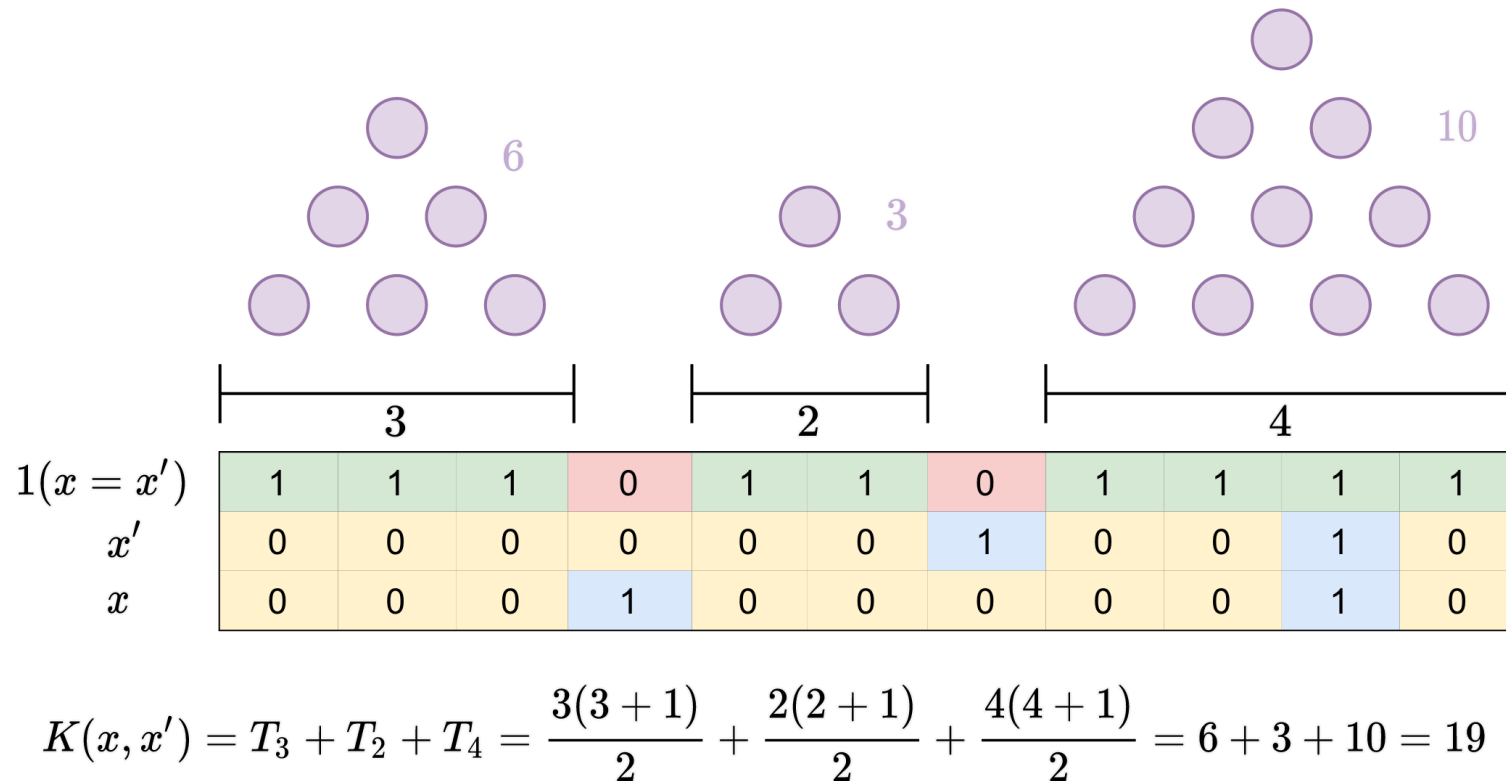
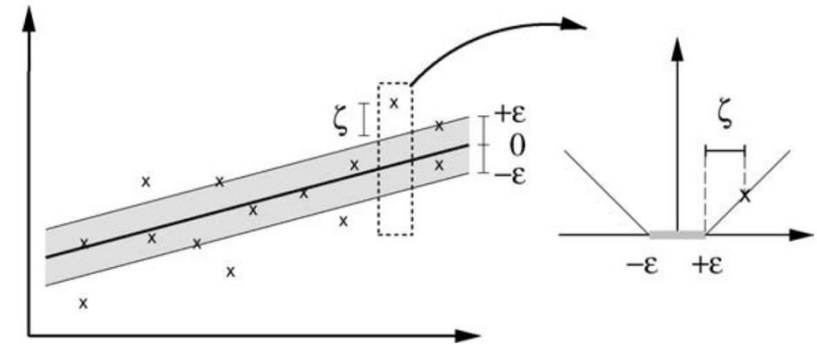


Figure 2. String kernel computation with triangular numbers visualized for two sequences, x and x' , of length eleven.

Support Vector Regression (SVR)

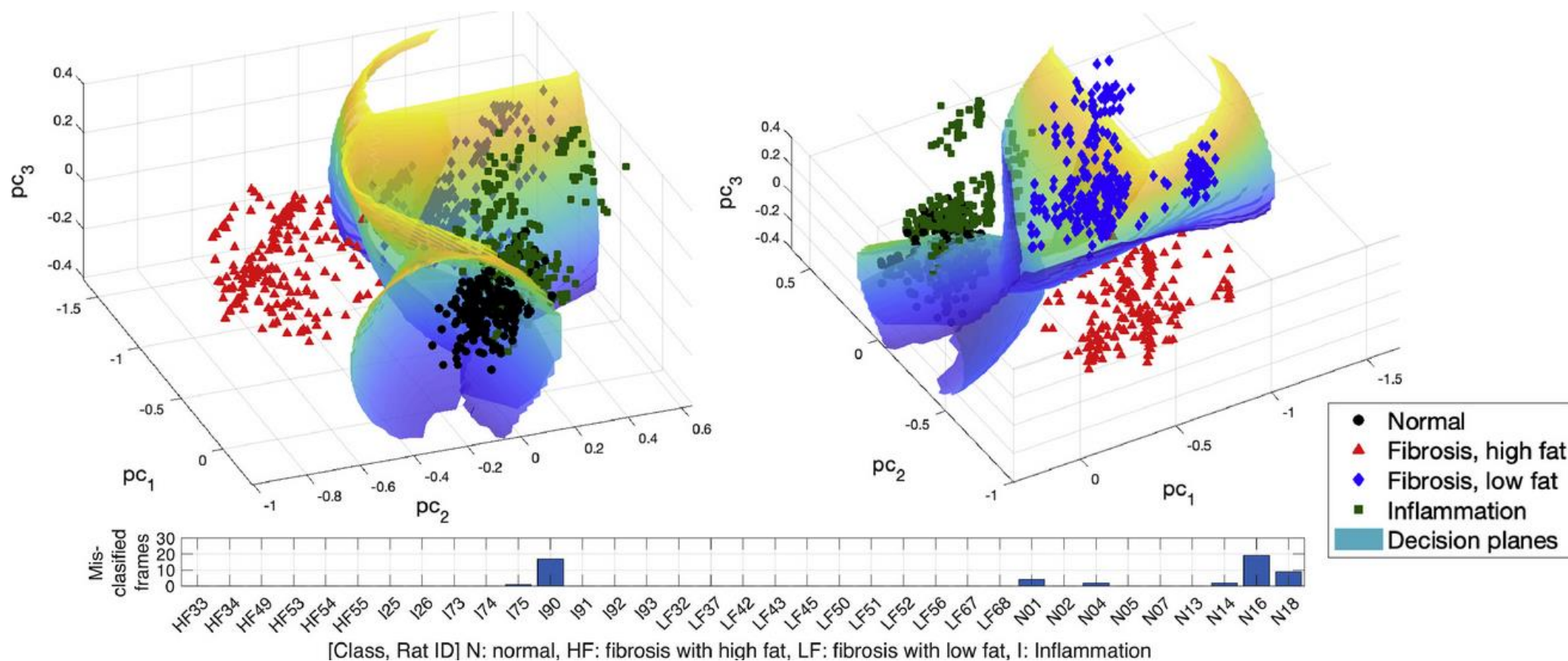
$$\min_{\mathbf{w}} \left\{ \|\mathbf{w}\|^2 + C \sum_{i=1}^N (\zeta_i + \zeta_i^*) \right\}$$

subject to
$$\begin{cases} y_i - \mathbf{w} \cdot \mathbf{x}_i - b \leq \varepsilon + \zeta_i \\ \mathbf{w} \cdot \mathbf{x}_i + b - y_i \leq \varepsilon + \zeta_i^*, & i = 1 \dots N \\ \zeta_i, \zeta_i^* \geq 0 \end{cases}$$



For dual form and kernel trick see [Smola & Schölkopf, Statistics and Computing, 2004](#)

Support vector machines (SVM)



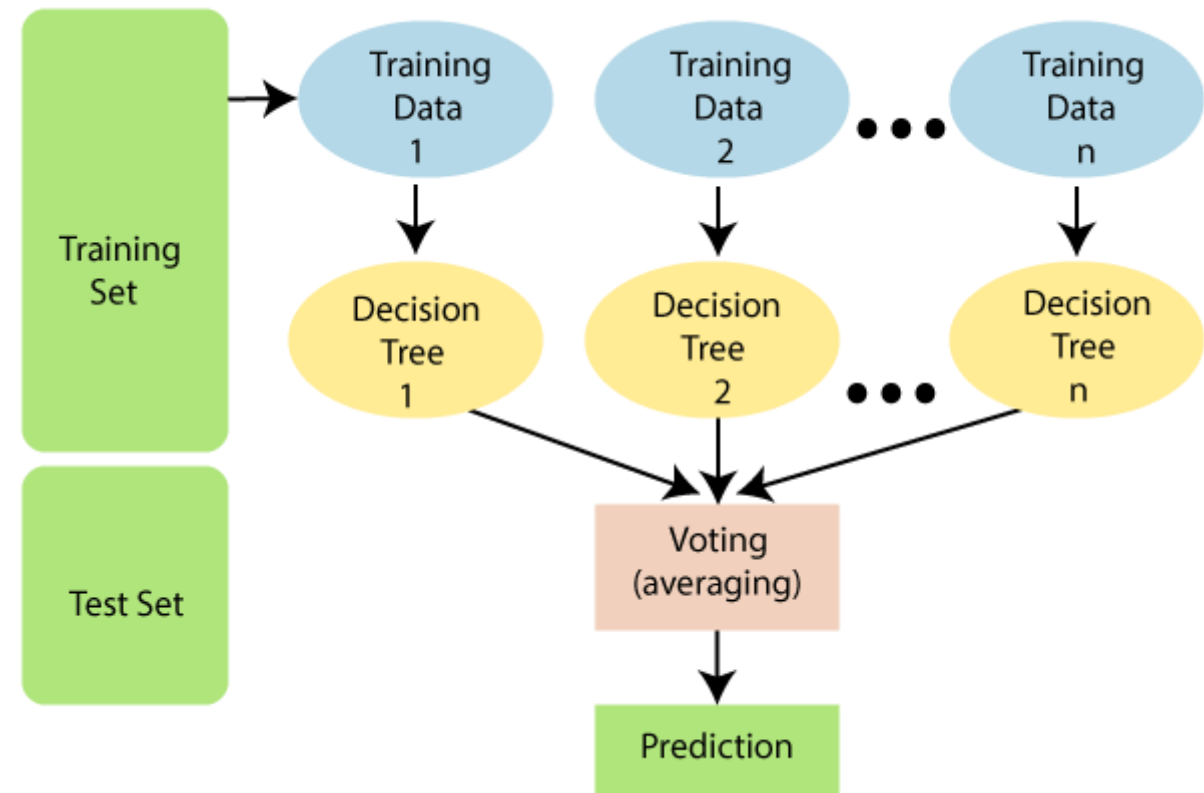
Baek et al. (2020) Ultrasound Med Biol 46(12):3379-3392.

Random forest

- popular machine learning algorithm which can be used for both Classification and Regression problems in ML
- based on the concept of **ensemble learning**, which is a process of *combining multiple classifiers to solve a complex problem and to improve the performance of the model*.
- Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.
- Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.
- The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

Random forest

- Bootstrap (select the features and the training samples at random, with replacement, to create the n training datasets)
- Hyperparameters:
 - Same as for the decision trees
 - Number of trees in the forest



Random forest

➤ [Proc Natl Acad Sci U S A](#). 2019 Sep 3;116(36):18119-18125. doi: 10.1073/pnas.1813645116.
Epub 2019 Aug 16.

Identification of the expressome by machine learning on omics data

Ryan C Sartor¹, Jaclyn Noshay², Nathan M Springer², Steven P Briggs³

Affiliations + expand

PMID: 31420517 PMCID: [PMC6731682](#) DOI: [10.1073/pnas.1813645116](#)

[Free PMC article](#)

Abstract

Accurate annotation of plant genomes remains complex due to the presence of many pseudogenes arising from whole-genome duplication-generated redundancy or the capture and movement of gene fragments by transposable elements. Machine learning on genome-wide epigenetic marks, informed by transcriptomic and proteomic training data, could be used to improve annotations through classification of all putative protein-coding genes as either constitutively silent or able to be expressed. Expressed genes were subclassified as able to express both mRNAs and proteins or only RNAs, and CG gene body methylation was associated only with the former subclass. More than 60,000 protein-coding genes have been annotated in the reference genome of maize inbred B73. About two-thirds of these genes are transcribed and are designated the filtered gene set (FGS). Classification of

Adaptive Boosting (AdaBoost)

- AdaBoost algorithms can be used for both classification and regression problem
- Combine multiple “weak classifiers” into a single “strong classifier”
- Based on the concept of **ensemble learning**: power of ensembling is such that we can still build powerful ensemble models even when the individual models in the ensembles are extremely simple.
- Boosting algorithms try to build a strong learner (predictive model) from the mistakes of several weaker models.
- Boosting basically tries to reduce the bias error which arises when models are not able to identify relevant trends in the data. This happens by evaluating the difference between the predicted value and the actual value.
- You start by creating a model from the training data. Then, you create a second model from the previous one by trying to reduce the errors from the previous model. Models are added sequentially, each correcting its predecessor, until the training data is predicted perfectly or the maximum number of models have been added.

Adaptive Boosting (AdaBoost)

- The weak learners in AdaBoost are decision trees with a single split, called **decision stumps**.
- Stumps alone are not a good way to make decisions. A full-grown tree combines the decisions from all variables to predict the target value. A stump, on the other hand, can only use one variable to make a decision
- Difference between Random Forest and AdaBoost:
 - AdaBoost uses a forest of stumps rather than trees.
 - AdaBoost stumps have different contributions to the prediction.
 - In AdaBoost, models are added sequentially, each correcting its predecessor.
- Each stump is made by taking the mistakes of the previous stumps into account by applying an importance weight on each sample. At the beginning of the algorithm, all the samples have the same weight but then after the first stump, the missclassified points will have a higher weight so that the next stumps really focuses on predicting them well.

Adaptive Boosting (AdaBoost)

1. A weak classifier (e.g. a decision stump) is made on top of the training data based on the weighted samples. Here, the weights of each sample indicate how important it is to be correctly classified. Initially, for the first stump, we give all the samples equal weights.
2. We create a decision stump for each variable and see how well each stump classifies samples to their target classes.
3. More weight is assigned to the incorrectly classified samples so that they're classified correctly in the next decision stump. Weight is also assigned to each classifier based on the accuracy of the classifier, which means high accuracy = high weight !
4. Reiterate from Step 2 until all the data points have been correctly classified, or the maximum iteration level has been reached.

Adaptive Boosting (AdaBoost)

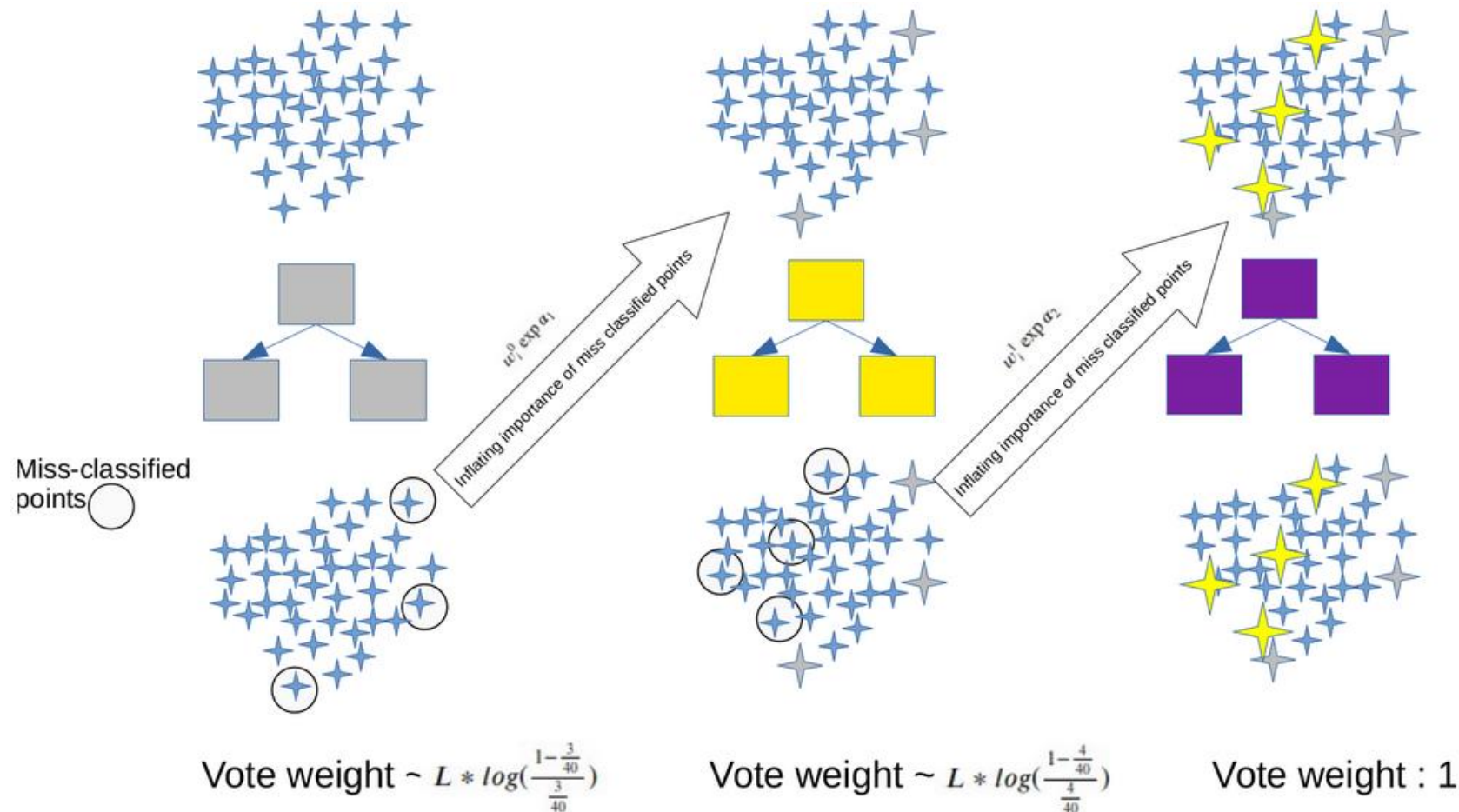
- Misclassification rate is calculated for each model:

$$\alpha = L * \ln\left(\frac{1-error}{error}\right) \text{ with } L = \text{learning rate and error} = \frac{\text{sum}(w(i) * \text{terror}(i))}{\text{sum}(w(i))}$$

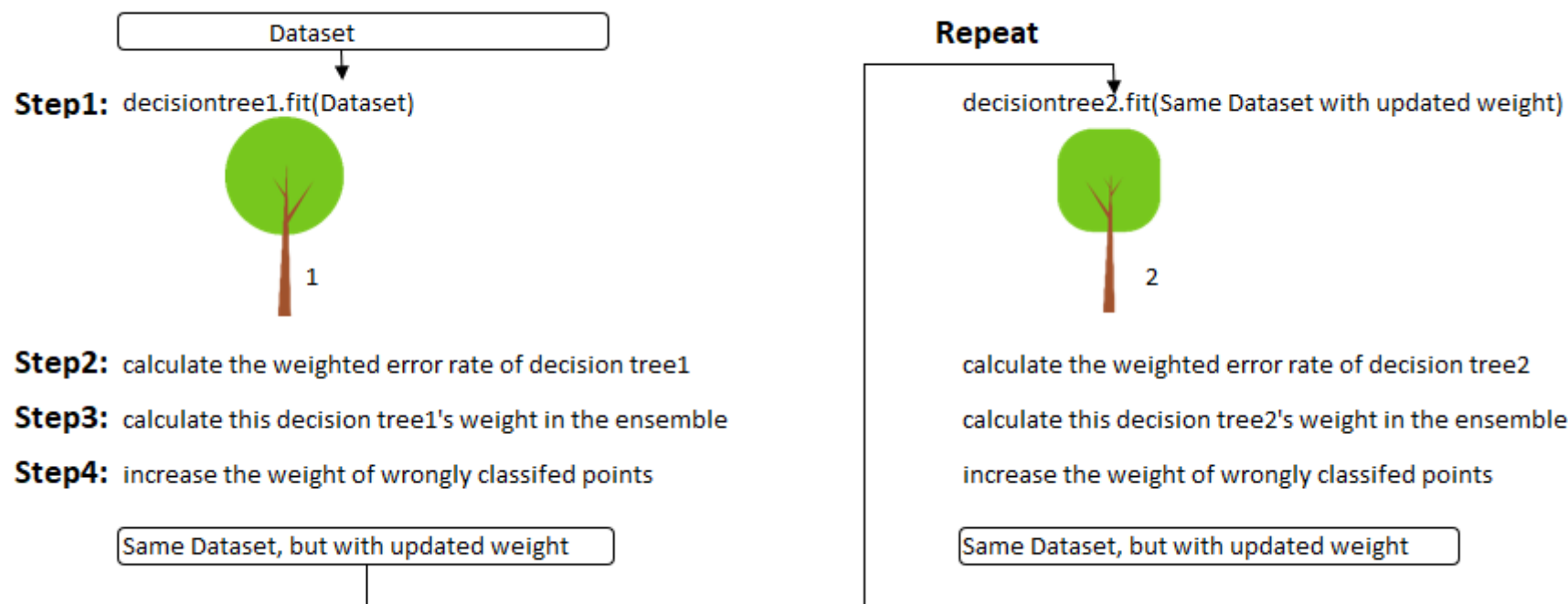
$$w_i^{t+1} = w_i^t \exp \alpha_{t+1} \text{ if } stump_{t+1} \text{ made a mistake for point } i$$

- Predictions are made by calculating the weighted average of the weak classifiers.
- Hyperparameters:
 - Number of stumps
 - Learning rate L: weight applied to each classifier at each boosting iteration
- Need high quality data !

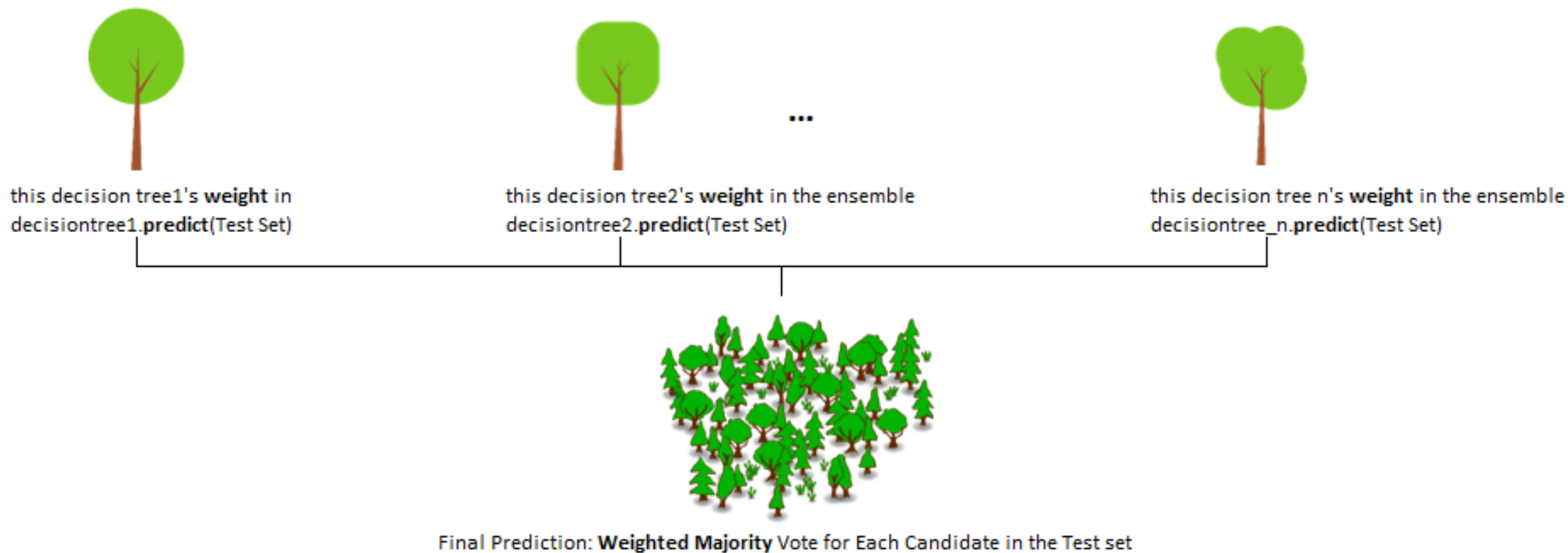
Adaptive Boosting (AdaBoost)



Adaptive Boosting (AdaBoost)



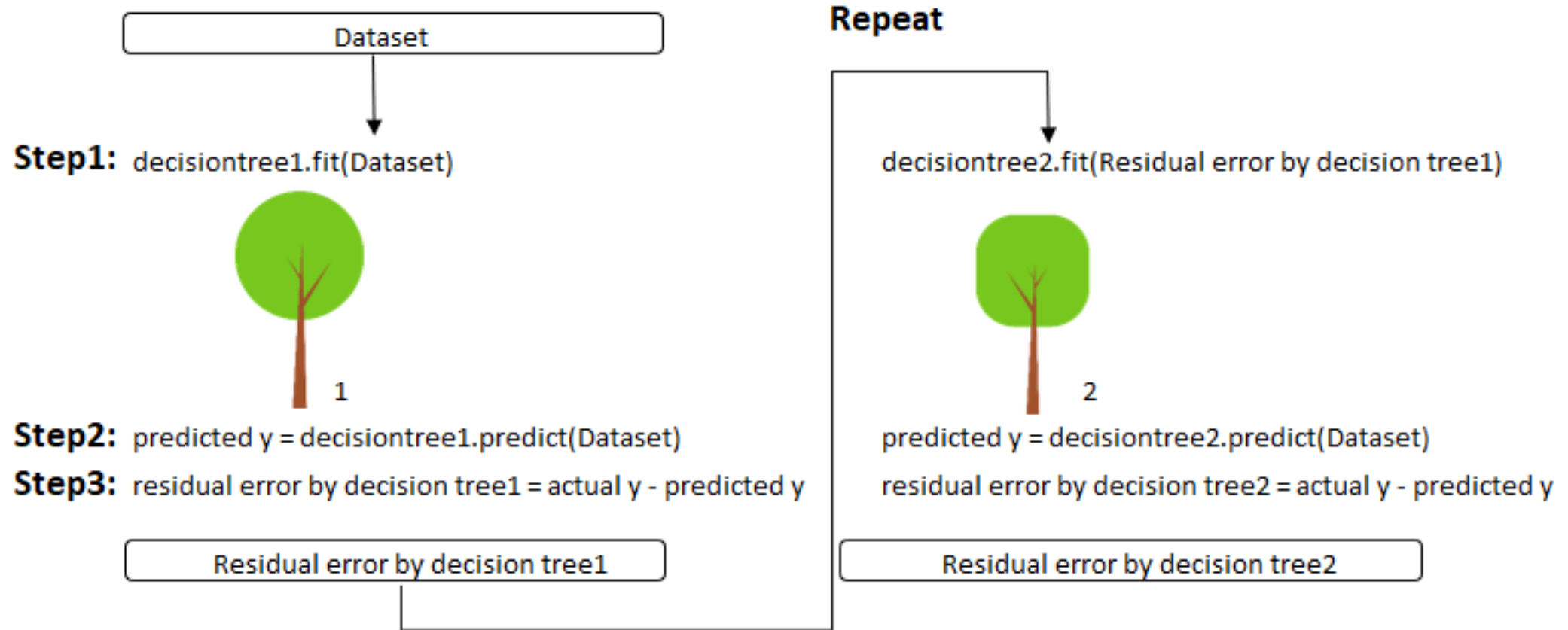
Adaptive Boosting (AdaBoost)



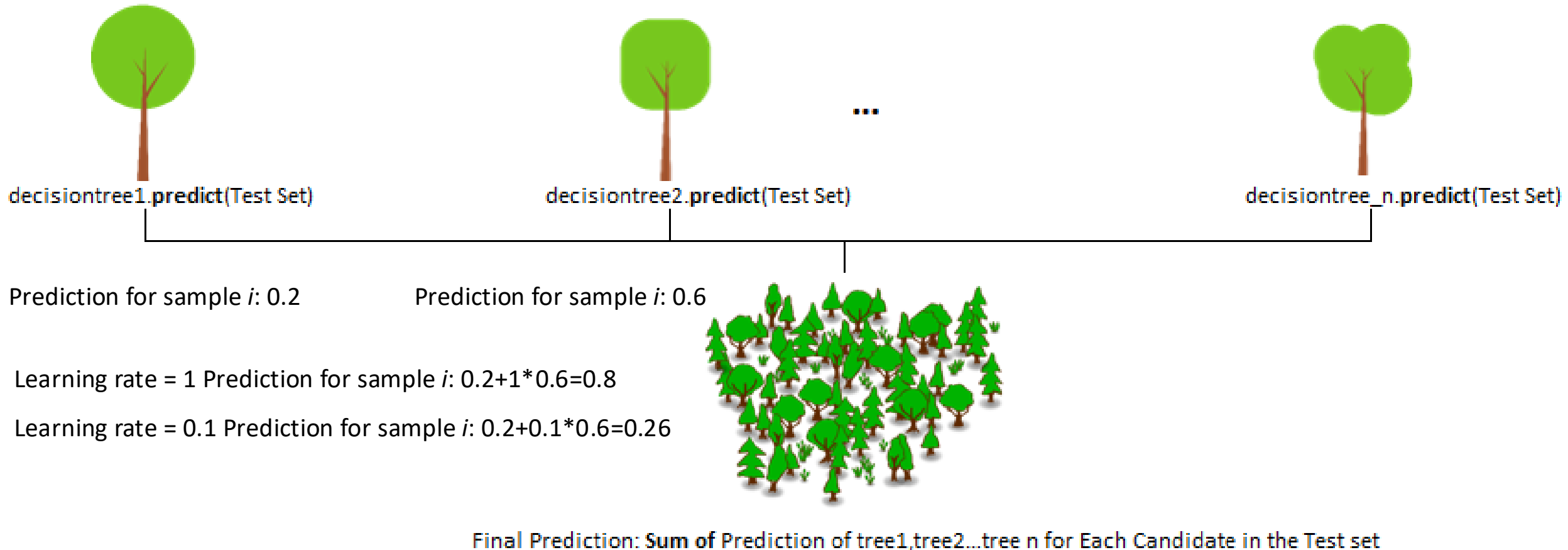
Gradient Boosting

- Another boosting model. Remember, boosting model's key is learning from the previous mistakes.
- Can be used for both classification and regression problem
- Gradient boosting involves three elements:
 1. A loss function to be optimized.
 2. A weak learner to make predictions: decision trees (with 4-to-8 levels)
 3. An additive model to add weak learners to minimize the loss function:
 - Trees are added one at a time, and existing trees in the model are not changed.
 - A gradient descent procedure is used to minimize the loss when adding trees.

Gradient Boosting



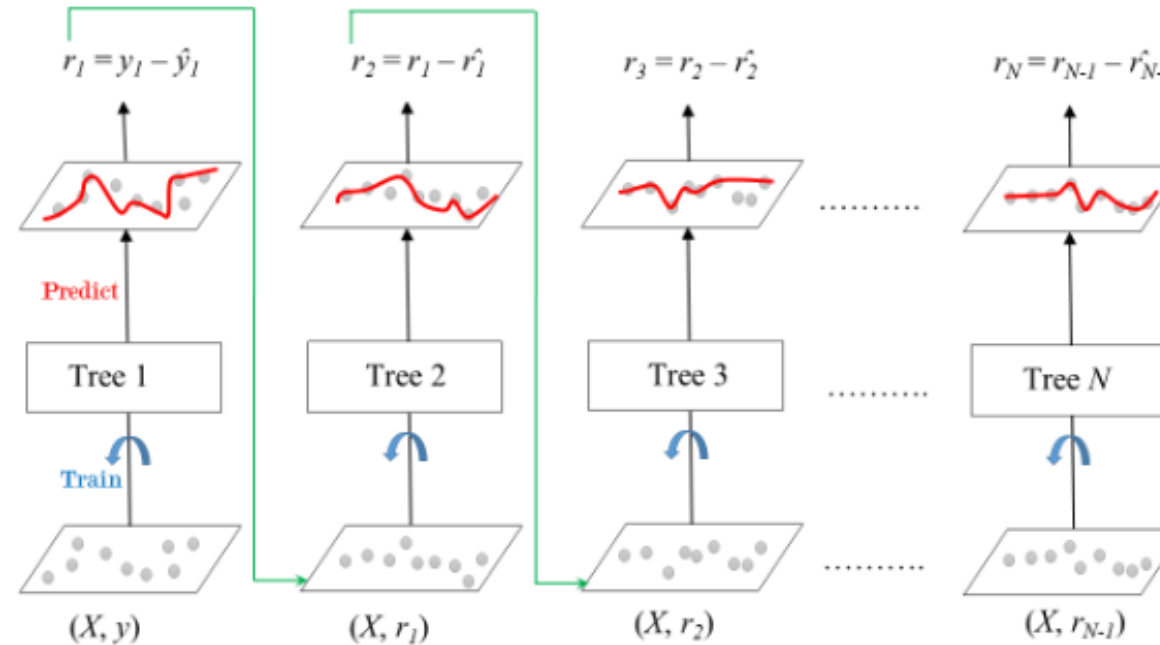
Gradient Boosting



Gradient Boosting

- The intuition behind gradient boosting algorithm is to repetitively leverage the patterns in residuals and strengthen a model with weak predictions and make it better
- Once we reach a stage that residuals do not have any pattern that could be modeled, we can stop modeling residuals (otherwise it might lead to overfitting). Algorithmically, we are minimizing our loss function, such that test loss reach its minima.

Gradient Boosting



- The ensemble consists of N trees. Tree1 is trained using the feature matrix X and the labels y . The predictions labelled $y1(\text{hat})$ are used to determine the training set residual errors $r1$. Tree2 is then trained using the feature matrix X and the residual errors $r1$ of Tree1 as labels. The predicted results $r1(\text{hat})$ are then used to determine the residual $r2$. The process is repeated until all the N trees forming the ensemble are trained.

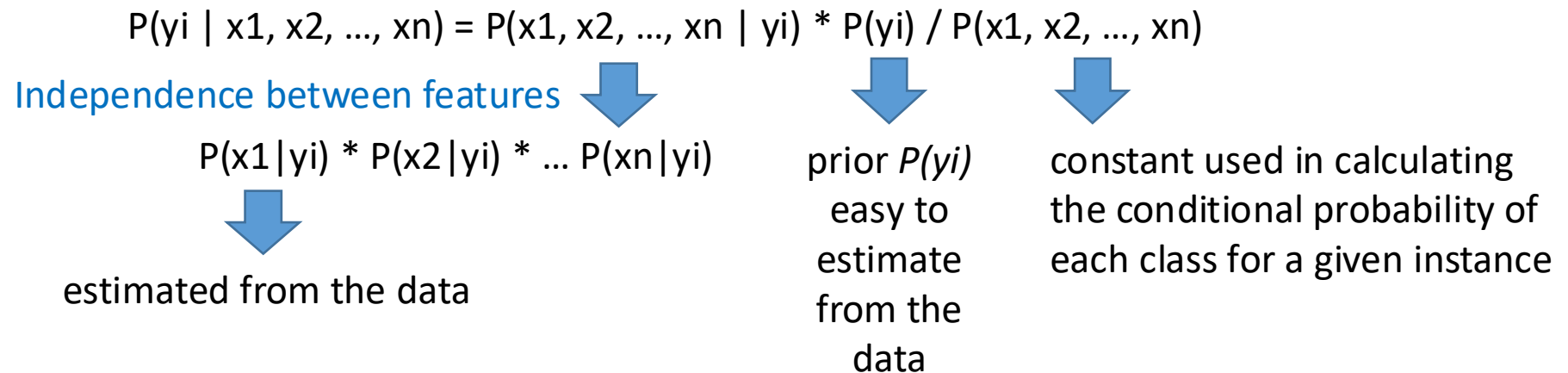
Gradient Boosting

- Gradient boosting is a greedy algorithm and can overfit a training dataset quickly.
- It can benefit from regularization methods that penalize various parts of the algorithm and generally improve the performance of the algorithm by reducing overfitting.
- Enhancements to basic gradient boosting:
 - Tree Constraints (small number of trees, short trees)
 - Shrinkage (learning rate)
 - Random sampling (bootstrap the training dataset)
- Hyperparameters:
 - Loss function: deviance (= in logistic regression) or exponential (= in AdaBoost)
 - Learning rate: shrinks the contribution of each tree
 - Number of trees, trees length, minimum number of samples required to split an internal node, minimum number of samples required to be at a leaf node
 - Number of features to consider when looking for the best split, etc.

Naive Bayes

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

- classification technique based on Bayes' Theorem with an assumption of independence among predictors (= features)
- a classification problem may have k class labels $y_1, y_2, \dots, y_k$ and n input variables, $X_1, X_2, \dots, X_n$. We can calculate the conditional probability for a class label with a given instance or set of input values for each column $x_1, x_2, \dots, x_n$ as follows: $P(y_i | x_1, x_2, \dots, x_n)$
- The conditional probability can then be calculated for each class label in the problem and the label with the highest probability can be returned as the most likely classification.



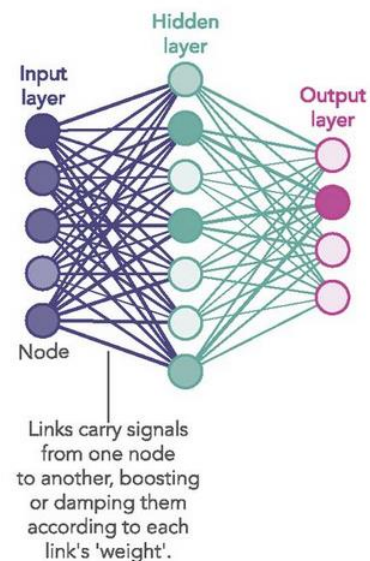
Naive Bayes

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

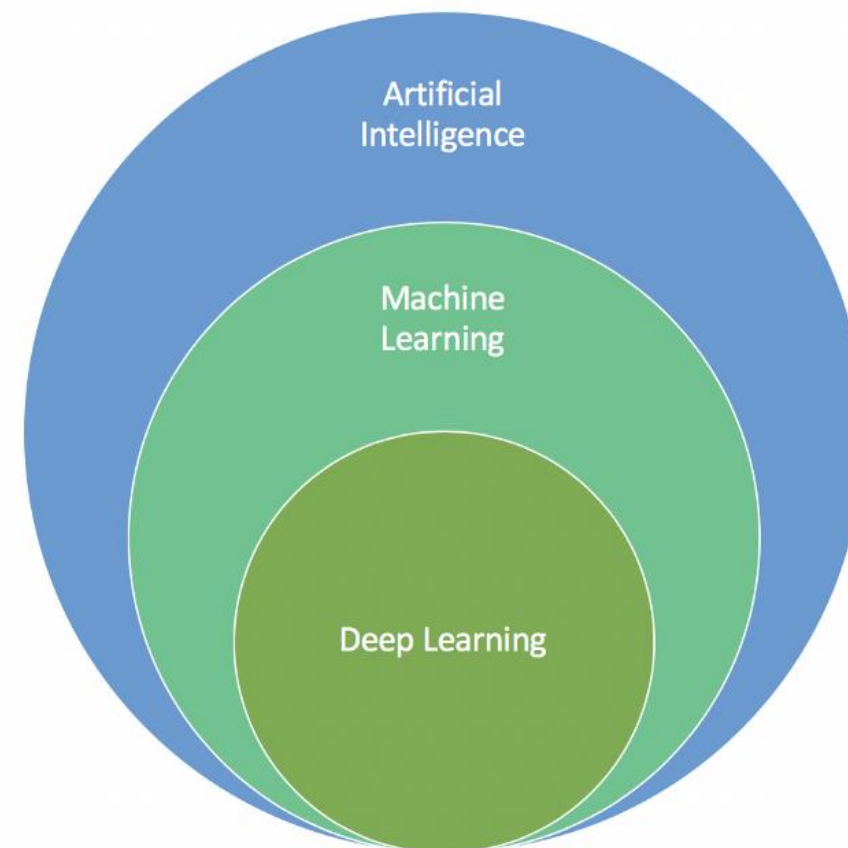
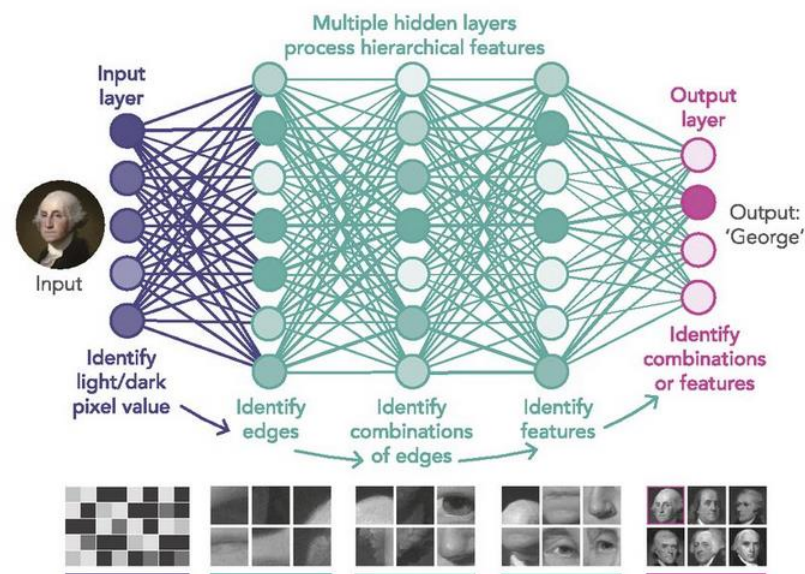
- Advantages:
 - very easy to build and particularly useful for very large data sets.
 - outperform even highly sophisticated classification methods.
 - a good choice when CPU and memory resources are a limiting factor.
 - a good method if something fast and easy that performs pretty well is needed.
- Main disadvantage: does not consider the interactions between features.
- Naive Bayes has been used in real-world applications such as:
 - mining housekeeping genes
 - genetic association studies
 - discovering Alzheimer genetic biomarkers from whole genome sequencing (WGS) data

Deep learning

1980S-ERA NEURAL NETWORK



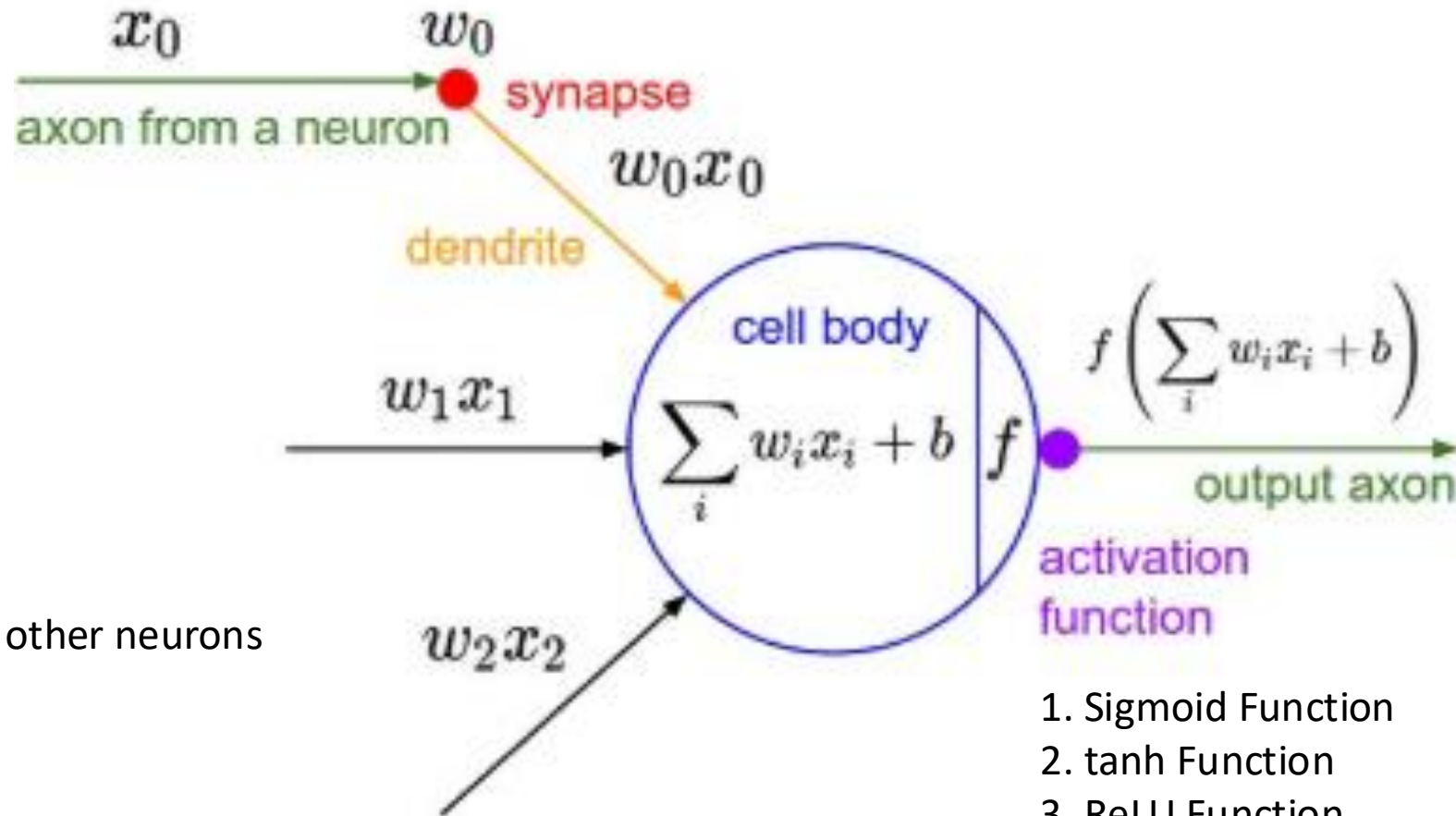
DEEP LEARNING NEURAL NETWORK



Deep learning

- Machine Learning consists of two steps:
 1. specify a template
 2. find the best parameters for that template.
- Deep Learning is simply a subset of the architectures (or templates) that employs “neural networks” which we can specify during Step 1.
- “Neural networks” (more specifically, artificial neural networks) are loosely based on how our human brain works, and the basic unit of a neural network is a neuron.
- At the basic level, a neuron does two things:
 1. Receive input from other neurons and combine them together
 2. Perform some kind of transformation to give the neuron’s output

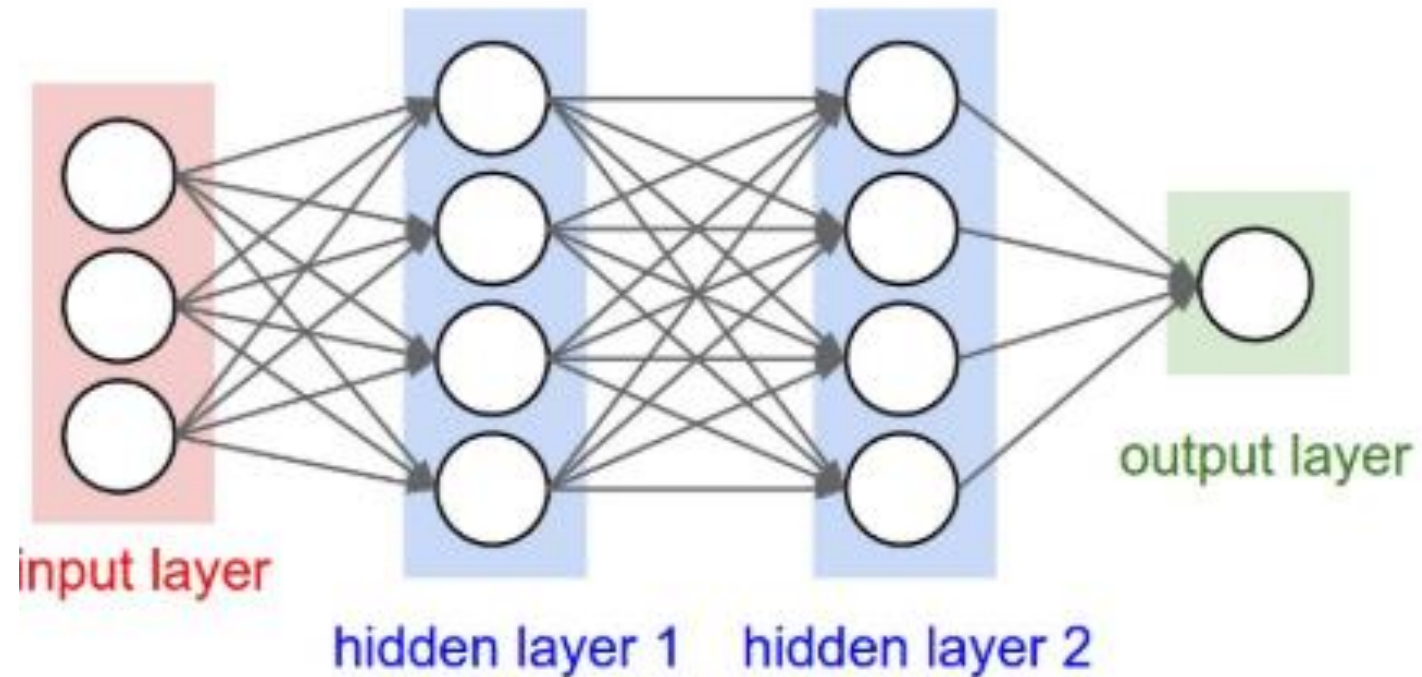
Neural networks



x_i : input from other neurons
 w_i : weights
 b : bias

1. Sigmoid Function
2. tanh Function
3. ReLU Function

Neural networks



A neural network is simply a complicated ‘template’ we specify which turns out to have the flexibility to model many complicated relationships between input and output.

Deep learning

- Model performance assessed with a loss function
- Minimizing the loss function using gradient descent (or other strategies)
- How to choose the best architecture of a neural network (hyperparameters: number of hidden layers, number of neurons in each hidden layer, type of activation functions etc.)
 1. Specify some hyper-parameters (the template)
 2. Train on the training dataset (filling in the parameters)
 3. Record the validation loss
 4. Repeat Steps 1 to 3 with a different set of hyper-parameters (many times)
 5. Pick the model with the lowest validation loss as our best model
 6. Run the chosen model on a separate test dataset to assess performance

Deep learning

Deep Learning in Omics Data Analysis and Precision Medicine

Jordi Martorell-Marugán ¹, Siham Tabik ², Yassir Benhammou ², Coral del Val ², Igor Zwir ²,
Francisco Herrera ², Pedro Carmona-Sáez ¹
Holger Husi ¹, editors.

In: Computational Biology [Internet]. Brisbane (AU): Codon Publications; 2019 Nov 21. Chapter 3.

Affiliations + expand

PMID: 31815397 Bookshelf ID: [NBK550335](#) DOI: [10.15586/computationalbiology.2019.ch3](#)

[Free Books & Documents](#)

Excerpt

The rise of omics techniques has resulted in an explosion of molecular data in modern biomedical research. Together with information from medical images and clinical data, the field of omics has driven the implementation of personalized medicine. Biomedical and omics datasets are complex and heterogeneous, and extracting meaningful knowledge from this vast amount of information is by far the most important challenge for bioinformatics and machine learning researchers. In this context, there is an increasing interest in the potential of deep learning (DL) methods to create predictive models and to identify complex patterns from these large datasets. This chapter provides an overview of the main applications of DL methods in biomedical research, with focus on omics data analysis and precision medicine applications. DL algorithms and the most popular architectures are introduced first. This is followed by a review of some of the main applications and problems approached by DL in omics data and medical image analysis. Finally, implementations for improving the diagnosis, treatment, and classification of complex diseases are discussed.

Deep learning

> [Sci Rep.](#) 2021 Apr 26;11(1):8992. doi: 10.1038/s41598-021-88172-0.

Machine learning and deep learning to predict mortality in patients with spontaneous coronary artery dissection

Chayakrit Krittanawong^{1 2}, Hafeez Ul Hassan Virk³, Anirudh Kumar⁴, Mehmet Aydar⁵, Zhen Wang^{6 7}, Matthew P Stewart^{8 9}, Jonathan L Halperin¹⁰

Affiliations + expand

PMID: 33903608 PMCID: [PMC8076284](#) DOI: [10.1038/s41598-021-88172-0](#)

[Free PMC article](#)

Abstract

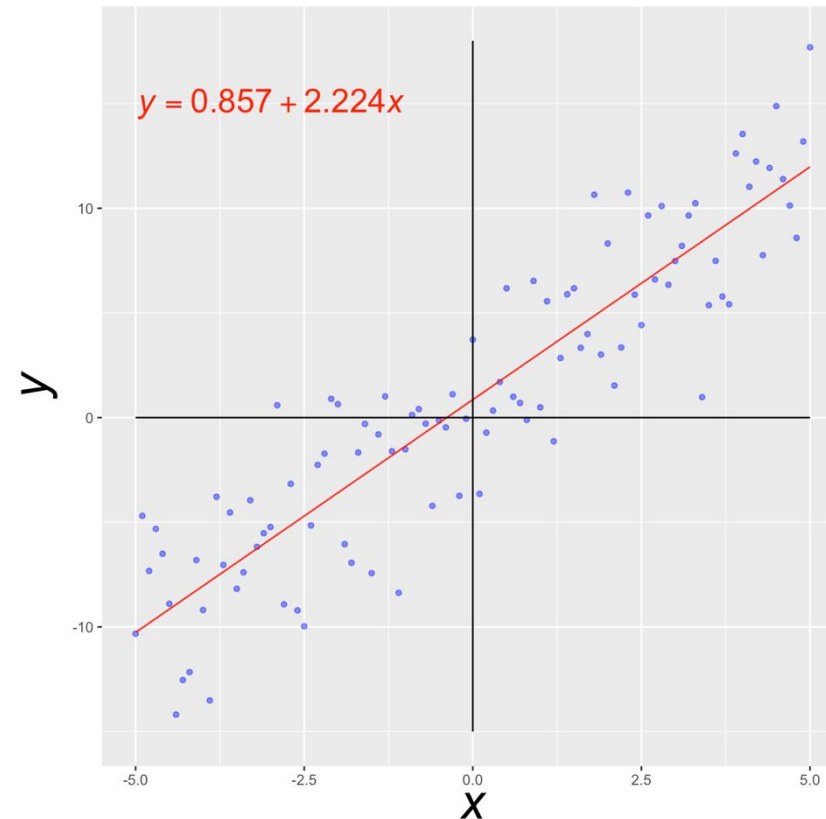
Machine learning (ML) and deep learning (DL) can successfully predict high prevalence events in very large databases (big data), but the value of this methodology for risk prediction in smaller cohorts with uncommon diseases and infrequent events is uncertain. The clinical course of spontaneous coronary artery dissection (SCAD) is variable, and no reliable methods are available to predict mortality. Based on the hypothesis that machine learning (ML) and deep learning (DL) techniques could enhance the identification of patients at risk, we applied a deep neural network to information available in electronic health records (EHR) to predict in-hospital mortality in patients with SCAD. We extracted patient data from the EHR of an extensive urban health system and applied several ML and DL models using candidate clinical variables potentially associated with mortality. We partitioned the data into training and evaluation sets with cross-validation. We estimated model performance based on the area under the receiver-operator characteristics curve (AUC) and balanced accuracy. As sensitivity analyses, we examined

Miscellaneous

- **NumPy** stands for Numerical Python
 - It is a Python library used for working with arrays.
 - It also has functions for working in domain of linear algebra, Fourier transform, and matrices.
 - NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.
- **Matplotlib** is a comprehensive library for creating static, animated, and interactive visualizations in Python.
- **Seaborn** is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.
- **Pandas** is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.
- **Scikit-learn** (formerly scikits.learn and also known as sklearn) is a free software machine learning library for the Python programming language.[3] It features various classification, regression and clustering algorithms including support vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

Linear regression

- Regression: a method of modelling a target value y_i as an affine combination of a feature vector $\mathbf{x}_i$: $y_i \sim w_0 + \mathbf{w}^T \mathbf{x}_i$
- 1D Example :



← Best fit straight line
to the data points

$$y \sim w_0 + w_1 x$$

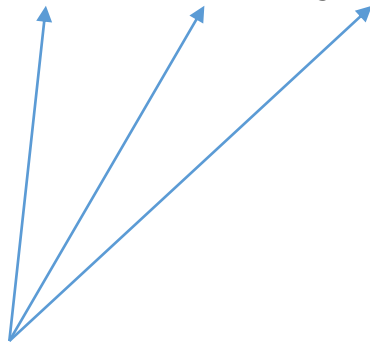
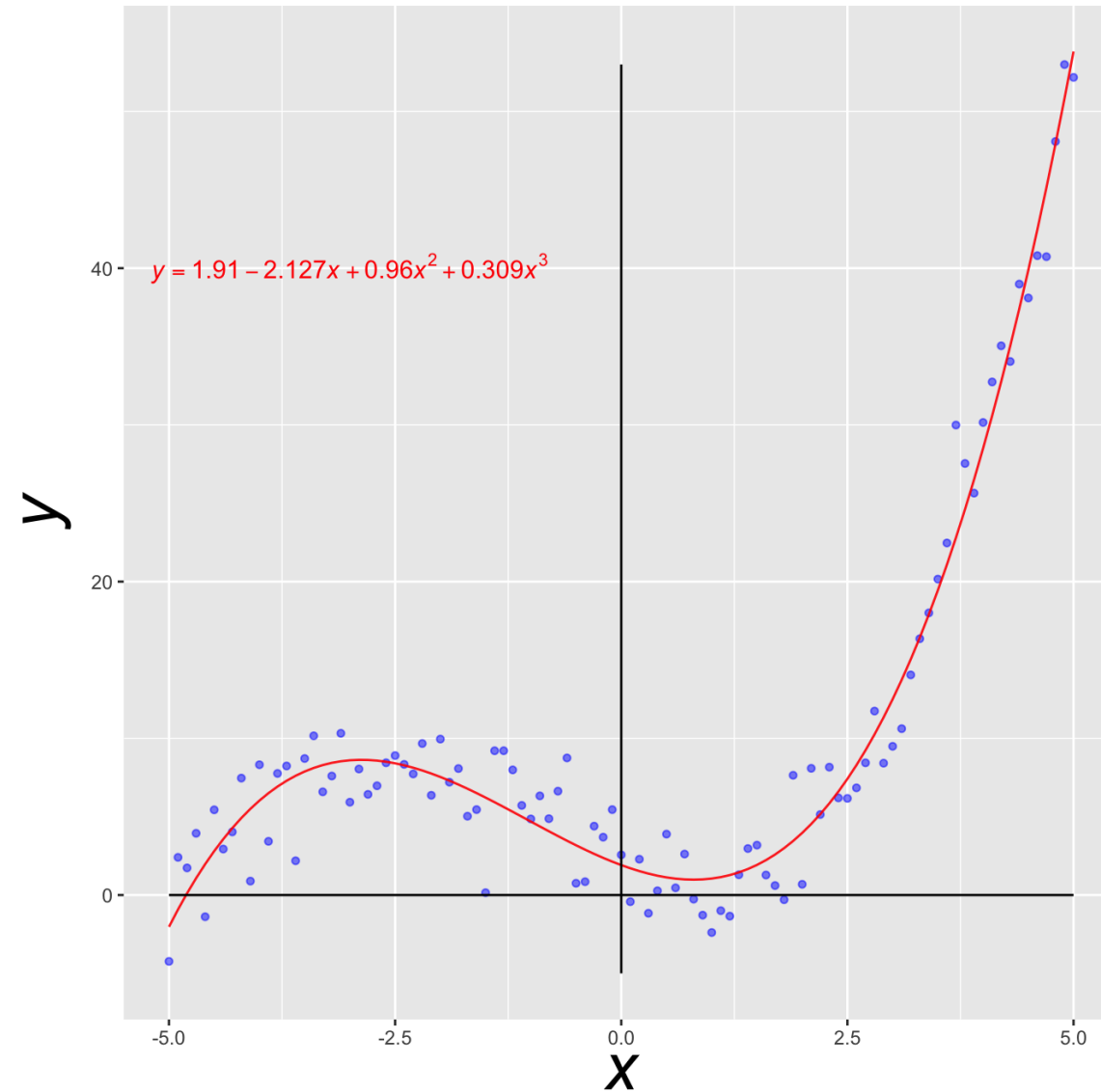
How to find the best
values for w_0 and w_1 ?

Linear regression

- Polynomial regression example :

- $$y \sim w_0 + w_1 \cdot x + w_2 \cdot x^2 + w_3 \cdot x^3$$

Features

Some Notations:

Constant term

D features

Data matrix: $\mathbf{X} = \begin{pmatrix} \textcolor{teal}{1} & x_{11} & \cdots & x_{1D} \\ \vdots & \vdots & \ddots & \vdots \\ \textcolor{teal}{1} & x_{N1} & \cdots & x_{ND} \end{pmatrix}$

N items

Outcome: $\mathbf{y} = (y_1, \dots, y_N)^T$

Outcome is a vector of real numbers

Feature vectors: $\mathbf{X} = (\boldsymbol{\varphi}_0 \quad \cdots \quad \boldsymbol{\varphi}_D)$

$\boldsymbol{\varphi}_i = \text{columns of } \mathbf{X}$

Least Square Linear Regression

- Coefficients for linear regression: $\mathbf{w} = (w_0, w_1, \dots, w_M)^T$
- Write $\mathbf{y}$ as a linear combination of feature values:

$$\mathbf{y} \sim \mathbf{X} \cdot \mathbf{w} = \begin{pmatrix} w_0 + \sum_{i=1}^D X_{1i} w_i \\ \vdots \\ w_0 + \sum_{i=1}^D X_{Ni} w_i \end{pmatrix} = \sum_{i=0}^D w_i \boldsymbol{\varphi}_i$$

- Residual sum-of-squares (RSS) error:

$$\text{RSS}(\mathbf{w}) = \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2 = (\mathbf{X} \cdot \mathbf{w} - \mathbf{y})^T (\mathbf{X} \cdot \mathbf{w} - \mathbf{y})$$

Least Square Linear Regression

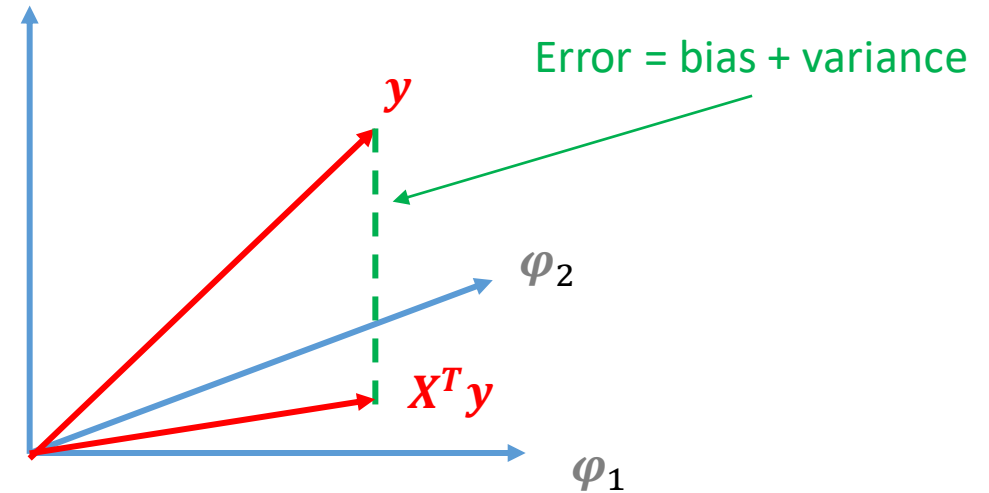
- Finding $\mathbf{w}$ which minimizes RSS: $\mathbf{w} = \underset{\mathbf{w}}{\operatorname{argmin}} \operatorname{RSS}(\mathbf{w})$
- RSS is quadratic in $\mathbf{w}$. The minimum of RSS is where $\nabla_{\mathbf{w}} \operatorname{RSS}(\mathbf{w}) = 0$
- $\nabla_{\mathbf{w}} \operatorname{RSS}(\mathbf{w}) = 2\mathbf{X}^T (\mathbf{X} \cdot \mathbf{w} - \mathbf{y}) = \mathbf{0} \Rightarrow \mathbf{X}^T \mathbf{X} \cdot \mathbf{w} = \mathbf{X}^T \mathbf{y}$

$$\Rightarrow \mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ is called the pseudo inverse

Pseudo inverse

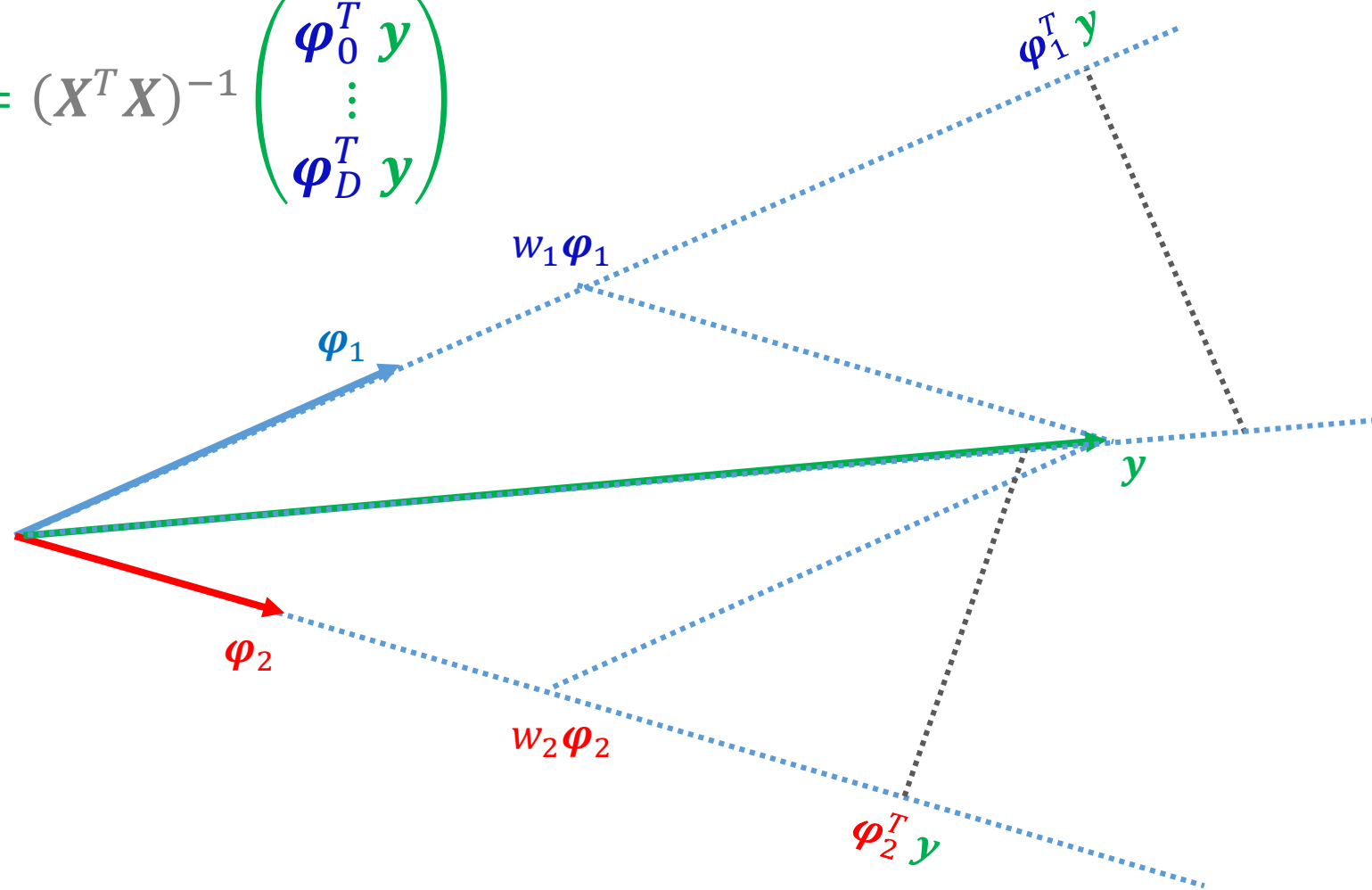
- $$\mathbf{w} = \underbrace{(\mathbf{X}^T \mathbf{X})^{-1}}_{\text{Calculate coordinates}} \underbrace{\mathbf{X}^T \mathbf{y}}_{\text{Projection to feature space}}$$



- $$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} \sum_{i=1}^N X_{i0} y_i \\ \vdots \\ \sum_{i=1}^N X_{iD} y_i \end{pmatrix} = \begin{pmatrix} \boldsymbol{\varphi}_0^T \mathbf{y} \\ \vdots \\ \boldsymbol{\varphi}_D^T \mathbf{y} \end{pmatrix}$$

Geometric interpretation of $(X^T X)^{-1} X^T y$

$$w = (X^T X)^{-1} X^T y = (X^T X)^{-1} \begin{pmatrix} \varphi_0^T y \\ \vdots \\ \varphi_D^T y \end{pmatrix}$$

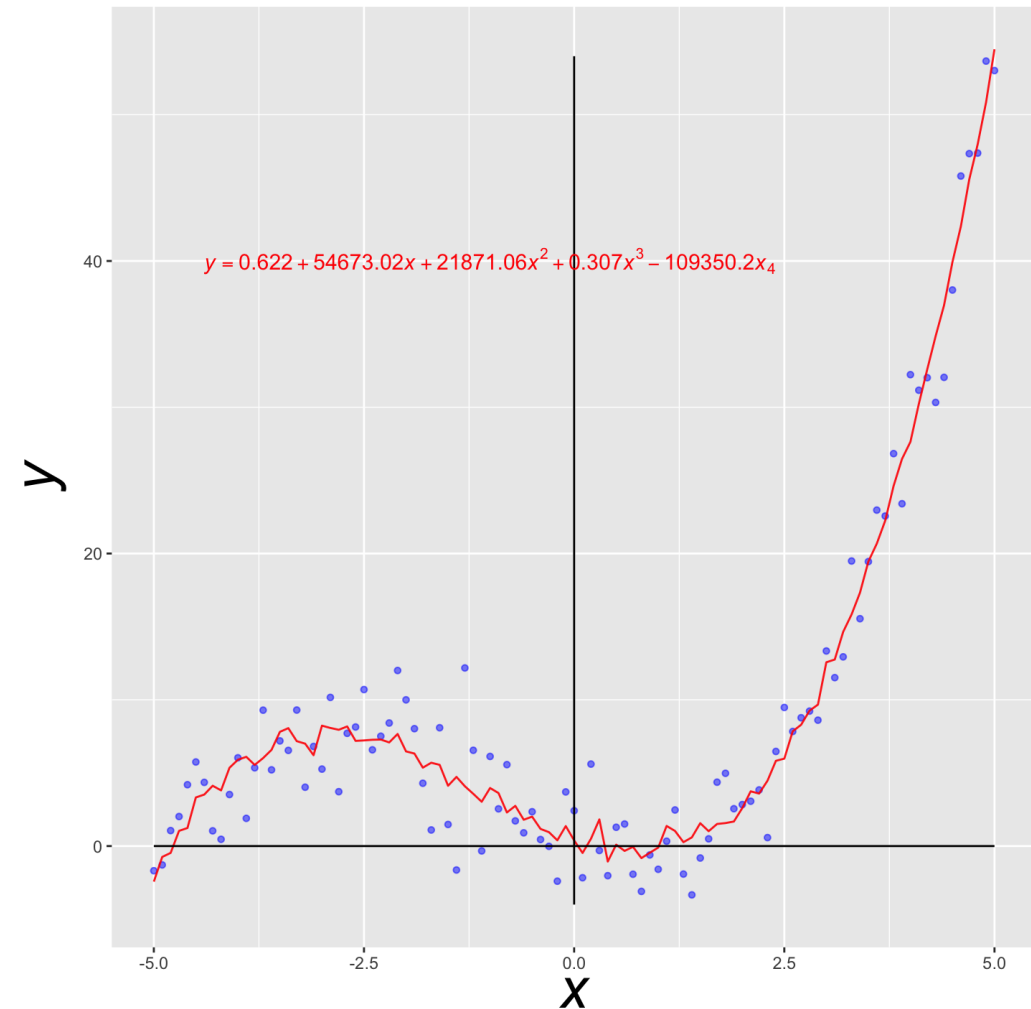
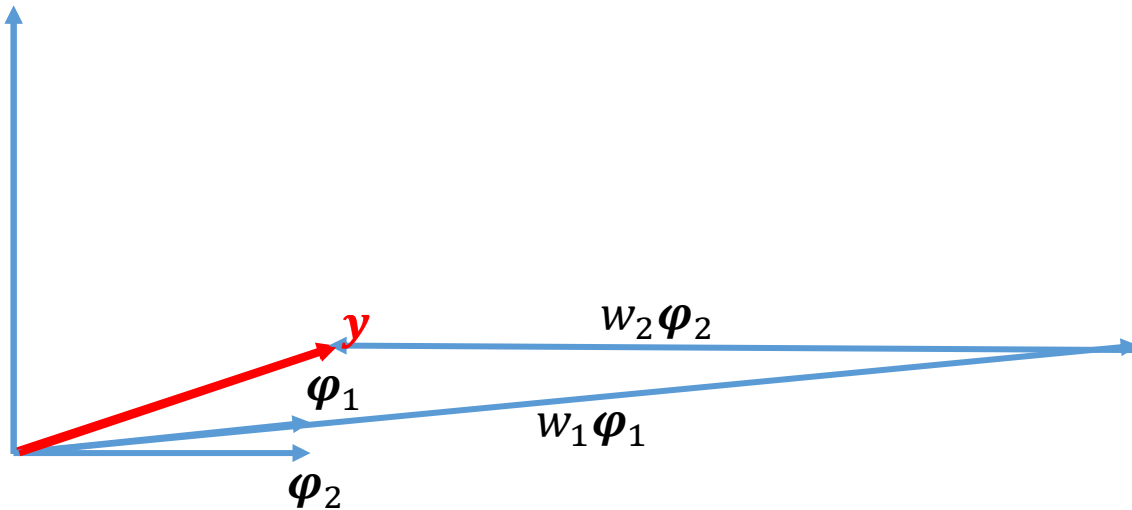


Pseudo inverse

- If $\mathbf{X}$ is whitened ($\boldsymbol{\varphi}_i^T \boldsymbol{\varphi}_j = \delta_{ij}$ and $\sigma_i^2 = 1$) then $(\mathbf{X}^T \mathbf{X})^{-1} = \mathbb{I}$.
- If $\mathbf{X}$ is whitened, then the weights w_i are comparable, i.e. they can be interpreted as the importance of feature i for the regression. Often $\boldsymbol{\varphi}_i^T \boldsymbol{\varphi}_j \neq \delta_{ij}$, but we can normalize $\|\boldsymbol{\varphi}_i\|^2 = 1$. If the feature vectors are not strongly correlated, the weights w_i can still be interpreted as approximate feature importances.
- If the feature vectors are strongly correlated, then $\mathbf{X}^T \mathbf{X}$ can become close to singular and the calculation of $(\mathbf{X}^T \mathbf{X})^{-1}$ unstable leading to large fluctuations in the values of $\mathbf{w}$.

Linear regression

- Polynomial regression example :
 - $y \sim w_0 + w_1 \cdot x + w_2 \cdot x^2 + w_3 \cdot x^3 + w_4 \cdot x_4$
 - $x_4 = 0.5x + 0.2x^2 + \varepsilon$



Make linear regression more stable

- Orthogonal feature vectors
 - $X_{i1} \perp X_{j1}, i \neq j$
- Force $\mathbf{w}$ to be small by introducing a penalty for large $\mathbf{w}$:
 - Ridge: $\text{RSS}(\mathbf{w}) = \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2 + \alpha \|\mathbf{w}\|^2; \alpha \geq 0$
 - Produces stable solutions even if features are linearly dependent.
 - Favors solutions with small $w_i \neq 0$
 - Lasso: $\text{RSS}(\mathbf{w}) = \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2 + \alpha \sum_{i=0}^D |w_i|; \alpha \geq 0$
 - Like ridge, but with many $w_i = 0$. Useful for feature selection.
 - Elastic net: $\text{RSS}(\mathbf{w}) = \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2 + \alpha \left(\gamma \sum_{i=0}^D |w_i| + \frac{(1-\gamma)}{2} \|\mathbf{w}\|^2 \right)$
 $\alpha \geq 0; 0 \leq \gamma \leq 1$
 - Mix between ridge and lasso. Lasso can produce solutions that are too sparse, whereas ridge solutions may contain many small values.
 - Values for α or γ are determined by CV

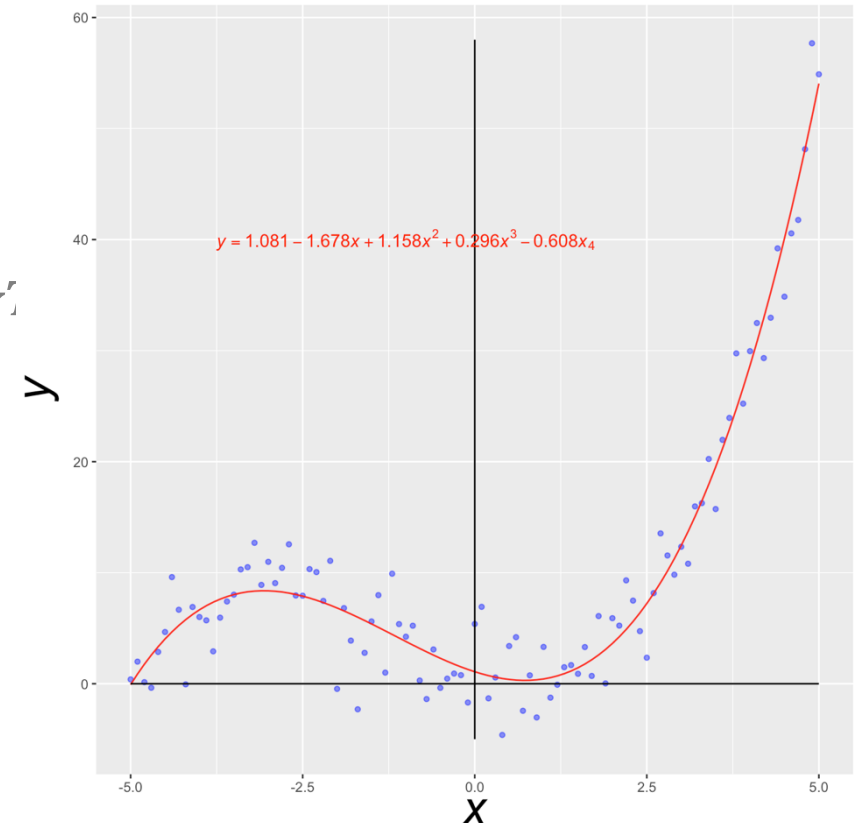
Ridge regression

- $\text{RSS}(\mathbf{w}) = \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2 + \alpha \|\mathbf{w}\|^2$; $\alpha \geq 0$: still quadratic in $\mathbf{w}$
- $\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w}) = 2\mathbf{X}^T (\mathbf{X} \cdot \mathbf{w} - \mathbf{y}) + 2\alpha \mathbf{w} = \mathbf{0}$

$$\Rightarrow (\mathbf{X}^T \mathbf{X} + \alpha \mathbb{I}) \cdot \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

$$\Rightarrow \mathbf{w} = (\mathbf{X}^T \mathbf{X} + \alpha \mathbb{I})^{-1} \mathbf{X}^T \mathbf{y}$$

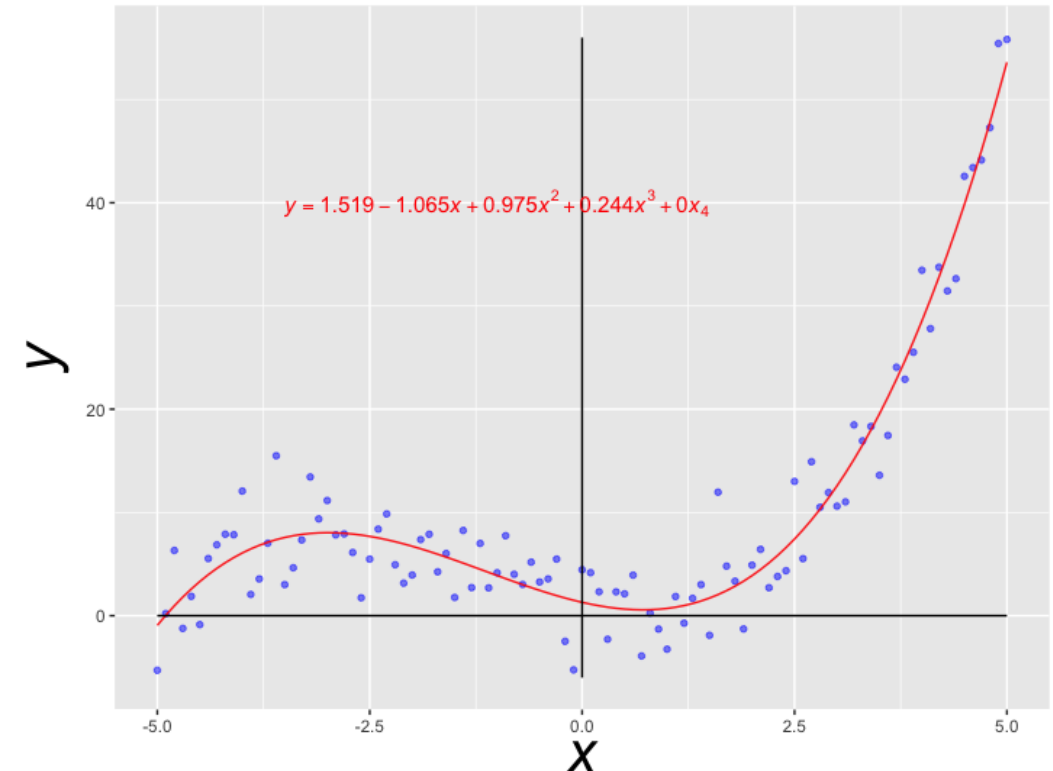
[sklearn.linear_model.Ridge](#)



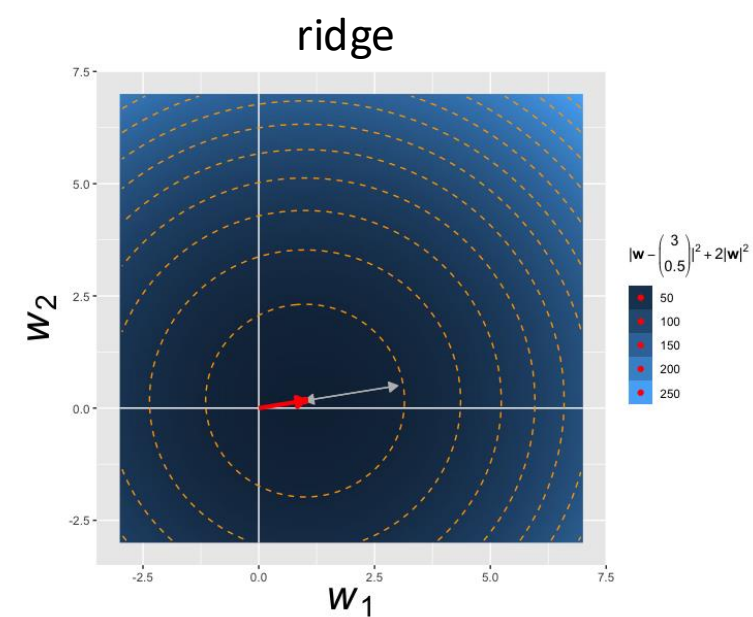
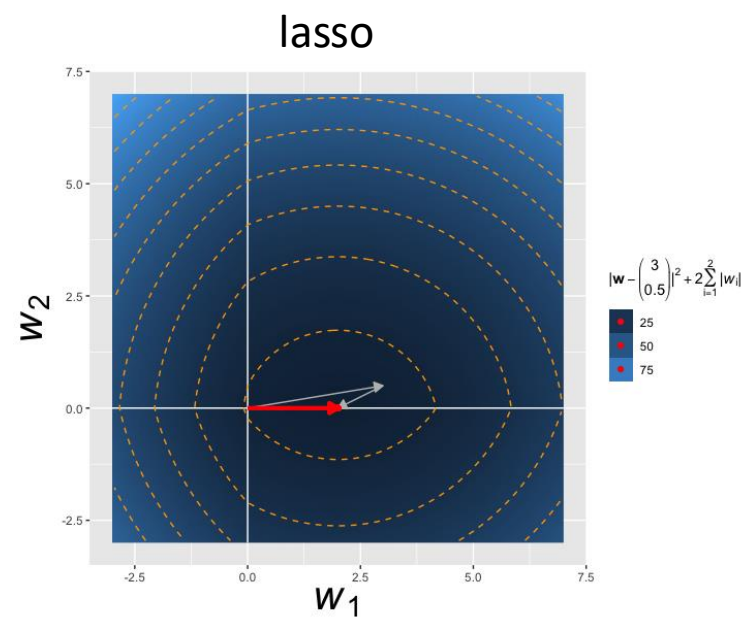
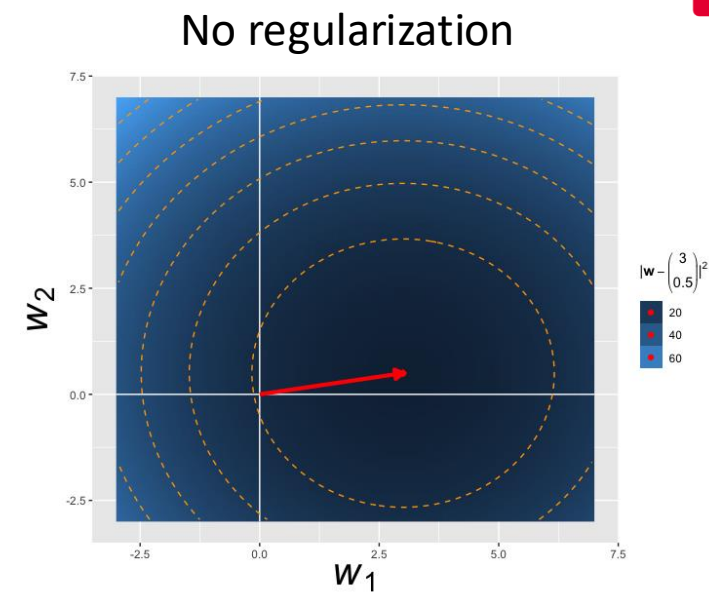
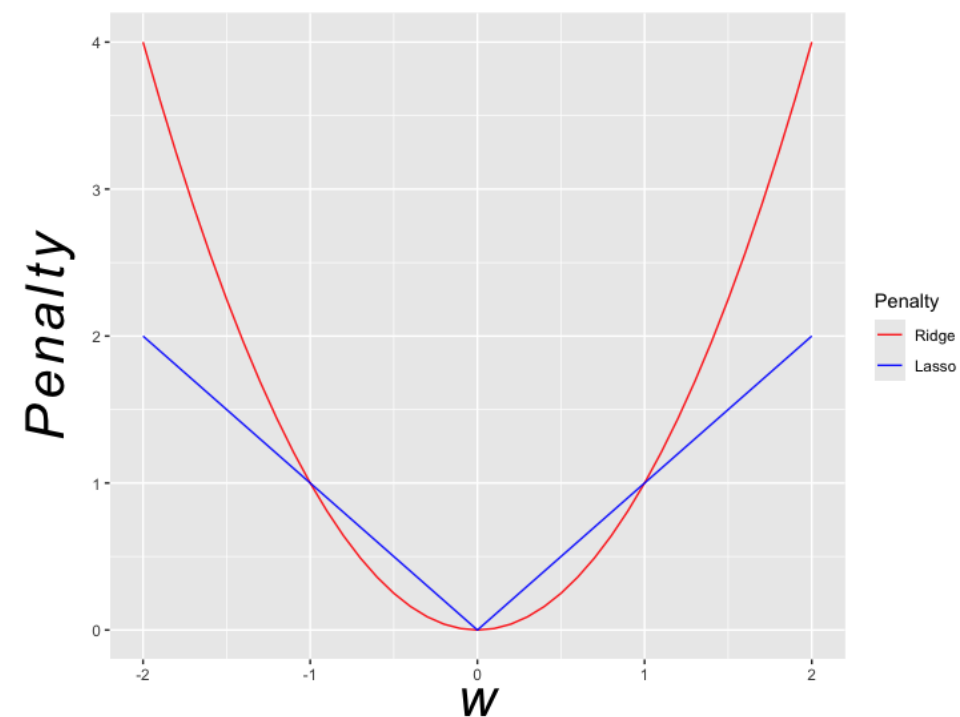
Lasso regression

- $\text{RSS}(\mathbf{w}) = \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2 + \alpha \sum_{i=0}^D |\mathbf{w}_i|$; $\alpha \geq 0$: not quadratic in $\mathbf{w}$
- $\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w}) = 2\mathbf{X}^T (\mathbf{X} \cdot \mathbf{w} - \mathbf{y}) + \alpha \text{sign}(\mathbf{w}) = \mathbf{0}$:
 - has no direct solution
 - gradient descend

[sklearn.linear_model.Lasso](#)



Lasso and ridge regularization



Elastic net linear regression

- $\text{RSS}(\mathbf{w}) = \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2 + \alpha \left(\gamma \sum_{i=0}^D |\mathbf{w}_i| + \frac{(1-\gamma)}{2} \|\mathbf{w}\|^2 \right)$
 $\alpha \geq 0; 0 \leq \gamma \leq 1$: not quadratic in $\mathbf{w}$
- $\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w}) = 2\mathbf{X}^T (\mathbf{X} \cdot \mathbf{w} - \mathbf{y}) + \alpha \gamma \text{sign}(\mathbf{w}) + \alpha(1 - \gamma)\mathbf{w} = \mathbf{0}$:
 - has no direct solution
 - gradient descent

[sklearn.linear_model.SGDRegressor](#)

